

Frankenstein Transformer: Biblioteca Unificada de Codificador-Decodificador, CLI y Notas de Diseño Basadas en Investigación

Erick F. Merino M.
Este trabajo no está afiliado
`erickfmm@gmail.com`

Febrero 2026

Resumen

Frankenstein Transformer presenta un conjunto de herramientas unificado y basado en configuración para la experimentación sistemática con arquitecturas modernas de transformers, abarcando diecisiete variantes de mezcladores de secuencia y veintitrés familias de optimizadores. El sistema admite tanto el modelado de lenguaje enmascarado (MLM) tipo codificador como la predicción autorregresiva (AR) del siguiente token tipo decodificador mediante una configuración flexible de clase de modelo y modo, con flujos de trabajo de ajuste fino especializados para ambas arquitecturas. Las contribuciones de investigación son tres: (i) un contrato de configuración estricto basado en esquemas que permite la experimentación reproducible a través de diversos mecanismos de atención, incluyendo atención softmax estándar, atención sigmoide, redes retentivas, modelos selectivos de espacio de estados, transformers de profundidad continua, enrutamiento adaptativo de profundidad (Mixture-of-Depths) [58], aumento de memoria condicional (Engram) [9], patrones de atención dispersa y mecanismos con compuerta; (ii) un marco integral de enrutamiento de optimizadores que admite métodos de reducción de varianza (MARS, Adan, AdEMAMix), variantes eficientes en memoria (Adafactor, GaLore, Lion), enfoques sin programación de tasa de aprendizaje, preconditionadores de segundo orden (Shampoo, SOAP, Sophia) y optimizadores de bajo rango de la familia APOLLO (Apollo, Apollo-Mini, Q-Apollo) [56]; y (iii) flujos de trabajo de extremo a extremo que abarcan el despliegue cuantizado mediante empaquetado ternario de pesos y el entrenamiento de incrustaciones de oraciones inspirado en SBERT, respaldados por una pila de aseguramiento de calidad ampliada con amplia cobertura de pruebas unitarias, validación de ejemplos YAML y ejecución de integración continua. El conjunto de herramientas implementa una interfaz de configuración basada en web que proporciona renderizado de formularios impulsado por esquemas con documentación en línea y validación en tiempo real. Este documento de referencia técnica incluye diagramas arquitectónicos, visualizaciones de flujo de ejecución, tablas de decisión y apéndices completos que sintetizan la literatura sobre arquitecturas de transformers, mecanismos de atención dispersa, arquitecturas con compuerta y familias de optimizadores.

Índice

1. Introducción	6
1.1. Motivación y Planteamiento del Problema	6
1.2. Contribuciones	7
1.3. Guía de Lectura	8
1.4. Constructor de Configuración Basado en Web	8

2. Trabajo Relacionado	9
2.1. Arquitecturas de Mezcladores de Secuencia	9
2.1.1. Líneas Base de Atención Densa	9
2.1.2. Arquitecturas Recurrentes y Retentivas	10
2.1.3. Mecanismos de Atención Dispersa	10
2.1.4. Mecanismos de Atención con Compuerta	10
2.2. Algoritmos de Optimización	10
3. Diseño y Arquitectura del Sistema	11
3.1. Arquitectura Centrada en Configuración	11
3.2. Alcance del Esquema y Reglas de Validación	13
3.3. Inventario Completo de Características del Modelo	13
3.4. Tipos de Tarea de Entrenamiento	14
3.5. Inventario Completo de Características de Entrenamiento	15
3.6. Contrato de Prefijo del Optimizador (Completo)	16
3.7. Seguridad de Entrenamiento y Semántica de Ejecución	17
3.8. Variantes de Normalización: RMSNorm, Tanh Dinámico y Erf Dinámico	17
3.9. RMSNorm	18
3.10. Tanh Dinámico (DyT)	18
3.11. Erf Dinámico (Derf)	18
3.12. Implicaciones del Esquema	18
4. Taxonomía de Arquitectura e Implementación	19
4.1. Familias de Atención y Mezcladores de Secuencias	19
4.2. Atención Estándar	19
4.3. Atención Sigmoide	20
4.4. Formulación Retentiva	20
4.5. SSM Selectivo (Mamba)	20
4.6. Actualizaciones Continuas estilo ODE	20
4.7. Memoria en Tiempo de Prueba (Titans)	20
4.8. Extensiones Dispersas y con Compuerta en el Código Base Actual	20
4.9. Bloques de Atención Dispersa Implementados (Detallado)	21
4.10. Bloques de Atención con Compuerta Implementados (Detallado)	22
5. Familias de Optimizadores y Dinámicas de Entrenamiento	24
6. Cuantización y Despliegue	25
6.1. Cuantización Ternaria	25
6.2. Cuantización de Activaciones	25
6.3. Estimaciones de Tamaño	26
7. Tareas Posteriores de SBERT	26
8. Tablas Resumen	26
8.1. Resumen de Atención y Mezcladores de Secuencia	26
8.2. Resumen de Optimizadores	27
9. Discusión	28
9.1. Compromisos de Diseño Basado en Esquemas	28
9.2. Cobertura Arquitectónica y Vacíos	28
9.3. Fragmentación del Panorama de Optimizadores	29
9.4. Consideraciones de Despliegue y Producción	29
9.5. Desafíos de Integración y Extensibilidad	30

10. Conclusión	30
10.1. Limitaciones y Direcciones Futuras	31
Bibliografía	32
A. Anexo A: Familias de Optimizadores	35
A.1. Comparación de Memoria y Complejidad	35
A.2. La Evolución de la Optimización en Redes Neuronales	36
A.3. Línea Base Estándar y Optimizadores Adaptativos	36
A.4. Momentum Avanzado y Reducción de Varianza (2024–2025)	39
A.5. Optimizadores de Lote Grande, Eficientes en Memoria y Sin Tasa de Aprendizaje	43
A.6. Optimizadores de Segundo Orden, Geométricos y de Ortogonalidad	49
B. Anexo B: Transformadores Densos, Recurrentes y Aumentados con Memoria	54
B.1. Atención Densa de Referencia: Estándar y Sigmoide	54
B.1.1. Atención Softmax Estándar	54
B.1.2. Atención Sigmoide	55
B.2. Arquitecturas Recurrentes y Retentivas	55
B.2.1. Redes Retentivas (RetNet)	55
B.2.2. Mamba: Modelos de Espacio de Estado Selectivos	56
B.3. Transformers de Profundidad Continua: Integración EDO	57
B.3.1. Transformer EDO	57
B.4. Memoria en Tiempo de Prueba: Titans	57
B.5. Comparación y Síntesis Arquitectónica	58
C. Annex C: Comprehensive Sparse Attention Mechanisms	59
C.1. Resumen Ejecutivo	59
C.2. Sparse Transformer: Patrones Estrideados y Fijos Factorizados	59
C.2.1. Formulación Matemática	59
C.2.2. Pseudocódigo Algorítmico	60
C.2.3. Características Clave	60
C.3. Longformer: Ventana Deslizante, Dilatación y Tokens Globales	60
C.3.1. Formulación Matemática	60
C.3.2. Pseudocódigo Algorítmico	61
C.3.3. Características Clave	61
C.4. BigBird: Grafo Disperso Aleatorio, Local y Global	61
C.4.1. Formulación Matemática	61
C.4.2. Garantía Teórica	62
C.4.3. Pseudocódigo Algorítmico	62
C.4.4. Características Clave	62
C.5. Atención SparseK: Selección Diferencial Top- k	63
C.5.1. Formulación Matemática	63
C.5.2. Pseudocódigo Algorítmico	63
C.5.3. Características Clave	63
C.6. NSA (Native Sparse Attention): Ramas Jerárquicas Alineadas con el Hardware	64
C.6.1. Formulación Matemática	64
C.6.2. Pseudocódigo Algorítmico	65
C.6.3. Características Clave	65
C.7. FASA: Atención Dispersa Consciente de la Frecuencia	65
C.7.1. Formulación Matemática	65
C.7.2. Pseudocódigo Algorítmico	66
C.7.3. Características Clave	66

C.8. SpargeAttn: Filtrado de Dos Etapas a Nivel de Bloques	67
C.8.1. Formulación Matemática	67
C.8.2. Pseudocódigo Algorítmico	68
C.8.3. Características Clave	68
C.9. Resumen Comparativo	68
C.10. Espacio de Diseño y Criterios de Selección	69
D. Annex D: Gated Attention Families—Complete Literature Analysis	69
D.1. Resumen Ejecutivo: Compuertas para el Control de Memoria	69
D.2. 1. Atención Lineal con Compuerta (GLA)	70
D.2.1. Fundamento Matemático	70
D.2.2. Entrenamiento Eficiente en Hardware	71
D.2.3. Fortalezas y Limitaciones	71
D.3. 2. DeltaNet: Atención Lineal con Corrección de Errores	72
D.3.1. Fundamento Matemático	72
D.3.2. Entrenamiento Paralelo Eficiente	72
D.3.3. Fortalezas y Limitaciones	73
D.4. 3. Gated DeltaNet: Síntesis de Compuerta y Corrección de Errores	73
D.4.1. Fundamento Matemático	73
D.4.2. Rendimiento Empírico y Compensaciones	74
D.5. 4. HGRN2: Compuerta Jerárquica con Expansión de Producto Externo	74
D.5.1. Fundamento Matemático	74
D.5.2. Escalado y Resultados Empíricos	75
D.6. 5. Forgetting Transformer (FoX): Compuerta en el Espacio de Logits Softmax	75
D.6.1. Fundamento Matemático	75
D.6.2. Integración con FlashAttention	76
D.6.3. Fortalezas y Limitaciones	76
D.7. 6. Atención con Compuerta (Post-SDPA Sigmoide)	76
D.7.1. Fundamento Matemático	76
D.7.2. Hallazgos Clave	77
D.8. Análisis Comparativo: Taxonomía de Mecanismos con Compuerta	77
D.9. Principio Unificado de Compuerta	79
D.10. Implementación y Orientación Práctica	79
D.10.1. Cuándo Usar Cada Arquitectura	79
D.10.2. Consideraciones de Hardware	80
E. Annex E: Conceptual Introduction—Transformers and Attention for Beginners	80
E.1. ¿Qué es un Transformer?	80
E.2. El Mecanismo de Atención: Una Analogía	80
E.3. Ejemplo Práctico Paso a Paso	81
E.3.1. Contexto 1: “El gato come pescado”	81
E.3.2. Contexto 2: “El restaurante come los márgenes de beneficio”	81
E.4. Cómo Funciona la Atención Matemáticamente (Versión Simple)	81
E.4.1. Paso 1: Consultas, Claves y Valores	81
E.4.2. Paso 2: Compatibilidad	81
E.4.3. Paso 3: Enfoque	82
E.4.4. Paso 4: Combinar	82
E.5. Múltiples Cabezas de Atención: Múltiples Perspectivas	82
E.6. Apilar Capas	82
E.7. ¿Por Qué es Importante?	82
E.8. Desafíos Prácticos	83

E.8.1. Complejidad Computacional	83
E.8.2. Requisitos de Memoria	83
E.8.3. Eficiencia del Dispositivo	83
E.9. Soluciones Modernas	83
E.10. Conclusiones Clave	83

1. Introducción

1.1. Motivación y Planteamiento del Problema

Las arquitecturas de transformers han transformado fundamentalmente el panorama del modelado de secuencias en el procesamiento del lenguaje natural [39], la visión por computadora y la biología computacional. El éxito de modelos como BERT [12], GPT y sus variantes ha establecido al transformer como el estándar de facto para el aprendizaje de representaciones en el aprendizaje profundo. Sin embargo, la rápida proliferación de innovaciones arquitectónicas presenta desafíos prácticos significativos para investigadores y profesionales.

Los últimos años han sido testigos de una explosión de mecanismos alternativos de atención y mezcla de secuencias, cada uno abordando limitaciones específicas de la atención softmax estándar: complejidad computacional cuadrática [10], requisitos de inferencia eficiente en memoria [37, 13], gestión selectiva de estados basada en contenido [3] y optimizaciones conscientes del hardware [31]. Simultáneamente, la literatura sobre optimización se ha diversificado más allá del AdamW clásico, introduciendo técnicas de reducción de varianza [43, 27, 50], variantes eficientes en memoria [34, 55], enfoques sin programación de tasa de aprendizaje [11] y métodos de preconditionamiento de segundo orden [15, 40].

La consecuencia práctica es un ecosistema de investigación fragmentado donde la comparación experimental entre arquitecturas y optimizadores requiere un esfuerzo de ingeniería significativo. Los investigadores deben implementar y depurar múltiples variantes desde cero, garantizar tuberías de entrenamiento consistentes y gestionar espacios de hiperparámetros complejos. Esta fragmentación dificulta la reproducibilidad, ralentiza el progreso científico y aumenta la barrera de entrada para nuevos investigadores.

Este trabajo aborda estos desafíos mediante un conjunto de herramientas de experimentación unificado y basado en configuración, disponible en <https://github.com/erickfmm/frankestein-transformer>, que proporciona:

1. **Diseño Basado en Esquemas:** Un contrato de configuración estricto y validado que garantiza la reproducibilidad mientras admite diecisiete arquitecturas de mezcladores de secuencia distintas y veintitrés familias de optimizadores.
2. **Entrenamiento Agnóstico a la Arquitectura:** Infraestructura de entrenamiento común que soporta líneas base de atención densa (estándar, sigmoide), alternativas recurrentes (RetNet, Mamba, bloques tipo EDO), enrutamiento adaptativo de profundidad (Mixture-of-Depths) [58], bloques de memoria condicional (Engram) [9], patrones de atención dispersa (Sparse Transformer, Longformer, BigBird, SparseK, NSA, SpargeAttn, FASA) y mecanismos con compuerta (GLA, DeltaNet, Gated DeltaNet, HGRN2, FoX, Gated Softmax).
3. **Marco de Enrutamiento de Optimizadores:** Grupos de hiperparámetros con prefijos que permiten un control detallado sobre incrustaciones, capas de normalización, bloques recurrentes, bloques de atención y otros subconjuntos de parámetros a través de diversos optimizadores, incluyendo la familia APOLLO (Apollo, Apollo-Mini, Q-Apollo) [56].
4. **Flujos de Trabajo de Extremo a Extremo:** Despliegue integrado mediante cuantización (empaquetado ternario de pesos, activaciones INT8) y capacidades de incrustación de oraciones inspiradas en SBERT [32].
5. **Configuración Interactiva:** Interfaz basada en web que proporciona renderizado de formularios impulsado por esquemas, validación en tiempo real y generación de comandos CLI.
6. **Herramientas de Confiabilidad:** Pruebas automatizadas integrales con suites de pruebas unitarias, validación de presets YAML y automatización de CI en versiones de Python compatibles.

La superficie de comandos del proyecto es:

`frankestein-transformer`

con subcomandos para flujos de trabajo de codificador y decodificador: `train`, `finetune`, `deploy`, `quantize`, `infer`, `sbert-train`, `sbert-infer`, `web-server`.

El comando `web-server` lanza un constructor de configuración basado en Streamlit que proporciona:

- Campos de formulario impulsados por esquemas con títulos de parámetros y descripciones detalladas Información sobre herramientas en tiempo real y texto de ayuda para cada opción de configuración
- Vista previa YAML en vivo y funcionalidad de descarga
- Comandos CLI generados para entrenamiento, despliegue, inferencia y flujos de trabajo SBERT

Esta interfaz interactiva sirve como alternativa a la edición manual de YAML, mejorando la usabilidad para usuarios que exploran las opciones de configuración disponibles y comprenden su impacto en el comportamiento del modelo y la dinámica de entrenamiento.

1.2. Contribuciones

Este trabajo realiza las siguientes contribuciones principales:

1. **Esquema de Configuración Unificado:** Un contrato de esquema basado en YAML con validación estricta que soporta diecisiete arquitecturas de mezcladores de secuencia distintas en cuatro categorías (líneas base densas, bloques recurrentes/retentivos, patrones de atención dispersa y mecanismos con compuerta), junto con controles de profundidad adaptativa y memoria condicional, mientras garantiza la reproducibilidad mediante restricciones `additionalProperties: false`.
2. **Soporte Integral de Arquitecturas:** Implementación de variantes modernas de transformers incluyendo atención softmax estándar [39], atención sigmoide [31], RetNet [37], Mamba [13], transformers continuos tipo EDO [53], atención con aumento de memoria Titans [3], capas de memoria condicional Engram [9], enrutamiento de tokens Mixture-of-Depths [58], mecanismos de atención dispersa [10, 4, 51, 24, 49, 54, 42] y arquitecturas de atención con compuerta [45, 46, 47, 29, 20, 30]. El sistema soporta tanto entrenamiento en modo codificador con atención bidireccional para modelado de lenguaje enmascarado (MLM) como entrenamiento en modo decodificador con enmascaramiento causal para predicción autorregresiva del siguiente token.
3. **Marco de Enrutamiento de Optimizadores:** Sistema de hiperparámetros con prefijos que permite control por grupo de parámetros a través de veintitrés optimizadores, incluyendo Anon con adaptabilidad ajustable [1], métodos de reducción de varianza (MARS, Adan, AdE-MAMix, Cautious AdamW), variantes eficientes en memoria (Adafactor, GaLore, Lion), enfoques sin programación de tasa de aprendizaje (Schedule-Free AdamW), métodos conscientes de curvatura (Sophia), preconditionadores de segundo orden (Shampoo, SOAP), optimizadores orientados a ortogonalidad (Muon, Turbo-Muon), escalado de lotes grandes (LAMB) y la familia APOLLO (Apollo, Apollo-Mini, Q-Apollo) [56].
4. **Cuantización y Despliegue:** Tubería de despliegue integrada que soporta empaquetado ternario de pesos y cuantización de activaciones INT8 con estimaciones de tamaño siguiendo la aproximación de almacenamiento de 1,58 bits.

5. **Flujos de Trabajo de Incrustación de Oraciones:** Tuberías de entrenamiento e inferencia inspiradas en SBERT que soportan puntuación de similitud, recuperación, agrupamiento y exportación persistente de incrustaciones.
6. **Interfaz de Configuración Interactiva:** Servidor web basado en Streamlit que proporciona generación de formularios impulsada por esquemas, validación en tiempo real, documentación en línea y generación de comandos CLI.
7. **Tubería de Validación Automatizada:** Integración continua y pruebas unitarias ampliadas que verifican las rutas de entrenamiento del modelo, la integración optimizador/esquema y la compatibilidad de ejemplos YAML.

1.3. Guía de Lectura

Este documento está organizado como una referencia técnica que aborda cuatro preocupaciones operativas:

1. **Requisitos Previos:** El Apéndice [E](#) proporciona una introducción accesible a los transformers y mecanismos de atención para lectores nuevos en el campo.
2. **Contrato de Configuración:** La Sección [3.1](#) describe el esquema YAML que garantiza experimentos válidos y la Sección [3.2](#) explica las reglas de validación.
3. **Selección de Arquitectura:** La Sección [3.8](#) cubre las opciones de normalización; la Sección [4.1](#) proporciona una comparación exhaustiva de las familias de mezcladores de secuencia; los Apéndices [B](#), [C](#) y [D](#) sintetizan la literatura de respaldo.
4. **Dinámica de Optimización:** La Sección [5](#) detalla el enrutamiento de optimizadores y la dinámica de entrenamiento; el Apéndice [A](#) proporciona un análisis exhaustivo de las familias de optimizadores.
5. **Despliegue e Inferencia:** Las Secciones [6](#) y [7](#) describen el despliegue cuantizado y los flujos de trabajo de incrustación de oraciones.

1.4. Constructor de Configuración Basado en Web

Además de la edición directa de YAML, este proyecto proporciona una interfaz web basada en Streamlit (accesible mediante el comando `web-server`) que mejora la accesibilidad y descubribilidad de la configuración. La interfaz presenta campos de esquema con:

- **Renderizado de formularios impulsado por esquemas** — Todos los campos se generan dinámicamente a partir del esquema autoritativo, garantizando consistencia y validación.
- **Documentación de parámetros en línea** — Cada campo del formulario muestra un *título* del esquema como su etiqueta, con la *descripción* mostrada como información sobre herramientas al pasar el cursor.
- **Vista previa de configuración en tiempo real** — Los usuarios ven la salida YAML en vivo mientras modifican los campos del formulario, permitiendo retroalimentación de validación inmediata.
- **Generación de comandos CLI** — La interfaz genera comandos CLI completos para entrenamiento, despliegue, inferencia y flujos de trabajo SBERT basados en la configuración actual.

- **Mejoras de accesibilidad** — La información sobre herramientas y los formularios estructurados facilitan la comprensión de las opciones de configuración, especialmente para usuarios nuevos en el proyecto o que exploran arquitecturas novedosas.

Este enfoque basado en web aborda barreras de usabilidad comunes en la experimentación impulsada por configuración:

- Reduce la necesidad de memorizar la estructura YAML y los nombres de los campos
- Previene errores tipográficos mediante la validación del esquema
- Proporciona contexto educativo a través de documentación en línea Permite la experimentación rápida con ajuste guiado de parámetros
- Sirve tanto como herramienta de configuración como recurso de aprendizaje

La implementación de la interfaz web utiliza widgets de formulario de Streamlit con:

- `st.checkbox()` para alternativas binarias con texto de ayuda
- `st.number_input()` para campos numéricos con tamaño de paso y formato
- `st.selectbox()` para opciones de enumeración con visualización de opciones
- `st.multiselect()` para selecciones de arreglo a partir de opciones definidas
- `st.text_input()` para campos de cadena
- `st.info()`, `st.caption()` para información complementaria

Los metadatos del esquema (campos de título y descripción) se extraen y renderizan sistemáticamente en todas las secciones del formulario, incluyendo:

- Parámetros de arquitectura del modelo (tamaño oculto, capas, cabezas de atención, etc.)
- Configuraciones de ejecución de entrenamiento (tamaño de lote, acumulación, programador)
- Configuración del optimizador con hiperparámetros por grupo de parámetros
- Opciones de despliegue y cuantización
- Parámetros específicos de entrenamiento e inferencia SBERT

2. Trabajo Relacionado

Este trabajo se sitúa en la intersección de tres áreas activas de investigación: arquitecturas de atención alternativas, mecanismos de atención dispersa y algoritmos de optimización avanzados. Esta sección proporciona un estudio conciso; las formulaciones matemáticas detalladas y las descripciones algorítmicas se difieren a los apéndices.

2.1. Arquitecturas de Mezcladores de Secuencia

2.1.1. Líneas Base de Atención Densa

La atención softmax estándar [39] logra un contexto global completo mediante interacciones por pares de n^2 , pero incurre en costos prohibitivos de memoria y cómputo para secuencias largas. La atención sigmoide [31] reemplaza la softmax por filas con una sigmoide elemento a elemento, logrando una convergencia más rápida y hasta un 17% de aceleración del núcleo, aunque requiere estabilización con normalización híbrida a escala. Ambas sirven como líneas base densas en este conjunto de herramientas.

2.1.2. Arquitecturas Recurrentes y Retentivas

RetNet [37] resuelve el “triángulo imposible” de entrenamiento paralelo, inferencia $\mathcal{O}(1)$ y rendimiento sólido mediante una formulación dual de atención-retención. Mamba [13] introduce selectividad dependiente de la entrada en modelos de espacio de estados, logrando complejidad lineal con algoritmos de exploración conscientes del hardware. Los transformers tipo EDO [53] tratan la profundidad como integración numérica sobre un sistema dinámico continuo. Titans [3] aumenta la inferencia con memorización neuronal en tiempo de prueba para contextos extremadamente largos. En el Apéndice B se proporcionan formulaciones detalladas.

2.1.3. Mecanismos de Atención Dispersa

Los métodos de atención dispersa reducen el cuello de botella cuadrático restringiendo las interacciones entre tokens: Sparse Transformer [10] utiliza patrones factorizados con paso y fijos ($\mathcal{O}(n\sqrt{n})$); Longformer [4] emplea ventanas deslizantes con tokens globales ($\mathcal{O}(n \cdot w)$); BigBird [51] combina caminos locales, aleatorios y globales; SparseK [24] aplica selección diferenciable top- k ; NSA [49] utiliza un diseño jerárquico de tres ramas; SpargeAttn [54] realiza poda a nivel de bloques en dos etapas; y FASA [42] aprovecha características de frecuencia de RoPE para la selección de tokens. Las descripciones completas se encuentran en el Apéndice C.

2.1.4. Mecanismos de Atención con Compuerta

Las arquitecturas con compuerta controlan el flujo de información mediante compuertas aprendibles: GLA [45] agrega compuertas diagonales dependientes de datos a la atención lineal; DeltaNet [46] aplica la regla de aprendizaje delta a las actualizaciones de estado recurrente; Gated DeltaNet [47] sintetiza ambos mecanismos; HGRN2 [29] introduce compuertas de olvido jerárquicas con expansión de estado mediante producto exterior; FoX [20] incrusta compuertas de olvido directamente en la atención softmax; y Gated Softmax [30] aplica compuertas de canal posteriores a SDPA. Las derivaciones matemáticas completas aparecen en el Apéndice D.

2.2. Algoritmos de Optimización

La optimización de arquitecturas de transformers altamente parametrizadas requiere navegar paisajes de pérdida no convexos con espectros de Hessiana heterogéneos. La literatura se ha diversificado en varias familias más allá de la línea base clásica de AdamW [23]:

- **Reducción de varianza y momento:** RAdam [22], Adan [43], ADOPT [38], AdEMAMix [27], MARS [50] y optimizadores Cautious [19] abordan la inestabilidad en pasos iniciales, las garantías de convergencia y la mezcla de historial con EMA dual.
- **Eficientes en memoria:** Adafactor [34], GaLore [55], Lion [8] y APOLLO [56] reducen la memoria del estado del optimizador mediante factorización, proyección de bajo rango o actualizaciones basadas en signo.
- **Sin programación de tasa y libres de parámetros:** Schedule-Free AdamW [11] y Prodigy [25] absorben el programador o el ajuste de la tasa de aprendizaje en la dinámica del optimizador.
- **De segundo orden y conscientes de curvatura:** Shampoo [15], SOAP [40] y Sophia [21] incorporan información aproximada de segundo orden.
- **Orientados a geometría:** Muon [35] y Turbo-Muon [5] remodelan la geometría de actualización mediante ortogonalización.

En el Apéndice A se proporcionan descripciones algorítmicas detalladas, pseudocódigo y una comparación exhaustiva de complejidad.

3. Diseño y Arquitectura del Sistema

3.1. Arquitectura Centrada en Configuración

La decisión de diseño principal en este repositorio es que la experimentación es *esquema primero*. En lugar de exponer una gran cantidad de indicadores laxamente verificados, el proyecto obliga a la topología del modelo, la familia de optimizadores, los límites de entrenamiento y las opciones de telemetría a través de un único documento de configuración validado. Esto reduce la ambigüedad al reproducir resultados y permite comparar muchas arquitecturas bajo una interfaz operativa consistente.

El contrato autoritativo es `configs/schema.yaml`. Este impone tres objetos de primer nivel:

- `model_class`
- `model`
- `training`

Selección de Clase de Modelo. El campo `model_class` determina la variante arquitectónica instanciada por el pipeline de entrenamiento. Se soportan tres opciones:

- **frankenstein:** Modelos encoder de arquitectura mixta que soportan diversos mecanismos de atención (estándar, sigmoide, retentiva, espacio de estados, dispersa y mezcladores con compuerta) con MoE (Mezcla de Expertos) y características avanzadas. Optimizado para entrenamiento bidireccional tipo encoder con objetivos de modelado de lenguaje enmascarado (MLM).
- **mini:** Variante de encoder simplificada diseñada para escenarios de entrenamiento a menor escala. Proporciona una sobrecarga de parámetros reducida e iteración más rápida para experimentación y creación de prototipos.
- **frankesteindecoder:** Decoder causal autorregresivo para generación de siguiente token estilo LLM. Permite el enmascaramiento de atención causal para tareas de generación secuencial de texto. Cuando se selecciona esta clase, en tiempo de ejecución se fuerza `mode='decoder'`.

Selección de Modo de Entrenamiento. El campo `model.mode` controla el comportamiento de enmascaramiento de atención en todo el modelo:

- **encoder:** Utiliza atención bidireccional donde todos los tokens atienden a todos los demás tokens en la secuencia. Adecuado para tareas de preentrenamiento de modelado de lenguaje enmascarado (MLM) donde el modelo aprende a predecir tokens enmascarados aleatoriamente basándose en el contexto completo.
- **decoder:** Utiliza enmascaramiento causal donde cada token solo puede atender a tokens anteriores en la secuencia. Requerido para tareas autorregresivas (AR) de predicción del siguiente token, como modelado de lenguaje y generación de texto. Cuando `model_class='frankesteindecoder'`, el sistema fuerza automáticamente `mode='decoder'` en tiempo de ejecución.

Este soporte de arquitectura dual permite al sistema manejar tanto preentrenamiento tipo encoder (MLM en contextos bidireccionales) como generación tipo decoder (predicción causal autorregresiva) a través de una interfaz de configuración unificada.

El `model.layer_pattern` soporta bloques heredados, dispersos y con compuerta:

- **Retentive Network (RetNet)** — referencia interna: `sun_retentive_2023` — nombre de código: `retnet`, `retnet_attn`

- **Mamba (Selective State Space Model)** — referencia interna: `gu_mamba_2023` — nombre de código: `mamba`
- **Bloque de Profundidad Continua estilo ODE** — referencia interna: `zhang_continuous_2021` — nombre de código: `ode`
- **Atención con Memoria Aumentada Titans** — referencia interna: `behrouz_titans_2025` — nombre de código: `titan_attn`
- **Atención Softmax Estándar** — referencia interna: `vaswani_attention_2017` — nombre de código: `standard_attn`
- **Auto-Atención Sigmoide** — referencia interna: `ramapuram_theory_2024` — nombre de código: `sigmoid_attn`
- **Transformer Disperso** — referencia interna: `child_sparse_transformer_2019` — nombre de código: `sparse_transformer_attn`
- **Longformer** — referencia interna: `beltagy_longformer_2020` — nombre de código: `longformer_attn`
- **BigBird** — referencia interna: `zaheer_bigbird_2020` — nombre de código: `bigbird_attn`
- **Atención SparseK** — referencia interna: `lou_sparsek_2024` — nombre de código: `sparsek_attn`
- **Atención Dispersa Nativa (NSA)** — referencia interna: `yuan_nsa_2025` — nombre de código: `nsa_attn`
- **SpargeAttn** — referencia interna: `zhang_spargeattn_2025` — nombre de código: `sparge_attn`
- **FASA (Frequency-aware Sparse Attention)** — referencia interna: `wang_fasa_2026` — nombre de código: `fasa_attn`
- **Atención Lineal con Compuerta (GLA)** — referencia interna: `yang_gla_2023` — nombre de código: `gla_attn`
- **DeltaNet** — referencia interna: `yang_deltanet_2024` — nombre de código: `deltanet_attn`
- **DeltaNet con Compuerta** — referencia interna: `yang_gated_deltanet_2024` — nombre de código: `gated_deltanet_attn`
- **HGRN2** — referencia interna: `qin_hgrn2_2024` — nombre de código: `hgrn2_attn`
- **Forgetting Transformer (FoX)** — referencia interna: `lin_forgetting_transformer_2025` — nombre de código: `fox_attn`
- **Atención Softmax con Compuerta** — referencia interna: `qiu_gated_attention_2025` — nombre de código: `gated_softmax_attn`

que corresponde a la literatura actual de atención y mezcladores de secuencias [37, 13, 53, 3, 31, 39, 10, 4, 51, 24, 49, 54, 42, 45, 46, 47, 29, 20, 30]

El `training.optimizer.optimizer_class` soporta una amplia familia de optimizadores: `sgd_momentum`, `adamw`, `adafactor`, `galore_adamw`, `prodigy`, `lion`, `sophia`, `muon`, `turbo_muon`, `radam`, `adan`, `adopt`, `ademamix`, `mars_adamw`, `cautious_adamw`, `lamb`, `schedulefree_adamw`, `shampoo`, `soap`, `anon`, `apollo`, `apollo_mini`, y `q_apollo`.

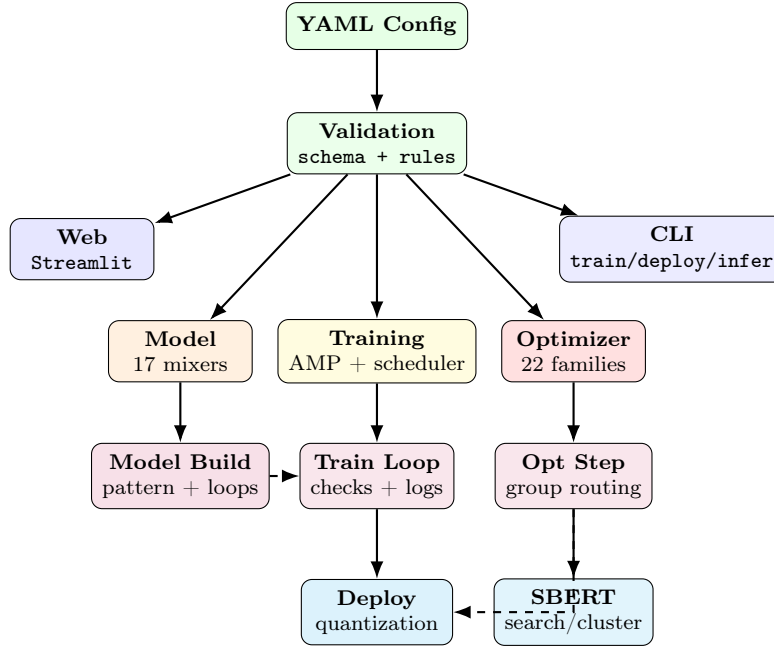


Figura 1: Arquitectura del sistema: la configuración fluye a través de la validación, se divide en modelo/entrenamiento/optimizador, ejecuta en tiempo real y produce artefactos de despliegue/SBERT.

3.2. Alcance del Esquema y Reglas de Validación

El esquema es estricto: los objetos de primer nivel y anidados establecen `additionalProperties: false`. Esto garantiza que las claves desconocidas fallen rápidamente en lugar de ser ignoradas silenciosamente. El objeto `training.optimizer.parameters` está adicionalmente restringido por reglas de prefijo específicas del optimizador mediante verificaciones de patrón `allOf+if/then`.

Los valores de normalización actualmente aceptados por el esquema son:

$$\text{norm_type} \in \{\text{layer_norm}, \text{dynamic_tanh}, \text{derf}\}$$

Por lo tanto, `rms_norm` **no** es un valor de esquema válido en el contrato actual.

3.3. Inventario Completo de Características del Modelo

Campo	Tipo/Rango	Significado
<code>model_class</code>	enum	Variante arquitectónica: <code>frankenstein</code> (encoder de arquitectura mixta), <code>mini</code> (encoder simplificado), <code>frankesteindecoder</code> (decoder autorregresivo).
<code>model.mode</code>	enum	Modo de atención: <code>encoder</code> (bidireccional, para MLM) o <code>decoder</code> (causal, para AR). Cuando <code>model_class='frankesteindecoder'</code> , fuerza <code>mode='decoder'</code> .
<code>vocab_size</code>	$\text{int} \geq 1$	Tamaño del vocabulario.
<code>hidden_size</code>	$\text{int} \geq 1$	Dimensión oculta.
<code>num_layers</code>	$\text{int} \geq 1$	Número de capas físicas.
<code>num_loops</code>	$\text{int} \geq 1$	Número de bucles lógicos (bloques en bucle).
<code>num_heads</code>	$\text{int} \geq 1$	Cabezas de atención.
<code>retention_heads</code>	$\text{int} \geq 1$	Cabezas de retención para mezcladores estilo RetNet.

Campo	Tipo/Rango	Significado
num_experts	int ≥ 1	Número de expertos en MoE.
top_k_experts	int ≥ 1	Enrutamiento top- k de expertos en MoE.
dropout	float [0, 1]	Dropout global.
layer_pattern	array enum	Lista ordenada de bloques: heredados (retnet, retnet_attn, mamba, ode, titan_attn, standard_attn, sigmoid_attn), dispersos (sparse_transformer_attn, longformer_attn, bigbird_attn, sparsek_attn, nsa_attn, sparge_attn, fasa_attn), y con compuerta (gla_attn, deltaret_attn, gated_deltaret_attn, hgrn2_attn, fox_attn, gated_softmax_attn).
ode_solver	enum	rk4 o euler.
ode_steps	int ≥ 1	Pasos de integración ODE.
use_bitnet	bool	Habilitar ruta BitLinear de bajo bit.
norm_type	enum	layer_norm, dynamic_tanh, derf.
use_factorized_embedding	bool	Habilitar embeddings factorizados.
factorized_embedding_dim	int ≥ 1	Dimensión reducida de embedding para factorización.
use_embedding_conv	bool	Habilitar Conv1d sobre el flujo de embeddings.
embedding_conv_kernel	int ≥ 1	Tamaño del kernel Conv1d.
hope_base	float ≥ 0	Valor base de HoPE (opcional en el esquema).
hope_damping	float ≥ 0	Amortiguamiento de HoPE (opcional en el esquema).
use_hope	bool	Aplicar HoPE en titan_attn.
use_moe	bool	Habilitar ruta FFN con MoE.
ffn_hidden_size	int ≥ 1	Ancho intermedio de la FFN.
ffn_activation	enum	silu o gelu.

La profundidad en bucle inducida por el esquema es:

$$L_{\text{logical}} = \text{num_layers} \times \text{num_loops}$$

que es la definición a nivel de configuración de los bloques en bucle.

3.4. Tipos de Tarea de Entrenamiento

El campo `training.task` determina el objetivo de entrenamiento, trabajando en conjunto con `model.mode` para definir cómo aprende el modelo:

Modelado de Lenguaje Enmascarado (MLM).

- **Modo:** Requiere `mode='encoder'` para atención bidireccional.
- **Objetivo:** Enmascarar aleatoriamente tokens en la secuencia de entrada (típicamente 15%) y entrenar al modelo para predecir los tokens enmascarados basándose en el contexto bidireccional completo.
- **Caso de Uso:** Preentrenamiento de encoders para aprendizaje de representaciones, siguiendo la metodología BERT [12]. El modelo aprende representaciones bidireccionales que capturan contexto tanto de izquierda como de derecha.
- **Configuración:** Utiliza el parámetro `mlm_probability` para controlar la fracción de enmascaramiento.

Predicción Autorregresiva (AR) del Siguiente Token.

- **Modo:** Requiere `mode='decoder'` para enmascaramiento causal.
- **Objetivo:** Entrenar al modelo para predecir el siguiente token en la secuencia dados solo los tokens anteriores.
- **Caso de Uso:** Generación de lenguaje y tareas estilo LLM siguiendo la metodología GPT. El modelo aprende dependencias secuenciales con atención causal donde cada token solo puede atender a tokens precedentes.
- **Clase de Modelo:** Típicamente usa `model_class='frankesteindecoder'` para arquitecturas de decoder autorregresivo.

Este soporte de tareas dual permite la experimentación unificada tanto en preentrenamiento tipo encoder (MLM para comprensión bidireccional) como en generación tipo decoder (AR para producción secuencial de texto) dentro del mismo código base.

3.5. Inventario Completo de Características de Entrenamiento

Campo	Tipo/Rango	Significado
<code>batch_size</code>	<code>int ≥ 1</code>	Tamaño de lote del cargador.
<code>dataloader_workers</code>	<code>int ≥ 0</code>	Trabajadores del dataloader de PyTorch.
<code>max_length</code>	<code>int ≥ 1</code>	Límite de longitud de secuencia.
<code>task</code>	<code>enum</code>	Objetivo de entrenamiento: <code>mlm</code> (modelado de lenguaje enmascarado) o <code>sbert</code> (embedding de oraciones).
<code>mlm_probability</code>	<code>float [0, 1]</code>	Probabilidad de enmascaramiento MLM (aplica solo cuando <code>task='mlm'</code>).
<code>max_samples</code>	<code>int ≥ 1</code>	Máximo de muestras transmitidas.
<code>dataset_batch_size</code>	<code>int ≥ 1</code>	Tamaño interno de fragmento del conjunto de datos en streaming.
<code>num_workers</code>	<code>int ≥ 0</code>	Trabajadores del conjunto de datos en streaming.
<code>cache_dir</code>	<code>string</code>	Directorio de caché del conjunto de datos.
<code>local_parquet_dir</code>	<code>string</code>	Ruta opcional local a parquet.
<code>prefer_local_cache</code>	<code>bool</code>	Preferir caché local cuando esté disponible.
<code>stream_local_parquet</code>	<code>bool</code>	Modo de transmisión desde parquet local.
<code>use_amp</code>	<code>bool</code>	Alternar precisión mixta.
<code>gradient_accumulation_steps</code>	<code>int ≥ 1</code>	Lote efectivo mediante acumulación.
<code>optimizer</code>	<code>object</code>	Contiene <code>optimizer_class</code> y <code>parameters</code> con prefijo.
<code>scheduler_total_steps</code>	<code>int ≥ 1</code>	Horizonte del scheduler.
<code>scheduler_warmup_ratio</code>	<code>float [0, 1]</code>	Proporción de calentamiento.
<code>scheduler_type</code>	<code>enum</code>	<code>cosine</code> , <code>constant</code> , <code>linear_warmup_then_constant</code> .
<code>grad_clip_max_norm</code>	<code>float ≥ 0</code>	Umbral de recorte de norma global.
<code>inf_post_clip_threshold</code>	<code>float ≥ 0</code>	Guarda contra gradientes explosivos después del recorte.
<code>max_nan_retries</code>	<code>int ≥ 0</code>	Presupuesto de reintentos para inestabilidad NaN/Inf.
<code>checkpoint_every_n_steps</code>	<code>int ≥ 1</code>	Frecuencia de checkpoint rodante.
<code>max_rolling_checkpoints</code>	<code>int ≥ 1</code>	Número de checkpoints rodantes a mantener.
<code>num_best_checkpoints</code>	<code>int ≥ 1</code>	Número de mejores checkpoints rastreados.
<code>nan_check_interval</code>	<code>int ≥ 1</code>	Cadencia de verificación NaN/Inf.

Campo	Tipo/Rango	Significado
log_gradient_stats	bool	Habilitar registro de estadísticas de gradientes.
gradient_log_interval	int ≥ 1	Cadencia de registro de gradientes.
csv_log_path	string	Ruta de salida CSV a nivel de paso.
csv_rotate_on_schema_change	bool	Rotar CSV si el esquema de registro cambia.
gpu_metrics_backend	enum	nvml o none.
nvml_device_index	int ≥ 0	Índice de dispositivo para telemetría NVML.
enable_block_grad_norms	bool	Incluir telemetría de norma de gradiente por bloque.
telemetry_log_interval	int ≥ 1	Intervalo de telemetría pesada (pasos de optimizador).
use_galore	bool	Habilitar estrategia GaLore.
galore_rank	int ≥ 1	Dimensión de proyección de bajo rango de GaLore.
galore_update_interval	int ≥ 1	Intervalo de actualización de proyección.
galore_scale	float ≥ 0	Escalado de gradientes en el espacio proyectado.
galore_max_dim	int ≥ 1	Dimensión máxima del tensor para proyección GaLore.

3.6. Contrato de Prefijo del Optimizador (Completo)

Los valores soportados de `optimizer_class` son: `sgd_momentum`, `adamw`, `adafactor`, `galore_adamw`, `prodigy`, `lion`, `sophia`, `muon`, `turbo_muon`, `radam`, `adan`, `adopt`, `ademamix`, `mars_adamw`, `cautious_adamw`, `lamb`, `schedulefree_adamw`, `shampoo`, `soap`, `apollo`, `apollo_mini`, `q_apollo`.

Las familias de sufijos compartidos por grupo (todos prefijados por el nombre del optimizador) son:

- Grupos de LR: `lr_embeddings`, `lr_norms`, `lr_ode`, `lr_retnet`, `lr_mamba`, `lr_attention`, `lr_other`
- Grupos de decaimiento de peso: `wd_embeddings`, `wd_norms`, `wd_ode`, `wd_retnet`, `wd_mamba`, `wd_attention`, `wd_other`
- Grupos de betas: `betas_embeddings`, `betas_norms`, `betas_ode`, `betas_retnet`, `betas_mamba`, `betas_attention`, `betas_other`
- Grupos de épsilon: `eps_embeddings`, `eps_norms`, `eps_ode`, `eps_retnet`, `eps_mamba`, `eps_attention`, `eps_other`

Sufijos globales específicos del optimizador:

- `sgd_momentum`: `momentum`, `nesterov`
- `adafactor`: `beta2_decay`, `clip_threshold`, `eps1`, `eps2`
- `galore_adamw`: `rank`, `update_proj_gap`
- `prodigy`: `d_coef`
- `sophia`: `rho`, `update_k`
- `muon` / `turbo_muon`: `momentum`, `nesterov`, `ns_steps`, `ns_eps`
- `cautious_adamw`: `cautious_clip`
- `apollo`: `rank`, `update_proj_gap`, `scale`, `scale_type`, `proj_type`, `scale_front`, `disable_nl`

Algorithm 1 Paso de Entrenamiento Guiado por Esquema con Controles de Estabilidad

Require: Flujo de lotes, configuración C

```
1: Inicializar contador de reintentos  $r \leftarrow 0$ 
2: for cada paso del optimizador do
3:   Acumular gradientes para  $K = C.gradient\_accumulation\_steps$  micro-lotes
4:   Aplicar recorte de norma global con  $\tau = C.grad\_clip\_max\_norm$ 
5:   if el gradiente post-recorte excede  $C.inf\_post\_clip\_threshold$  o se detecta NaN/Inf
   then
6:     if  $r < C.max\_nan\_retries$  then
7:       restaurar estado seguro / saltar paso;  $r \leftarrow r + 1$ 
8:       continue
9:     else
10:      detener entrenamiento con estado de fallo
11:    end if
12:  end if
13:  ejecutar paso del optimizador seleccionado por optimizer_class
14:  actualizar scheduler (cosine, constant, o linear_warmup_then_constant)
15:  if paso mod checkpoint_every_n_steps = 0 then
16:    guardar checkpoint rodante y podar a max_rolling_checkpoints
17:  end if
18:  actualizar mejores checkpoints hasta num_best_checkpoints
19:  emitir CSV + telemetría según gradient_log_interval y telemetry_log_interval
20: end for
```

- `apollo_mini`: `update_proj_gap`, `scale`, `proj_type`, `scale_front`, `disable_nl`
- `q_apollo`: `rank`, `update_proj_gap`, `scale`, `scale_type`, `proj_type`, `scale_front`, `disable_nl`, `quant_bits`

Todas las demás clases en la lista anterior aceptan solo grupos compartidos con prefijo.

3.7. Seguridad de Entrenamiento y Semántica de Ejecución

Las características de seguridad a nivel de esquema incluyen acumulación, recorte, verificación de explosión post-recorte y reintentos por NaN:

$$g_{acc} = \frac{1}{K} \sum_{i=1}^K g_i, \quad K = \text{gradient_accumulation_steps}$$

$$g_{clip} = g_{acc} \cdot \min \left(1, \frac{\tau}{\|g_{acc}\|_2 + \epsilon} \right), \quad \tau = \text{grad_clip_max_norm}$$

luego los guardas de desbordamiento usan `inf_post_clip_threshold` y lógica de reintentos limitada por `max_nan_retries`.

3.8. Variantes de Normalización: RMSNorm, Tanh Dinámico y Erf Dinámico

La normalización determina cómo se controla la escala de activación a través de la profundidad. En este repositorio, la normalización no es solo una elección de modelado sino también una cuestión de compatibilidad con el esquema, porque solo ciertos valores son actualmente aceptados por `norm_type`. Las tres formulaciones más relevantes para este código base son:

3.9. RMSNorm

RMSNorm elimina la centralización de la media y solo reescala por la magnitud de la raíz cuadrada media [52]:

$$\text{RMS}(x) = \sqrt{\frac{1}{d} \sum_{i=1}^d x_i^2 + \epsilon}, \quad y_i = \gamma_i \frac{x_i}{\text{RMS}(x)}$$

En comparación con LayerNorm, RMSNorm es computacionalmente más simple (sin resta de la media de características) y se usa a menudo cuando es importante reducir la sobrecarga de normalización.

3.10. Tanh Dinámico (DyT)

El Tanh Dinámico propone reemplazar la normalización explícita con un mapeo elemento a variable acotado [57]:

$$\text{DyT}(x) = \tanh(\alpha x)$$

donde α se aprende. La idea central es que la contracción no lineal acotada puede proporcionar un escalado de señal estable sin calcular explícitamente estadísticas de normalización por token.

3.11. Erf Dinámico (Derf)

Derf extiende la misma dirección libre de normalización usando un mapeo basado en la función error [7]:

$$\text{Derf}(x) = \text{erf}(\alpha x + s)$$

con escala/desplazamiento aprendibles. Los resultados reportados en el trabajo citado indican un rendimiento superior al de DyT y las líneas base de normalización comunes en múltiples dominios.

3.12. Implicaciones del Esquema

El contrato de configuración actual en este repositorio permite:

$$\text{norm_type} \in \{\text{layer_norm}, \text{dynamic_tanh}, \text{derf}\}$$

por lo que DyT y Derf están directamente disponibles en ejecuciones guiadas por esquema, mientras que RMSNorm no es actualmente un valor de enumeración aceptado y requeriría extensión de código/esquema.

Método	Fórmula	Estadísticas Necesarias	Notas
RMSNorm	$\gamma_i x_i / \sqrt{\frac{1}{d} \sum_j x_j^2 + \epsilon}$	Solo RMS	Menor sobrecarga; línea base ampliamente usada [52].
Tanh Dinámico	$\tanh(\alpha x)$	ninguna	Transformación acotada libre de normalización; reemplazo directo [57].
Erf Dinámico	$\text{erf}(\alpha x + s)$	ninguna	Alternativa libre de normalización; mejora sobre DyT [7].

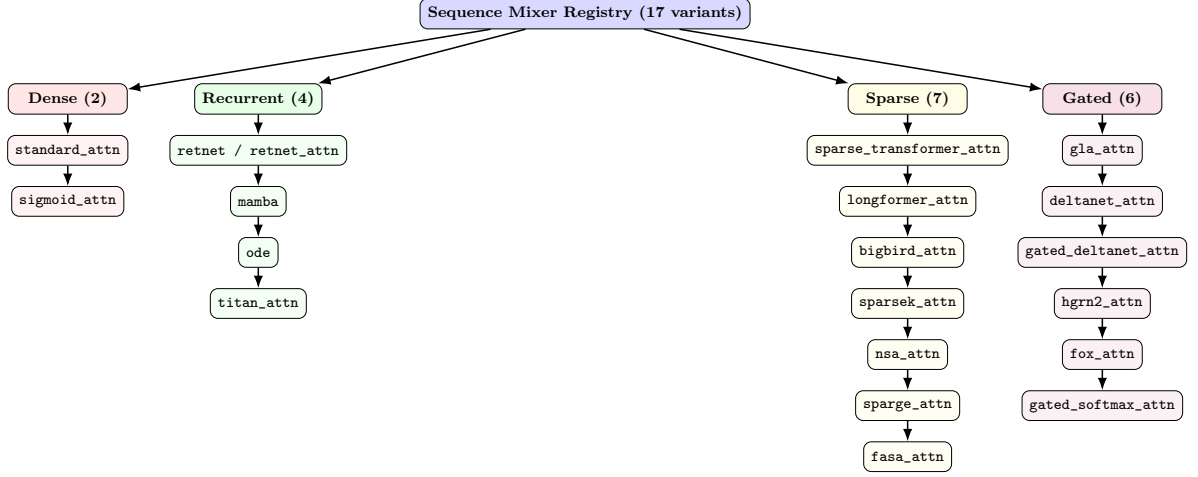


Figura 2: Taxonomía completa de las diecisiete arquitecturas de mezcladores de secuencias soportadas. Las líneas base densas proporcionan enrutamiento global completo a costo cuadrático. Las arquitecturas recurrentes permiten inferencia en tiempo constante mediante compresión de estado. Las variantes dispersas reducen la complejidad mediante patrones estructurados o selección de tokens. Los mecanismos con compuerta introducen control dependiente de datos sobre la retención y el olvido de memoria.

4. Taxonomía de Arquitectura e Implementación

4.1. Familias de Atención y Mezcladores de Secuencias

Este sistema implementa diecisiete arquitecturas de mezcladores de secuencias distintas organizadas en cinco categorías funcionales que reflejan las tendencias de investigación en el diseño de modelado de secuencias. La organización taxonómica refleja la comprensión evolutiva de cómo equilibrar expresividad, eficiencia computacional y restricciones de memoria.

1. **Líneas Base de Atención Densas:** La atención softmax estándar y la atención sigmoide proporcionan contextualización global completa a costo computacional cuadrático, sirviendo como líneas base de referencia para comparación con alternativas más eficientes.
2. **Arquitecturas Recurrentes y Retentivas:** RetNet, Mamba, bloques estilo ODE y Titans mantienen representaciones de estado que permiten un costo de inferencia $\mathcal{O}(1)$ mientras preservan la expresividad mediante dinámicas recurrentes, parámetros selectivos o adaptación de memoria en tiempo de prueba.
3. **Patrones de Atención Dispersa:** Siete variantes dispersas (Sparse Transformer, Longformer, BigBird, SparseK, NSA, SpargeAttn, FASA) reducen la complejidad cuadrática mediante dispersión estructurada, selección de tokens o estrategias de poda sin entrenamiento.
4. **Mecanismos de Memoria con Compuerta:** Seis arquitecturas con compuerta (GLA, DeltaNet, Gated DeltaNet, HGRN2, FoX, Gated Softmax) introducen control dependiente de datos sobre la retención de memoria, el olvido y la fuerza de actualización.

4.2. Atención Estándar

Dadas las matrices proyectadas (Q, K, V) :

$$\text{Attn}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

Este es el mecanismo base para el enrutamiento de contenido [39].

4.3. Atención Sigmoide

La atención sigmoide elimina la normalización de probabilidad por fila:

$$\text{SigmoidAttn}(Q, K, V) = \sigma \left(\frac{QK^\top}{\sqrt{d_k}} + b \right) V$$

y tiene diferentes requisitos de estabilidad de entrenamiento, a menudo con normalización adicional [31].

4.4. Formulación Retentiva

RetNet usa retención con matriz de decaimiento D :

$$\text{Retention}(Q, K, V) = \left(QK^\top \odot D \right) V$$

con forma recurrente:

$$S_n = \gamma S_{n-1} + k_n^\top v_n, \quad o_n = q_n S_n$$

permitiendo inferencia recurrente de bajo costo [37, 44].

4.5. SSM Selectivo (Mamba)

La recurrencia discreta selectiva de espacio de estados es:

$$h_t = \bar{A}_t h_{t-1} + \bar{B}_t x_t, \quad y_t = C_t h_t$$

donde $(\bar{A}_t, \bar{B}_t, C_t)$ dependen de la entrada, preservando escalado en tiempo lineal con escaneo consciente del hardware [13, 16].

4.6. Actualizaciones Continuas estilo ODE

El marco de profundidad continua:

$$\frac{dh(t)}{dt} = f_\theta(h(t), t)$$

con integradores RK prácticos para ejecución discreta [53].

4.7. Memoria en Tiempo de Prueba (Titans)

Una actualización aumentada con memoria puede escribirse:

$$M_t = (1 - \alpha_t) M_{t-1} + S_t, \quad S_t = \eta_t S_{t-1} - \theta_t \nabla \ell(M_{t-1}; x_t)$$

para adaptar la memoria en tiempo de inferencia [3, 2].

4.8. Extensiones Dispersas y con Compuerta en el Código Base Actual

El registro de mezcladores ahora incluye bloques dispersos: `sparse_transformer_attn`, `longformer_attn`, `bigbird_attn`, `sparsek_attn`, `nsa_attn`, `sparge_attn`, y `fasa_attn`; y bloques con compuerta: `gla_attn`, `deltanet_attn`, `gated_deltanet_attn`, `hgrn2_attn`, `fox_attn`, y `gated_softmax_attn`.

La implementación impone una política de ejecución explícita para métodos dispersos sin entrenamiento: `fasa_attn` y `sparge_attn` son solo para evaluación/inferencia y generan errores en tiempo de ejecución si se usan mientras el modelo está en modo de entrenamiento.

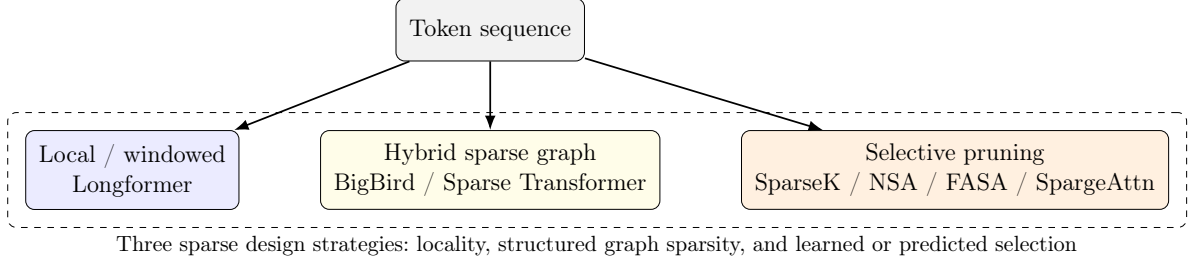


Figura 3: Mapa conceptual de las estrategias de diseño de atención dispersa utilizadas en el código base. Diferentes métodos reducen el costo restringiendo vecindarios, construyendo grafos dispersos o seleccionando solo tokens/bloques de alto valor.

4.9. Bloques de Atención Dispersa Implementados (Detallado)

Este código base incluye siete familias de atención dispersa [10, 4, 51, 24, 49, 54, 42].

Sparse Transformer (`sparse_transformer_attn`). Utiliza máscaras dispersas factorizadas (con zancada + fijas) para aproximar la conectividad densa a menor costo que la atención completa $\mathcal{O}(n^2)$:

$$\text{Attn}_i = \text{softmax}\left(\frac{q_i K_{A_i}^\top}{\sqrt{d_k}}\right) V_{A_i}$$

donde A_i es la vecindad dispersa inducida por reglas de zancada/fijas. [10]

Longformer (`longformer_attn`). Utiliza localidad de ventana deslizante con tokens globales opcionales:

$$A_i = \{j : |i - j| \leq w/2\} \cup \mathcal{G}$$

produciendo escalado lineal en longitud de secuencia para ventana fija w . [4]

BigBird (`bigbird_attn`). Combina ventanas locales, enlaces aleatorios y tokens globales:

$$A_i = A_i^{\text{window}} \cup A_i^{\text{random}} \cup A_i^{\text{global}}$$

para preservar una fuerte conectividad de contexto largo con cómputo disperso. [51]

SparseK (`sparsek_attn`). Utiliza una proyección diferenciable tipo top- k sobre puntuaciones de importancia antes de la atención, de modo que solo los pares KV seleccionados participan en la ruta costosa de producto punto. [24]

NSA (`nsa_attn`). Implementa un diseño disperso de tres ramas: rama comprimida, rama seleccionada y rama de ventana local, y luego las combina con compuertas aprendidas:

$$o_t = \sum_{c \in \{\text{cmp}, \text{sel}, \text{win}\}} g_t^c \text{Attn}(q_t, \tilde{K}_t^c, \tilde{V}_t^c)$$

[49]

SpargeAttn (`sparge_attn`). Filtrado de bloques en dos etapas sin entrenamiento: primero predice interacciones de bloques insignificantes, luego aplica poda consciente de softmax para eliminar bloques de baja contribución. [54]

FASA (`fasa_attn`). Atención sin entrenamiento consciente de frecuencia: utiliza fragmentos de frecuencia RoPE dominantes para la predicción de importancia de tokens, luego aplica atención completa solo en los tokens seleccionados. [42]

Bloque	Entrenable	Tendencia Asintótica	Unidad de Dispersión Principal	Notas de Integración Actual	Ref
Sparse Transformer	Sí	sub-cuadrática	patrón de máscara (a nivel de token)	Máscaras factorizadas con zancada/fijas dentro del pipeline SDPA.	[10]
Longformer	Sí	lineal en n (w fijo)	ventana deslizante + tokens globales	Máscara de ventana con índices globales opcionales.	[4]
BigBird	Sí	casi lineal	bordes ventana + aleatorios + globales	Máscara dispersa aleatorizada más rutas local/global.	[51]
SparseK	Sí	similar a lineal (KV seleccionados)	selección diferenciable top- k de KV	Red de puntuación aprendida + proyección SparseK + atención con KV agrupados.	[24]
NSA	Sí	multi-rama con tokens reducidos	bloques comprimidos + bloques seleccionados + ventana local	Tres ramas dispersas combinadas con compuerta en un tensor de salida.	[49]
SpargeAttn	No (sin entrenamiento)	dependiente del bloque disperso	dispersión de bloque predecida a nivel de bloque	Solo evaluación en este repositorio; genera error en modo entrenamiento.	[54]
FASA	No (sin entrenamiento)	dependiente del token seleccionado	fragmentos de frecuencia dominantes + tokens seleccionados	Solo evaluación en este repositorio; genera error en modo entrenamiento.	[42]

4.10. Bloques de Atención con Compuerta Implementados (Detallado)

Este código base incluye siete bloques con compuerta [45, 46, 47, 37, 29, 20, 30].

La idea unificadora es que las compuertas controlan *qué información sobrevive*. Algunas compuertas actúan sobre actualizaciones de estado recurrente (GLA, variantes de DeltaNet, HGRN2), mientras que otras modifican la ruta de atención completa (FoX y Gated Softmax). Esto hace que las compuertas sean especialmente útiles cuando el modelo debe equilibrar recuerdo, actualidad y memoria limitada.

GLA (`gla_attn`). La Atención Lineal con Compuerta aplica decaimiento multiplicativo dependiente de datos en las actualizaciones de estado recurrente:

$$S_t = G_t \odot S_{t-1} + v_t k_t^\top, \quad o_t = S_t q_t$$

para controlar la acumulación de memoria. [45]

DeltaNet (`deltanet_attn`). Utiliza una escritura correctora de error tipo delta-regla con fuerza de escritura aprendida β_t :

$$S_t = S_{t-1}(I - \beta_t k_t k_t^\top) + \beta_t v_t k_t^\top$$

lo que mejora el reemplazo de memoria dirigido. [46]

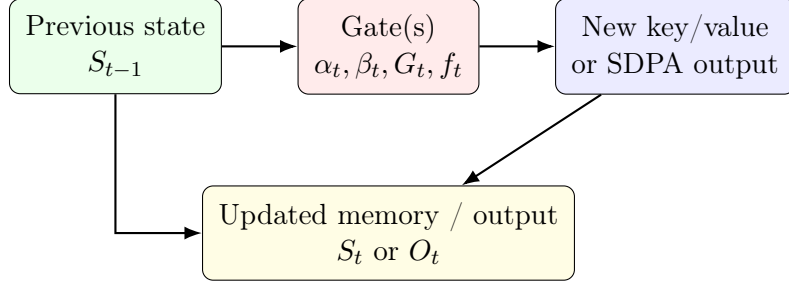


Figura 4: Plantilla de compuerta genérica. Una compuerta puede atenuar la memoria existente, regular la fuerza de escritura o modular las salidas de atención densa, dependiendo de la familia del bloque.

Gated DeltaNet (gated_deltanet_attn). Añade compuerta de decaimiento α_t sobre las escrituras de delta-regla:

$$S_t = \alpha_t S_{t-1} (I - \beta_t k_t k_t^\top) + \beta_t v_t k_t^\top$$

tanto para olvido global como para actualizaciones correctivas locales. [47]

Alias RetNet Attn (retnet_attn). Proporciona un alias explícito con compuerta que envuelve el comportamiento de retención multiescala para consistencia de nomenclatura en los registros de capas. [37]

HGRN2 (hgrn2_attn). Utiliza compuertas de olvido con cota inferior y expansión de estado con producto exterior:

$$S_t = \text{diag}(g_t) S_{t-1} + v_t k_t^\top$$

para aumentar la expresividad del estado recurrente mientras se mantiene la eficiencia. [29]

FoX (fox_attn). Inyecta sesgo de olvido a nivel de token directamente en los logits de softmax:

$$O = \text{softmax}(QK^\top + D)V$$

donde D se deriva de compuertas acumulativas de log-olvido. [20]

Gated Softmax (gated_softmax_attn). Aplica una compuerta sigmoide post-SDPA:

$$Y' = \text{SDPA}(Q, K, V) \odot \sigma(XW_g)$$

lo que añade compuerta de canal multiplicativa sin reemplazar la atención softmax. [30]

Bloque	Tipo de Estado	Mecanismo de Compuerta	Ruta Softmax	Notas de Integración Actual	Ref
GLA	estado recurrente matricial	decaimiento multiplicativo dependiente de datos	No (recurrente lineal)	Actualización recurrente con proyección de compuerta de bajo rango.	[45]
DeltaNet	estado recurrente matricial	compuerta de escritura (β)	No (recurrente lineal)	Corrección delta-regla con Q/K normalizados.	[46]

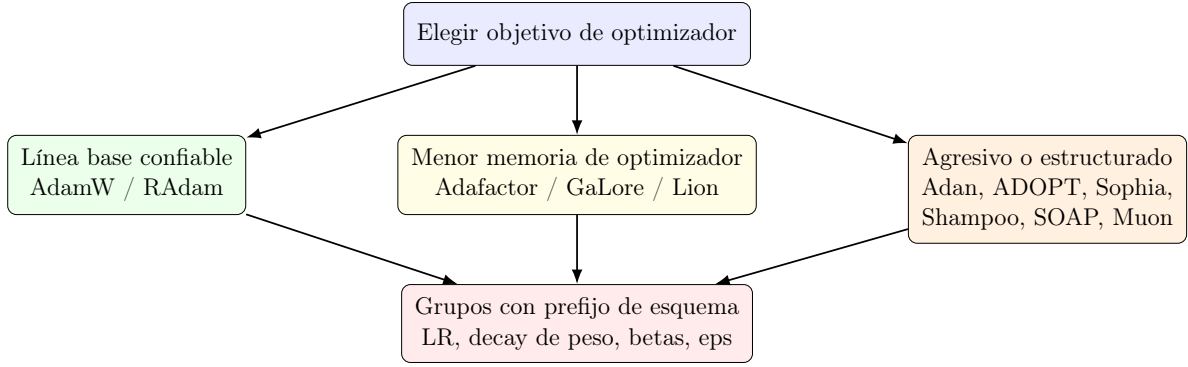


Figura 5: Selección de optimizador en la práctica: la clase determina la regla de actualización, mientras que el esquema controla cómo se aplican los hiperparámetros entre embeddings, normas, bloques recurrentes, bloques de atención y otros parámetros.

Bloque		Tipo de Estado	Mecanismo de Compuerta	Ruta Softmax	Notas de Integración Actual	Ref
Gated Net	Delta	estado recurrente matricial	compuertas de decaimiento + escritura (α, β)	No (recurrente lineal)	Olvido combinado y escritura dirigida.	[47]
RetNet	Attn	estado recurrente matricial	decaimiento multiescala fijo	No (retención)	Envoltorio alias sobre el mezclador RetNet existente.	[37]
HGRN2		estado recurrente matricial	compuerta de olvido con cota inferior	No (recurrente lineal)	Compuerta recurrente jerárquica con productos exteriores.	[29]
FoX		matriz de atención completa	compuerta de olvido en espacio de logits	Sí	Sesgo de olvido añadido antes de softmax.	[20]
Gated Softmax		matriz de atención completa	compuerta sigmoide post-atención	Sí	Compuerta sigmoide aplicada después de la salida SDPA.	[30]

5. Familias de Optimizadores y Dinámicas de Entrenamiento

La optimización de arquitecturas transformer altamente parametrizadas presenta desafíos significativos debido a paisajes de pérdida no convexos, puntos de silla y heterogeneidad de bloques entre grupos de parámetros. Este sistema aborda estos desafíos mediante un marco unificado que soporta veintitrés familias de optimizadores que abarcan seis categorías algorítmicas: (1) líneas base clásicas (SGD, AdamW), (2) momento avanzado y reducción de varianza (Adan, ADOPT, AdEMAMix, MARS, Cautious), (3) variantes eficientes en memoria (Adafactor, GaLore, Lion, APOLLO, APOLLO-Mini, Q-APOLLO), (4) métodos sin programación y sin parámetros (Schedule-Free AdamW, Prodigy) más escalado de lotes grandes mediante LAMB, (5) conscientes de curvatura y segundo orden (Sophia, Shampoo, SOAP), (6) orientados a geometría (Muon, Turbo-Muon), y (7) optimizadores con adaptabilidad ajustable (Anon). Las descripciones algorítmicas detalladas, pseudocódigo y una tabla comparativa de memoria/complejidad para todos los optimizadores se proporcionan en el Apéndice A.

Algorithm 2 Paso Adelante del Mezclador Guiado por Patrón (Conceptual)

Require: Estados ocultos H , patrón P , índice de capa ℓ

```
1:  $m \leftarrow P[\ell \bmod |P|]$ 
2: if  $m \in \{\text{fasa\_attn}, \text{sparge\_attn}\}$  y el modelo está en modo entrenamiento then
3:   lanzar error de configuración/tiempo de ejecución (bloque sin entrenamiento en modo
   entrenar)
4: else if  $m = \text{standard\_attn}$  then
5:    $H \leftarrow \text{softmax-attention}(H)$ 
6: else if  $m = \text{sigmoid\_attn}$  then
7:    $H \leftarrow \text{sigmoid-attention}(H)$ 
8: else if  $m \in \{\text{retnet}, \text{retnet\_attn}\}$  then
9:    $H \leftarrow \text{retention}(H)$ 
10: else if  $m = \text{mamba}$  then
11:    $H \leftarrow \text{selective-ssm}(H)$ 
12: else if  $m = \text{ode}$  then
13:    $H \leftarrow \text{rk-step}(H)$ 
14: else if  $m$  es una clave de atención dispersa then
15:    $H \leftarrow \text{sparse-attention-family}(H)$ 
16: else if  $m$  es una clave de atención con compuerta then
17:    $H \leftarrow \text{gated-attention-family}(H)$ 
18: else
19:    $H \leftarrow \text{memory-augmented-attn}(H)$ 
20: end if
21: return  $H$ 
```

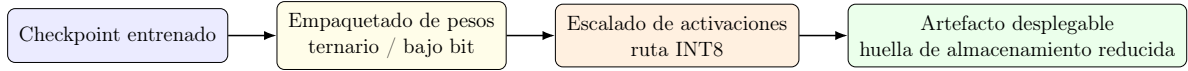


Figura 6: Ruta de despliegue desde un checkpoint entrenado hasta un artefacto compacto. El código base trata la cuantización como una transformación de la etapa de despliegue en lugar de una familia de modelos separada.

6. Cuantización y Despliegue

El stack de despliegue utiliza empaquetado ternario de pesos más cuantización de activaciones INT8 para producir artefactos eficientes.

6.1. Cuantización Ternaria

Dado el tensor de pesos W , un escalado práctico es:

$$s = \text{mean}(|W|), \quad \tilde{W} = \text{clip} \left(\text{round} \left(\frac{W}{s} \right), -1, 1 \right)$$

que aproxima las actualizaciones de bajo bit estilo BitNet [41]. El mapeo empaquetado utiliza dos bits por símbolo de peso para eficiencia de almacenamiento.

6.2. Cuantización de Activaciones

Para las activaciones x :

$$q = \text{round} \left(x \cdot \frac{127}{\text{máx}(|x|) + \epsilon} \right), \quad q \in [-128, 127]$$

con des-cuantización $x \approx q/\alpha$.

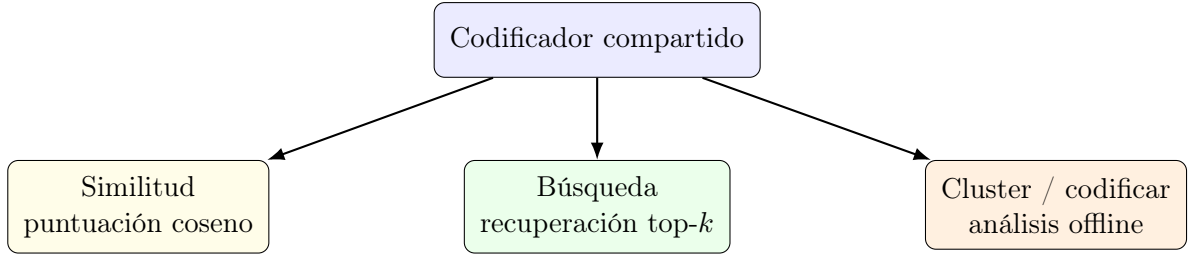


Figura 7: Reutilización del flujo de trabajo SBERT. Un único codificador soporta puntuación de pares en línea, recuperación sobre corpus, clustering y exportación persistente de embeddings.

6.3. Estimaciones de Tamaño

Para N parámetros:

$$\text{Tamaño FP32} \approx 4N, \quad \text{Tamaño FP16} \approx 2N, \quad \text{Tamaño 1.58-bit} \approx \frac{1,58}{8}N$$

antes de metadatos y sobrecarga de empaquetado. Esto se alinea con objetivos de despliegue ligero [33, 6].

7. Tareas Posteriores de SBERT

El embedding de oraciones se construye sobre entrenamiento estilo Siamese [32]. Para el par de oraciones (s_1, s_2) con embeddings (e_1, e_2) :

$$\cos(e_1, e_2) = \frac{e_1^\top e_2}{\|e_1\| \|e_2\|}$$

y pérdida coseno estilo regresión:

$$\mathcal{L}_{\cos} = (\cos(e_1, e_2) - y)^2$$

con $y \in [-1, 1]$ en este pipeline.

Modos posteriores soportados:

- **Similitud:** puntuación por pares entre dos oraciones.
- **Búsqueda:** k vecinos más cercanos sobre un corpus.
- **Clustering:** agrupación de embeddings (ej., k-means).
- **Codificación:** exportación persistente de embeddings para recuperación posterior.

8. Tablas Resumen

8.1. Resumen de Atención y Mezcladores de Secuencia

Tipo	Ecuación	Princi- pal	Entrenar	Inferir	Notas
Atención tándar	Es- $\text{softmax}(QK^\top / \sqrt{d_k})V$		$\mathcal{O}(n^2 d)$	$\mathcal{O}(n)$ /paso	Enrutamiento global expresivo de línea base [39].

Tipo		Ecuación Principal	Principio	Entrenar	Inferir	Notas
Atención moide	Sig-	$\sigma(QK^\top/\sqrt{d_k} + b)V$		$\mathcal{O}(n^2d)$	$\mathcal{O}(n)/\text{paso}$	Compuerta elemento a elemento; a menudo necesita norma de estabilización [31].
RetNet		$(QK^\top \odot D)V$		$\mathcal{O}(n^2d)$ o por fragmentos	$\mathcal{O}(1)/\text{paso}$	Forma dual paralela/recurrente con retención de decaimiento [37].
Mamba		$h_t = \bar{A}_t h_{t-1} + \bar{B}_t x_t$		$\mathcal{O}(nd)$	$\mathcal{O}(1)/\text{paso}$	Espacio de estados selectivo con barrido consciente de hardware [13].
Bloque EDO	estilo	$\frac{dh}{dt} = f_\theta(h, t)$		depende del solver	depende del solver	Interpretación de profundidad continua; integración RK [53].
Memoria tans	Ti-	$M_t = (1 - \alpha_t)M_{t-1} + S_t$		aprox. $\mathcal{O}(nd)$	centrado en recuperación	Actualizaciones de memoria en tiempo de prueba con dinámica impulsada por sorpresa [3].

8.2. Resumen de Optimizadores

Optimizador	Familia	Costo de Estado	Idea Clave	Ref
SGD+Momentum	Primer orden clásico	Bajo	Línea base acelerada por momento con estado mínimo	[28]
AdamW	Adaptativo primer/segundo momento	Alto	Línea base con decaimiento de peso desacoplado	[23]
RAdam	Adaptativo con corrección de varianza	Alto	Rectifica la varianza adaptativa temprana	[22]
Adan	Momento + reducción de varianza	Alto	Actualización adaptativa estilo Nesterov	[43]
ADOPT	Variante de Adam	Alto	Actualizaciones reordenadas con mejores garantías de convergencia	[38]
AdEMAMix	Adaptativo multi-EMA	Alto	Mezcla EMAs de horizonte corto y largo	[27]
MARS	Precondicionado con reducción de varianza	Alto	Corrección de momento recursiva	[50]
Cautious AdamW	Momento enmascarado	Alto	Aplica actualizaciones solo en direcciones consistentes con signo	[19]
LAMB	Momentos adaptativos por capa	Alto	Escalado de razón de confianza para entrenamiento con lotes muy grandes	[48]
Schedule-free AdamW	Adaptativo sin programador	Alto	Elimina la dependencia de programación LR explícita	[11]
Adafactor	Adaptativo eficiente en memoria	Medio	Segundos momentos factorizados para tensores matriciales	[34]
GaLore AdamW	Proyección de gradiente de bajo rango	Medio	Optimiza en espacio de gradiente de bajo rango proyectado	[55]
APOLLO	Proyección adaptativa de bajo rango	Medio	Escalado estructurado a partir de momentos estilo Adam proyectados	[56]

Optimizador	Familia	Costo de Estado	Idea Clave	Ref
APOLLO-Mini	Proyección adaptativa rango-1	Bajo	Variante APOLLO escalada por tensor para ahorro extremo de memoria	[56]
Q-APOLLO	Proyección de bajo rango cuantizada	Bajo	Estado APOLLO cuantizado para entrenamiento de memoria ultra-baja	[56]
Anon	Similar a Adam con adaptabilidad ajustable	Alto	Adaptabilidad $\gamma \in \mathbb{R}$ ajustable continuamente con actualización de retardo incremental (IDU) para convergencia	[1]
Prodigy	Adaptación sin parámetros	Medio	Calibración de paso adaptativa a distancia	[25]
Lion	Momento de signo	Bajo	Actualización de signo de momento, estado reducido	[8]
Sophia	Segundo orden aproximado	Medio	Precondicionamiento diagonal de Hessiana con recorte	[21]
Shampoo	Precondicionador matricial	Alto	Estadísticas de segundo orden estructuradas Kronecker	[15]
SOAP	Shampoo + base Adam	Alto	Seguimiento similar a Adam en base propia del precondicionador	[40]
Muon	Basado en ortogonalidad	Medio	Actualizaciones matriciales ortogonalizadas	[35]
Turbo-Muon	Ortogonalización acelerada	Medio	Aceleración Newton-Schulz preconditionada	[5]

9. Discusión

Las decisiones de diseño en Transformer Encoder Frankenstein reflejan varias tensiones de ingeniería e investigación en las herramientas modernas de aprendizaje profundo.

9.1. Compromisos de Diseño Basado en Esquemas

El enfoque basado en esquemas proporciona beneficios significativos de reproducibilidad al imponer contratos explícitos y fallar rápidamente ante configuraciones inválidas. Sin embargo, este enfoque también introduce rigidez: agregar nuevas arquitecturas u optimizadores requiere extensiones del esquema en lugar de argumentos de línea de comandos flexibles. El sistema de hiperparámetros prefijados permite un control detallado pero aumenta la complejidad de configuración para usuarios acostumbrados a interfaces más simples.

La decisión de imponer **additionalProperties: false** en todos los niveles del esquema elimina la absorción silenciosa de parámetros que ha afectado a sistemas de configuración anteriores, pero esta rigurosidad requiere un mantenimiento cuidadoso del esquema al extender las capacidades del sistema. Cada nuevo mecanismo de atención o variante de optimizador debe integrarse adecuadamente en el marco de validación, incluyendo definiciones de campos del esquema con tipos y restricciones apropiados, mapeo de hiperparámetros prefijados para grupos específicos de optimizadores, valores predeterminados alineados con las mejores prácticas de investigación y cadenas de documentación para la representación en la interfaz web.

9.2. Cobertura Arquitectónica y Vacíos

Las diecisiete arquitecturas de mezclador implementadas abarcan las principales direcciones de investigación en modelado de secuencias, pero ciertos vacíos permanecen. El sistema carece de arquitecturas híbridas recientes como Griffin [36] y Jamba [14], que combinan compuertas con

Algorithm 3 Enrutador de Modo de Inferencia SBERT

Require: modo m , modelo E , entradas X

```
1: if  $m = \text{similarity}$  then  
2:   retornar  $\cos(E(x_1), E(x_2))$   
3: else if  $m = \text{search}$  then  
4:   retornar top- $k$  por producto-punto/coseno contra embeddings del corpus  
5: else if  $m = \text{cluster}$  then  
6:   retornar etiquetas de clustering sobre  $E(X)$   
7: else  
8:   retornar embeddings serializados  $E(X)$   
9: end if
```

modelos de espacio de estados. El enrutamiento MoE (Mezcla de Expertos) está implementado para capas FFN pero no para el cómputo de atención, donde trabajos recientes han mostrado beneficios [18].

La cobertura de atención dispersa es completa, pero la implementación de métodos sin entrenamiento (FASA, SpargeAttn) genera errores en tiempo de ejecución durante el entrenamiento, reflejando restricciones arquitectónicas: estos métodos requieren checkpoints preentrenados de modelos de atención completa o procedimientos específicos de ajuste fino que actualmente no están automatizados.

La cobertura de mecanismos de compuerta es sólida en todas las categorías principales (GLA, DeltaNet, Gated DeltaNet, HGRN2, FoX, Gated Softmax).

9.3. Fragmentación del Panorama de Optimizadores

El soporte para veintidós familias de optimizadores en seis categorías algorítmicas demuestra exhaustividad pero también resalta el estado fragmentado de la investigación en optimización. Los usuarios enfrentan una complejidad significativa de decisión al elegir entre métodos de reducción de varianza (Adan, MARS), variantes eficientes en memoria (GaLore, Adafactor, APOLLO) y enfoques conscientes de curvatura (Sophia, Shampoo). El sistema de hiperparámetros prefijados, aunque poderoso, requiere comprender qué parámetros son relevantes para cada clase de optimizador.

La calidad de implementación varía entre optimizadores: los métodos clásicos (AdamW, SGD con momento) están altamente optimizados en PyTorch, mientras que los métodos más nuevos (Muon, Turbo-Muon, SOAP) pueden requerir implementaciones personalizadas que afectan la estabilidad numérica y las características de rendimiento.

9.4. Consideraciones de Despliegue y Producción

El pipeline de cuantización demuestra preocupaciones prácticas de despliegue pero realiza compromisos de ingeniería específicos. El empaquetado ternario de pesos reduce el almacenamiento a aproximadamente 1,58 bits por parámetro, pero esta compresión agresiva puede degradar el rendimiento, especialmente para modelos más pequeños donde el error de cuantización es más significativo. La implementación actual aplica cuantización uniforme entre todos los tipos de parámetros.

Los flujos de trabajo SBERT proporcionan utilidad práctica para tareas de similitud semántica y recuperación, pero la implementación asume estrategias de pooling estándar (token CLS, pooling promedio). Avances recientes como los embeddings Matryoshka [26] y los refinamientos de aprendizaje contrastivo [17] aún no están incorporados.

9.5. Desafíos de Integración y Extensibilidad

La estructura actual del código base, aunque funcional, presenta desafíos de mantenimiento a medida que se expanden las familias de arquitecturas y optimizadores. El patrón de despachador para la selección de mezcladores y el enrutamiento de optimizadores maneja la extensibilidad pero corre el riesgo de convertirse en un “cajón de sastre” de lógica condicional. Las versiones futuras se beneficiarían de arquitecturas basadas en plugins donde nuevos mezcladores y optimizadores puedan registrarse de forma declarativa en lugar de modificar la lógica central de despacho.

La interfaz de configuración web proporciona mejoras significativas de usabilidad pero introduce complejidad de despliegue: ejecutar Streamlit junto con trabajos de entrenamiento requiere recursos adicionales y consideraciones de infraestructura que pueden no ser apropiadas para todos los entornos, particularmente clústeres HPC sin acceso web.

10. Conclusión

Transformer Encoder Frankenstein presenta una plataforma de experimentación unificada, impulsada por configuración, que aborda desafíos críticos en la investigación moderna de aprendizaje profundo: fragmentación arquitectónica entre atención densa, modelos recurrentes, patrones dispersos y mecanismos de compuerta; complejidad del panorama de optimizadores que abarca líneas base clásicas, métodos de reducción de varianza, variantes eficientes en memoria, enfoques sin programación, algoritmos conscientes de curvatura y métodos orientados a geometría; y flujos de trabajo de despliegue de extremo a extremo que abarcan cuantización y aplicaciones de embeddings de oraciones.

Las contribuciones principales del sistema son:

1. **Diseño Basado en Esquemas:** Un contrato de configuración estricto basado en YAML con validación y enrutamiento de hiperparámetros prefijados que permite experimentos reproducibles en diecisiete arquitecturas de mezclador y veintitrés familias de optimizadores.
2. **Soporte Arquitectónico Integral:** Implementación que abarca las principales categorías de investigación incluyendo líneas base densas (atención estándar, sigmoide), alternativas recurrentes (RetNet, Mamba, estilo EDO, Titans), atención dispersa (Sparse Transformer, Longformer, BigBird, SparseK, NSA, SpargeAttn, FASA) y mecanismos de compuerta (GLA, DeltaNet, Gated DeltaNet, HGRN2, FoX, Gated Softmax).
3. **Marco Unificado de Optimizadores:** Grupos de hiperparámetros prefijados que permiten control detallado sobre embeddings, capas de normalización, bloques recurrentes, pesos de atención y parámetros FFN en líneas base clásicas (SGD+Momentum, AdamW), reducción de varianza (Adan, ADOPT, AdEMAMix, MARS, Cautious), eficientes en memoria (Adafactor, GaLore, Lion, APOLLO, APOLLO-Mini, Q-APOLLO), lotes grandes y simplificación de programación (LAMB, Schedule-Free AdamW, Prodigy), conscientes de curvatura (Sophia), segundo orden (Shampoo, SOAP) y orientados a geometría (Muon, Turbo-Muon).
4. **Flujos de Trabajo de Extremo a Extremo:** Pipeline de despliegue integrado que soporta empaquetado ternario de pesos y cuantización de activaciones INT8; entrenamiento e inferencia inspirados en SBERT para similitud semántica, recuperación y tareas de clustering.
5. **Configuración Interactiva:** Interfaz web basada en Streamlit que proporciona generación de formularios impulsada por esquemas, validación en tiempo real, documentación en línea y síntesis de comandos CLI.

Este sistema permite iteración experimental rápida mientras mantiene reproducibilidad mediante contratos de configuración estrictos. Al consolidar diversas contribuciones de investigación

en un conjunto de herramientas unificado, reduce las barreras para explorar arquitecturas y estrategias de optimización novedosas, particularmente para investigadores que pueden carecer de recursos para implementar y validar cada variante de forma independiente.

10.1. Limitaciones y Direcciones Futuras

Varias limitaciones y direcciones prometedoras para trabajo futuro surgen del diseño e implementación de este sistema:

1. **Integración Arquitectónica:** Arquitecturas híbridas recientes (Griffin, Jamba, Mamba-X) demuestran beneficios de combinar múltiples mecanismos en bloques unificados. Versiones futuras deberían integrar estas arquitecturas y explorar patrones de composición sistemática.
2. **Cuantización Avanzada:** La implementación actual utiliza empaquetado ternario uniforme en todos los parámetros. La investigación sobre cuantización por capas, por canales y consciente de importancia sugiere que estrategias más sofisticadas podrían mejorar los compromisos calidad-eficiencia.
3. **Extensibilidad Basada en Plugins:** El patrón de despacho actual se vuelve cada vez más complejo con cada nueva adición. Una arquitectura de plugins que permita el registro declarativo de nuevos mezcladores, optimizadores y métodos de normalización mejoraría el mantenimiento y reduciría el riesgo de errores en la lógica central de despacho.
4. **Optimización Automatizada de Hiperparámetros:** El esquema soporta espacios extensos de hiperparámetros, pero los usuarios deben explorar estos espacios manualmente. La integración con optimización bayesiana, estrategias de bandidos multi-brazo o ajuste de hiperparámetros basado en gradientes podría automatizar el descubrimiento de configuraciones efectivas.
5. **Despliegue en Producción:** La interfaz web mejora la usabilidad pero puede no ser apropiada para todos los entornos de despliegue. Modos de configuración sin interfaz gráfica, gestión de configuración basada en API o ergonomía CLI mejorada podrían servir a flujos de trabajo HPC y de producción.
6. **Evaluación Comparativa:** Si bien el sistema permite entrenamiento con diversas arquitecturas, una evaluación comparativa exhaustiva que compare el rendimiento entre mezcladores y optimizadores en tareas estandarizadas proporcionaría orientación valiosa para la selección de configuraciones.
7. **Garantías de Estabilidad de Entrenamiento:** La implementación actual incluye guardas contra NaN/Inf y recorte de gradiente, pero el análisis formal de condiciones de estabilidad para diferentes combinaciones mezclador-optimizador, particularmente con bloques en bucle y cuantización agresiva, sigue abierto.
8. **Extensiones Multimodales y Específicas de Tarea:** El diseño actual se centra en modelado de secuencias. Extensiones para modelos de visión-lenguaje, arquitecturas multimodales y flujos de trabajo de ajuste fino específicos de tarea (ej., ajuste por instrucciones, RLHF) ampliarían la aplicabilidad.

La trayectoria de investigación del modelado de secuencias continúa hacia enfoques híbridos que combinan fortalezas de múltiples paradigmas—compresión de la recurrencia, selectividad de la atención, compuertas para gestión de memoria y dispersión para eficiencia. Una plataforma de experimentación unificada como Transformer Encoder Frankenstein es cada vez más valiosa a medida que esta convergencia se acelera, permitiendo a los investigadores explorar sistemáticamente este creciente espacio de diseño con infraestructura reproducible y bien diseñada.

Bibliografia

- [1] Anonymous authors. Anon: Extrapolating adaptivity beyond sgd and adam, 2026. URL <https://arxiv.org/abs/2605.02317>.
- [2] Ali Behrouz, Peilin Zhong, and Vahab Mirrokni. Titans: Learning to memorize at test time, . URL <http://arxiv.org/abs/2501.00663>.
- [3] Ali Behrouz, Peilin Zhong, and Vahab Mirrokni. Titans: Learning to memorize at test time, . URL <https://arxiv.org/abs/2501.00663>. Version Number: 1.
- [4] Iz Beltagy, Matthew E. Peters, and Arman Cohan. Longformer: The long-document transformer, 2020. URL <https://arxiv.org/abs/2004.05150>.
- [5] Thibaut Boissin, Thomas Massena, Franck Mamalet, and Mathieu Serrurier. Turbo-muon: Accelerating orthogonality-based optimization with pre-conditioning. URL <https://arxiv.org/abs/2512.04632>. Version Number: 1.
- [6] Riccardo Bravin, Massimo Pavan, Hazem Hesham Yousef Shalby, Fabrizio Pittorino, and Manuel Roveri. EmbBERT: Attention under 2 MB memory. URL <http://arxiv.org/abs/2502.10001>.
- [7] Mingzhi Chen, Taiming Lu, Jiachen Zhu, Mingjie Sun, and Zhuang Liu. Stronger normalization-free transformers, . URL <http://arxiv.org/abs/2512.10938>.
- [8] Xiangning Chen, Chen Liang, Da Huang, Esteban Real, Kaiyuan Wang, Yao Liu, Hieu Pham, Xuanyi Dong, Thang Luong, Cho-Jui Hsieh, Yifeng Lu, and Quoc V. Le. Symbolic discovery of optimization algorithms, . URL <https://arxiv.org/abs/2302.06675>. Version Number: 4.
- [9] Xin Cheng, Wangding Zeng, Damai Dai, Qinyu Chen, Bingxuan Wang, Zhenda Xie, Kezhao Huang, Xingkai Yu, Zhewen Hao, Yukun Li, Han Zhang, Huishuai Zhang, Dongyan Zhao, and Wenfeng Liang. Conditional memory via scalable lookup: A new axis of sparsity for large language models, 2026. URL <https://arxiv.org/abs/2601.07372>.
- [10] Rewon Child, Scott Gray, Alec Radford, and Ilya Sutskever. Generating long sequences with sparse transformers, 2019. URL <https://arxiv.org/abs/1904.10509>.
- [11] Aaron Defazio, Xingyu Alice Yang, Harsh Mehta, Konstantin Mishchenko, Ahmed Khaled, and Ashok Cutkosky. The road less scheduled. URL <https://arxiv.org/abs/2405.15682>. Version Number: 4.
- [12] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. URL <http://arxiv.org/abs/1810.04805>.
- [13] Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. URL <https://arxiv.org/abs/2312.00752>. Version Number: 2.
- [14] Albert Gu, AI21 Labs, et al. Jamba: A hybrid transformer-mamba language model, 2024. URL <https://arxiv.org/abs/2403.19887>.
- [15] Vineet Gupta, Tomer Koren, and Yoram Singer. Shampoo: Preconditioned stochastic tensor optimization. URL <https://arxiv.org/abs/1802.09568>. Version Number: 2.
- [16] Sukjun Hwang, Aakash Lahoti, Tri Dao, and Albert Gu. Hydra: Bidirectional state space models through generalized matrix mixers. URL <http://arxiv.org/abs/2407.09941>.

- [17] Prannay Khosla, Piotr Teterwak, Chen Wang, Aaron Sarna, Yonglong Tian, Phillip Isola, Aaron Maschinot, Ce Liu, and Dilip Krishnan. Supervised contrastive learning, 2020. URL <https://arxiv.org/abs/2004.11362>.
- [18] Mike Lewis, Shruti Bhosale, Tim Dettmers, Douwe Kiela, and Luke Zettlemoyer. Base layers: Simplifying training of large, sparse models, 2021. URL <https://arxiv.org/abs/2103.16716>.
- [19] Kaizhao Liang, Lizhang Chen, Bo Liu, and Qiang Liu. Cautious optimizers: Improving training with one line of code. URL <https://arxiv.org/abs/2411.16085>. Version Number: 4.
- [20] Zhixuan Lin, Ke Wang, et al. Forgetting transformer: Softmax attention with a forget gate, 2025. URL <https://arxiv.org/abs/2503.02130>.
- [21] Hong Liu, Zhiyuan Li, David Hall, Percy Liang, and Tengyu Ma. Sophia: A scalable stochastic second-order optimizer for language model pre-training, . URL <https://arxiv.org/abs/2305.14342>. Version Number: 4.
- [22] Liyuan Liu, Haoming Jiang, Pengcheng He, Weizhu Chen, Xiaodong Liu, Jianfeng Gao, and Jiawei Han. On the variance of the adaptive learning rate and beyond, . URL <https://arxiv.org/abs/1908.03265>. Version Number: 4.
- [23] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. URL <https://arxiv.org/abs/1711.05101>. Version Number: 3.
- [24] Tianyu Lou, Zheyu Chen, Tao Yu, et al. Efficient sparse attention for long-range transformers, 2024. URL <https://arxiv.org/abs/2406.16747>.
- [25] Konstantin Mishchenko and Aaron Defazio. Prodigy: An expeditiously adaptive parameter-free learner. URL <https://arxiv.org/abs/2306.06101>. Version Number: 4.
- [26] Niklas Muennighoff et al. Matryoshka representation learning, 2022. URL <https://arxiv.org/abs/2205.13147>.
- [27] Matteo Pagliardini, Pierre Ablin, and David Grangier. The AdEMAMix optimizer: Better, faster, older. URL <https://arxiv.org/abs/2409.03137>. Version Number: 2.
- [28] Boris T. Polyak. Some methods of speeding up the convergence of iteration methods. *USSR Computational Mathematics and Mathematical Physics*, 4(5):1–17, 1964. doi: 10.1016/0041-5553(64)90137-5.
- [29] Zhen Qin, Xu Han, et al. Hgrn2: Gated linear rnns with state expansion, 2024. URL <https://arxiv.org/abs/2404.07904>.
- [30] Yuxiang Qiu, Qwen Team, et al. Gated attention for large language models, 2025. URL <https://arxiv.org/abs/2505.06708>.
- [31] Jason Ramapuram, Federico Danieli, Eeshan Dhekane, Floris Weers, Dan Busbridge, Pierre Ablin, Tatiana Likhomanenko, Jagrit Digani, Zijin Gu, Amitis Shidani, and Russ Webb. Theory, analysis, and best practices for sigmoid self-attention. URL <https://arxiv.org/abs/2409.04431>. Version Number: 2.
- [32] Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using siamese BERT-networks. URL <http://arxiv.org/abs/1908.10084>.
- [33] Hema Hariharan Samson. Lightweight transformer architectures for edge devices in real-time applications. URL <http://arxiv.org/abs/2601.03290>.

- [34] Noam Shazeer and Mitchell Stern. Adafactor: Adaptive learning rates with sublinear memory cost. URL <https://arxiv.org/abs/1804.04235>. Version Number: 1.
- [35] Wei Shen, Ruichuan Huang, Minhui Huang, Cong Shen, and Jiawei Zhang. On the convergence analysis of muon. URL <https://arxiv.org/abs/2505.23737>. Version Number: 1.
- [36] Rafael Soares et al. Griffin: Mixing gated linear recurrences with local attention for efficient sequence modeling, 2024. URL <https://arxiv.org/abs/2402.19427>.
- [37] Yutao Sun, Li Dong, Shaohan Huang, Shuming Ma, Yuqing Xia, Jilong Xue, Jianyong Wang, and Furu Wei. Retentive network: A successor to transformer for large language models. URL <https://arxiv.org/abs/2307.08621>. Version Number: 4.
- [38] Shohei Taniguchi, Keno Harada, Gouki Minegishi, Yuta Oshima, Seong Cheol Jeong, Go Nagahara, Tomoshi Iiyama, Masahiro Suzuki, Yusuke Iwasawa, and Yutaka Matsuo. ADOPT: Modified adam can converge with any β_2 with the optimal rate. URL <https://arxiv.org/abs/2411.02853>. Version Number: 3.
- [39] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. URL <https://arxiv.org/abs/1706.03762>. Version Number: 7.
- [40] Nikhil Vyas, Depen Morwani, Rosie Zhao, Mujin Kwun, Itai Shapira, David Brandfonbrener, Lucas Janson, and Sham Kakade. SOAP: Improving and stabilizing shampoo using adam. URL <https://arxiv.org/abs/2409.11321>. Version Number: 2.
- [41] Hongyu Wang, Shuming Ma, Li Dong, Shaohan Huang, Huaijie Wang, Lingxiao Ma, Fan Yang, Ruiping Wang, Yi Wu, and Furu Wei. BitNet: Scaling 1-bit transformers for large language models. URL <http://arxiv.org/abs/2310.11453>.
- [42] Zhe Wang, Ming Liu, et al. Fasa: Frequency-aware sparse attention, 2026. URL <https://arxiv.org/abs/2602.03152>.
- [43] Xingyu Xie, Pan Zhou, Huan Li, Zhouchen Lin, and Shuicheng Yan. Adan: Adaptive nesterov momentum algorithm for faster optimizing deep models. URL <https://arxiv.org/abs/2208.06677>. Version Number: 5.
- [44] Haiqi Yang, Zhiyuan Li, Yi Chang, and Yuan Wu. A survey of retentive network. URL <http://arxiv.org/abs/2506.06708>.
- [45] Songlin Yang, Bailin Wang, et al. Gated linear attention transformers with hardware-efficient training, 2023. URL <https://arxiv.org/abs/2312.06635>.
- [46] Songlin Yang, Bailin Wang, et al. Parallelizing linear transformers with the delta rule over sequence length, 2024. URL <https://arxiv.org/abs/2406.06484>.
- [47] Songlin Yang, Bailin Wang, et al. Gated delta networks: Improving mamba2 with delta rule, 2024. URL <https://arxiv.org/abs/2412.06464>.
- [48] Yang You, Jing Li, Sashank Reddi, Jonathan Hseu, Sanjiv Kumar, Srinadh Bhojanapalli, Xiaodan Song, James Demmel, and Cho-Jui Hsieh. Large batch optimization for deep learning: Training BERT in 76 minutes. URL <https://arxiv.org/abs/1904.00962>. Version Number: 3.
- [49] Han Yuan, DeepSeek-AI, et al. Native sparse attention: Hardware-aligned and natively trainable sparse attention, 2025. URL <https://arxiv.org/abs/2502.11089>.

- [50] Huizhuo Yuan, Yifeng Liu, Shuang Wu, Xun Zhou, and Quanquan Gu. MARS: Unleashing the power of variance reduction for training large models. URL <https://arxiv.org/abs/2411.10438>. Version Number: 4.
- [51] Manzil Zaheer, Guru Guruganesh, Kumar Avinava Dubey, Joshua Ainslie, Chris Alberti, Santiago Ontanon, Pike Pham, Anirudh Ravula, Qifan Wang, Li Yang, and Amr Ahmed. Big bird: Transformers for longer sequences. In *Advances in Neural Information Processing Systems*, 2020. doi: 10.48550/ARXIV.2007.14062. URL <https://arxiv.org/abs/2007.14062>.
- [52] Biao Zhang and Rico Sennrich. Root Mean Square Layer Normalization. URL <http://arxiv.org/abs/1910.07467>.
- [53] Jing Zhang, Peng Zhang, Baiwen Kong, Junqiu Wei, and Xin Jiang. Continuous self-attention models with neural ODE networks. 35(16):14393–14401. ISSN 2374-3468, 2159-5399. doi: 10.1609/aaai.v35i16.17692. URL <https://ojs.aaai.org/index.php/AAAI/article/view/17692>.
- [54] Yichi Zhang, Yizhong Wang, et al. Accurate and training-free sparse attention accelerating any model inference, 2025. URL <https://arxiv.org/abs/2502.18137>.
- [55] Jiawei Zhao, Zhenyu Zhang, Beidi Chen, Zhangyang Wang, Anima Anandkumar, and Yuan-dong Tian. GaLore: Memory-efficient LLM training by gradient low-rank projection. URL <https://arxiv.org/abs/2403.03507>. Version Number: 2.
- [56] Hanqing Zhu, Zhenyu Zhang, Wenyan Cong, Xi Liu, Sem Park, Vikas Chandra, Bo Long, David Z. Pan, Zhangyang Wang, and Jinwon Lee. Apollo: Sgd-like memory, adamw-level performance, 2025. URL <https://arxiv.org/abs/2412.05270>.
- [57] Jiachen Zhu, Xinlei Chen, Kaiming He, Yann LeCun, and Zhuang Liu. Transformers without normalization. URL <http://arxiv.org/abs/2503.10622>.
- [58] Lianghui Zhu, Yuxin Fang, Bencheng Liao, Shijie Wang, Tianheng Cheng, Zilong Huang, Chen Chen, Lai Wei, Yutao Zeng, Ya Wang, Yi Lin, Yu Li, and Xinggang Wang. Mixture-of-depths attention, 2026. URL <https://arxiv.org/abs/2603.15619>.

A. Anexo A: Familias de Optimizadores

A.1. Comparación de Memoria y Complejidad

La Tabla 8 resume la sobrecarga de memoria (número de búferes de estado por parámetro), la complejidad computacional por paso y los hiperparámetros clave para todos los optimizadores soportados. La sobrecarga de memoria se expresa en términos del número de tensores de estado mantenidos por parámetro del modelo, donde cada tensor tiene la misma forma que el parámetro.

Optimizador	Búferes de Estado	Costo por Paso	Hiperparámetros Clave
SGD+Momentum	1 (m)	$\mathcal{O}(n)$	lr, momentum, wd
AdamW	2 (m, v)	$\mathcal{O}(n)$	lr, β_1, β_2 , eps, wd
RAdam	2 (m, v)	$\mathcal{O}(n)$	lr, β_1, β_2 , eps, wd
Adan	3 (m, v, s)	$\mathcal{O}(n)$	lr, $\beta_1, \beta_2, \beta_3$, eps, wd
ADOPT	2 (m, v)	$\mathcal{O}(n)$	lr, β_1, β_2 , eps, wd
AdEMAMix	3 (m_1, m_2, v)	$\mathcal{O}(n)$	lr, $\beta_1, \beta_2, \beta_3$, eps, wd

continúa

Optimizador	Búferes de Estado	Costo por Paso	Hiperparámetros Clave
MARS	3 (m, v, z)	$\mathcal{O}(n)$	lr, β_1, β_2 , eps, wd, γ
Cautious AdamW	2 (m, v)	$\mathcal{O}(n)$	lr, β_1, β_2 , eps, wd
LAMB	2 (m, v)	$\mathcal{O}(n)$	lr, β_1, β_2 , eps, wd
Schedule-Free	3 (z, x, n)	$\mathcal{O}(n)$	lr, β_1, β_2 , wd
Adafactor	1–2 (row/col)	$\mathcal{O}(n)$	lr, β_1, β_2 , eps, wd, factored
GaLore	2 (m, v) + SVD/proj	$\mathcal{O}(nr)$	lr, rank r , β_1, β_2 , eps, wd
Prodigy	3 (m, v, d)	$\mathcal{O}(n)$	β_1, β_2 , eps, wd, d_0
Lion	1 (m)	$\mathcal{O}(n)$	lr, β_1, β_2 , wd
Shampoo	$2d (L_i, R_i)$	$\mathcal{O}(n^{1+2/d})$	lr, eps, wd, matrix eps
SOAP	$2d + 2 (m, v)$	$\mathcal{O}(n^{1+1/d})$	lr, β_1, β_2 , eps, wd, shampoo eps
Sophia	3 (m, v, h)	$\mathcal{O}(n)$	lr, β_1, β_2 , eps, wd, k
Muon	2 (m, v)	$\mathcal{O}(n \cdot k)$	lr, momentum, wd, NS steps k
Turbo-Muon	2 (m, v)	$\mathcal{O}(n \cdot k)$	lr, momentum, wd, NS steps k
APOLLO	2 bajo rango (m_R, v_R) + proy	$\mathcal{O}(nr)$	lr, rank r , update gap, scale, beta
APOLLO-Mini	2 rango-1 (m_R, v_R) + proy	$\mathcal{O}(n)$	lr, update gap, scale, betas, eps,
Q-APOLLO	2 bajo rango cuantizados + proy	$\mathcal{O}(nr)$	lr, rank r , update gap, scale, qua
Anon	3 (m, v , fixed_lr)	$\mathcal{O}(n)$	lr, β_1, β_2 , eps, wd, γ

Cuadro 8: Comparación de memoria y complejidad de todos los optimizadores soportados. n = número de parámetros, d = dimensionalidad del tensor, r = dimensión de rango bajo, k = número de iteraciones de Newton–Schulz. Búferes de estado indica el número de tensores de estado del optimizador mantenidos por grupo de parámetros. El costo por paso se enfoca en el costo adicional dominante más allá del propio cálculo del gradiente.

A.2. La Evolución de la Optimización en Redes Neuronales

El estudio de optimizadores enmarca la optimización de transformadores como una respuesta a tres presiones estructurales: paisajes de pérdida no convexos, heterogeneidad severa de curvatura entre bloques de parámetros y el costo de memoria de almacenar el estado del optimizador para modelos muy grandes. El informe sostiene que el campo se ha diversificado en varias trayectorias: líneas base adaptativas de primer orden, métodos de reducción de varianza, métodos eficientes en memoria, preconditionadores estructurados de segundo orden, métodos sin programación de tasa de aprendizaje y actualizaciones orientadas a ortogonalidad.

A.3. Línea Base Estándar y Optimizadores Adaptativos

SGD con Momentum. La actualización clásica acumula un búfer de momentum y luego aplica una tasa de aprendizaje fija. En cada iteración t , el algoritmo mantiene un promedio móvil exponencial m_t de los gradientes pasados:

$$m_t = \beta m_{t-1} + g_t \quad (1)$$

$$\theta_{t+1} = \theta_t - \eta m_t \quad (2)$$

donde $g_t = \nabla f(\theta_t)$, $\beta \in [0, 0.99]$ controla la decadencia del momentum y η es la tasa de aprendizaje fija. El término de momentum actúa como velocidad: acelera el movimiento en direcciones consistentes mientras amortigua oscilaciones en direcciones variables. Sus fortalezas son la baja sobrecarga de memoria (un único búfer de momentum por parámetro) y una fuerte

generalización cuando se ajusta cuidadosamente. Su principal debilidad en cargas de trabajo de transformadores es la escasa robustez frente a la heterogeneidad de curvatura (diferentes espectros del Hessiano entre grupos de parámetros) y una fuerte dependencia de las programaciones de tasa de aprendizaje.

Algorithm 4 SGD con Momentum

Require: Parámetros iniciales θ_0 , tasa de aprendizaje η , coeficiente de momentum β , decadencia de peso λ

Ensure: Parámetros actualizados θ_T

```

1: Inicializar: búfer de momentum  $m \leftarrow 0$ 
2: for  $t = 0, 1, 2, \dots, T - 1$  do
3:   Calcular gradiente:  $g_t \leftarrow \nabla f(\theta_t)$ 
4:   if  $\text{decadencia\_de\_peso} > 0$  then
5:      $g_t \leftarrow g_t + \lambda \theta_t$  ▷ Regularización L2
6:   end if
7:   Actualizar momentum:  $m \leftarrow \beta \cdot m + g_t$ 
8:   Actualizar parámetros:  $\theta_{t+1} \leftarrow \theta_t - \eta \cdot m$ 
9: end for return  $\theta_T$ 

```

Adam y AdamW. Adam rastrea promedios móviles exponenciales tanto del primer momento (media) como del segundo momento (varianza no centrada) para lograr tasas de aprendizaje adaptativas elemento a elemento. La regla de actualización es:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \quad (3)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \quad (4)$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t} \quad (\text{corrección de sesgo}) \quad (5)$$

$$\theta_{t+1} = \theta_t - \eta \left(\frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} \right) \quad (6)$$

AdamW introduce una modificación crucial: la decadencia de peso se aplica directamente a los parámetros (desacoplada de los gradientes): $\theta_t \leftarrow \theta_t(1 - \eta\lambda)$ en lugar de añadir $\lambda\theta_t$ al gradiente. Este desacoplamiento evita que el escalado adaptativo interfiera con la fuerza de la regularización. El informe trata a AdamW como la línea base práctica para el ajuste fino de transformadores porque converge rápidamente y es relativamente tolerante a la variación de hiperparámetros. La desventaja es el costo de memoria: ambos tensores de momento deben almacenarse para cada parámetro, duplicando la memoria del estado del optimizador en relación con SGD.

Algorithm 5 AdamW (Adam con Decadencia de Peso Desacoplada)

Require: Parámetros iniciales θ_0 , tasa de aprendizaje η , tasas de decadencia exponencial $\beta_1, \beta_2 \in [0, 1)$

Require: Constante de momentum ϵ , decadencia de peso λ

Ensure: Parámetros actualizados θ_T

- 1: Inicializar: primer momento $m \leftarrow 0$, segundo momento $v \leftarrow 0$, contador de pasos $t \leftarrow 0$
 - 2: **for** $t = 1, 2, \dots, T$ **do**
 - 3: Calcular gradiente: $g_t \leftarrow \nabla f(\theta_{t-1})$
 - 4: Decadencia de peso (desacoplada): $\theta_{t-1} \leftarrow \theta_{t-1}(1 - \eta\lambda)$
 - 5: Actualizar momentos: $m \leftarrow \beta_1 m + (1 - \beta_1)g_t$
 - 6: $v \leftarrow \beta_2 v + (1 - \beta_2)g_t^2$
 - 7: Corrección de sesgo: $\hat{m} \leftarrow m/(1 - \beta_1^t)$, $\hat{v} \leftarrow v/(1 - \beta_2^t)$
 - 8: Actualizar parámetros: $\theta_t \leftarrow \theta_{t-1} - \eta(\hat{m}/(\sqrt{\hat{v}} + \epsilon))$
 - 9: **end for** **return** θ_T
-

RAdam (Adam Rectificado). RAdam aborda la inestabilidad de Adam en el entrenamiento temprano rectificando dinámicamente la tasa de aprendizaje adaptativa. La observación clave es que el segundo momento v_t tiene una varianza muy alta en los primeros pasos, causando un escalado adaptativo poco fiable. RAdam calcula la longitud efectiva de la ventana de promedio móvil simple (SMA):

$$\rho_t = \rho_\infty - \frac{2t\beta_2^t}{1 - \beta_2^t}, \quad \text{donde} \quad \rho_\infty = \frac{2}{1 - \beta_2} - 1$$

Cuando $\rho_t > 4$ (suficientes muestras para la estimación de varianza), RAdam aplica el escalado adaptativo con un término de rectificación:

$$r_t = \sqrt{\frac{(\rho_t - 4)(\rho_t - 2)\rho_\infty}{(\rho_\infty - 4)(\rho_\infty - 2)\rho_t}}, \quad \theta_{t+1} = \theta_t - \eta r_t \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

Cuando $\rho_t \leq 4$, RAdam retrocede a SGD con momentum: $\theta_{t+1} = \theta_t - \eta \hat{m}_t$. Esta transición gradual elimina la necesidad de programaciones manuales de calentamiento de la tasa de aprendizaje y mejora la robustez.

Algorithm 6 RAdam (Adam Rectificado)

Require: Parámetros iniciales θ_0 , tasa de aprendizaje η , β_1, β_2 , decadencia de peso λ

Ensure: Parámetros actualizados θ_T

```
1: Inicializar:  $m \leftarrow 0, v \leftarrow 0, t \leftarrow 0$ 
2: Calcular:  $\rho_\infty \leftarrow 2/(1 - \beta_2) - 1$ 
3: for  $t = 1, 2, \dots, T$  do
4:    $g_t \leftarrow \nabla f(\theta_{t-1})$ 
5:   if decadencia_de_peso  $> 0$  then
6:      $\theta_{t-1} \leftarrow \theta_{t-1}(1 - \eta\lambda)$ 
7:   end if
8:    $m \leftarrow \beta_1 m + (1 - \beta_1)g_t$ 
9:    $v \leftarrow \beta_2 v + (1 - \beta_2)g_t^2$ 
10:   $\hat{m} \leftarrow m/(1 - \beta_1^t), \hat{v} \leftarrow v/(1 - \beta_2^t)$ 
11:   $\rho_t \leftarrow \rho_\infty - 2t\beta_2^t/(1 - \beta_2^t)$ 
12:  if  $\rho_t > 4$  then
13:     $r_t \leftarrow \sqrt{((\rho_t - 4)(\rho_t - 2)\rho_\infty)/((\rho_\infty - 4)(\rho_\infty - 2)\rho_t)}$ 
14:     $\theta_t \leftarrow \theta_{t-1} - \eta r_t \hat{m}/(\sqrt{\hat{v}} + \epsilon)$ 
15:  else
16:     $\theta_t \leftarrow \theta_{t-1} - \eta \hat{m}$  ▷ SGD con momentum
17:  end if
18: end for return  $\theta_T$ 
```

A.4. Momentum Avanzado y Reducción de Varianza (2024–2025)

Adan (Momentum Nesterov Adaptativo). Adan reformula la aceleración de Nesterov sin el cálculo de gradiente adicional requerido por el SGD Nesterov clásico. El algoritmo mantiene tres búferes de momentum:

$$m_t = (1 - \beta_1)m_{t-1} + \beta_1 g_t \quad (\text{primer momento}) \quad (7)$$

$$v_t = (1 - \beta_2)v_{t-1} + \beta_2(g_t - g_{t-1}) \quad (\text{velocidad/diferencia de gradientes}) \quad (8)$$

$$n_t = (1 - \beta_3)n_{t-1} + \beta_3[g_t + (1 - \beta_1)(g_t - g_{t-1})]^2 \quad (\text{segundo momento Nesterov}) \quad (9)$$

El término de **Estimación de Momentum Nesterov** (NME) $\bar{g}_t = g_t + (1 - \beta_1)(g_t - g_{t-1})$ estima el gradiente en una posición futura sin evaluarlo. La actualización combina el momentum con la velocidad:

$$\bar{m}_t = m_t + (1 - \beta_1)v_t, \quad \theta_{t+1} = \theta_t - \eta \frac{\bar{m}_t}{\sqrt{n_t} + \epsilon}$$

Adan logra una convergencia rápida en diversas arquitecturas (CNN, GAN, Transformadores) mediante esta aceleración. El costo es mantener tres búferes similares al momentum, aumentando la sobrecarga de memoria en relación con Adam.

Algorithm 7 Adan (Momentum Nesterov Adaptativo)

Require: Parámetros iniciales θ_0 , tasa de aprendizaje η , $\beta_1, \beta_2, \beta_3 \in (0, 1)$

Ensure: Parámetros actualizados θ_T

```
1: Inicializar:  $m \leftarrow 0$ ,  $v \leftarrow 0$ ,  $n \leftarrow 0$ ,  $g_{-1} \leftarrow 0$  (gradiente anterior)
2: for  $t = 1, 2, \dots, T$  do
3:    $g_t \leftarrow \nabla f(\theta_{t-1})$ 
4:    $m \leftarrow (1 - \beta_1)m + \beta_1 g_t$ 
5:    $\Delta g_t \leftarrow g_t - g_{t-1}$ 
6:    $v \leftarrow (1 - \beta_2)v + \beta_2 \Delta g_t$ 
7:   Estimación Nesterov:  $\bar{g}_t \leftarrow g_t + (1 - \beta_1)\Delta g_t$ 
8:    $n \leftarrow (1 - \beta_3)n + \beta_3 \bar{g}_t^2$ 
9:   Momentum combinado:  $\bar{m} \leftarrow m + (1 - \beta_1)v$ 
10:   $\theta_t \leftarrow \theta_{t-1} - \eta(\bar{m}/(\sqrt{n} + \epsilon))$ 
11:   $g_{t-1} \leftarrow g_t$ 
12: end for return  $\theta_T$ 
```

ADOPT (Adam con Ajuste Óptimo Podado). ADOPT corrige un problema teórico fundamental en Adam: el gradiente aparece tanto en la estimación del primer momento como en la del segundo momento, creando circularidad. ADOPT **desacopla** utilizando el segundo momento del paso anterior para el denominador:

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \quad (10)$$

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) \frac{g_t}{\sqrt{v_{t-1}} + \epsilon} \quad (\text{usa } v_{t-1}, \text{ no } v_t) \quad (11)$$

$$\theta_{t+1} = \theta_t - \eta m_t \quad (12)$$

Este simple reordenamiento logra la tasa de convergencia óptima $O(1/\sqrt{T})$ con **cualquier** elección de $\beta_2 \in (0, 1)$, sin suposiciones de ruido acotado. La consecuencia práctica es que ADOPT es un reemplazo directo de Adam con garantías teóricas más sólidas y un rendimiento empírico comparable o superior en los dominios de visión, PLN, RL y modelado generativo.

Algorithm 8 ADOPT (Adam con Ajuste Óptimo Podado)

Require: Parámetros iniciales θ_0 , tasa de aprendizaje η , β_1, β_2 , tolerancia ϵ

Ensure: Parámetros actualizados θ_T

```
1: Inicializar:  $m \leftarrow 0$ ,  $v \leftarrow 0$ 
2: for  $t = 1, 2, \dots, T$  do
3:    $g_t \leftarrow \nabla f(\theta_{t-1})$ 
4:   if  $\text{degradencia\_de\_peso} > 0$  then
5:      $\theta_{t-1} \leftarrow \theta_{t-1}(1 - \eta\lambda)$ 
6:   end if
7:   Usar segundo momento anterior:  $\text{denom} \leftarrow \sqrt{\max(v, \epsilon)} + \epsilon$ 
8:   Actualizar primer momento usando varianza anterior:  $m \leftarrow \beta_1 m + (1 - \beta_1)(g_t/\text{denom})$ 
9:   Actualizar segundo momento con gradiente actual:  $v \leftarrow \beta_2 v + (1 - \beta_2)g_t^2$ 
10:   $\theta_t \leftarrow \theta_{t-1} - \eta m$ 
11: end for return  $\theta_T$ 
```

AdEMAMix (Mezcla de Promedios Móviles Exponenciales). AdEMAMix reemplaza el EMA único de gradientes de Adam por una **mezcla de dos EMA**: uno de decadencia rápida y otro de decadencia lenta. Esto aborda la observación de que los gradientes siguen siendo

informativos durante decenas de miles de pasos, no solo cientos:

$$m_{1,t} = \beta_1 m_{1,t-1} + (1 - \beta_1) g_t \quad (\text{EMA rápido, } \beta_1 \approx 0,9) \quad (13)$$

$$m_{2,t} = \beta_3 m_{2,t-1} + (1 - \beta_3) g_t \quad (\text{EMA lento, } \beta_3 \approx 0,9999) \quad (14)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \quad (15)$$

$$m_t = m_{1,t} + \alpha_t m_{2,t} \quad (\text{mezcla con peso programado } \alpha_t) \quad (16)$$

$$\theta_{t+1} = \theta_t - \eta \frac{m_t}{\sqrt{v_t} + \epsilon} \quad (17)$$

El peso de la mezcla α_t típicamente aumenta durante el entrenamiento, permitiendo que el EMA rápido proporcione adaptación inmediata mientras que el EMA lento acumula correlaciones de gradiente a largo plazo. Empíricamente, un modelo de 1.3B parámetros en 101B tokens alcanza una pérdida similar a AdamW en 197B tokens (ganancia del 95 % en eficiencia de datos), lo que sugiere que el EMA lento reduce significativamente el olvido del modelo.

Algorithm 9 AdEMAMix (Mezcla de Promedios Móviles Exponenciales)

Require: Parámetros iniciales θ_0 , tasa de aprendizaje η , β_1 (rápido), β_3 (lento), β_2 (segundo momento)

Require: Peso de mezcla α_t (típicamente creciente), tolerancia ϵ

Ensure: Parámetros actualizados θ_T

```

1: Inicializar:  $m_1 \leftarrow 0$  (EMA rápido),  $m_2 \leftarrow 0$  (EMA lento),  $v \leftarrow 0$ 
2: for  $t = 1, 2, \dots, T$  do
3:    $g_t \leftarrow \nabla f(\theta_{t-1})$ 
4:   if  $\text{decadencia\_de\_peso} > 0$  then
5:      $\theta_{t-1} \leftarrow \theta_{t-1}(1 - \eta\lambda)$ 
6:   end if
7:    $m_1 \leftarrow \beta_1 m_1 + (1 - \beta_1) g_t$  ▷ EMA rápido
8:    $m_2 \leftarrow \beta_3 m_2 + (1 - \beta_3) g_t$  ▷ EMA lento
9:    $v \leftarrow \beta_2 v + (1 - \beta_2) g_t^2$ 
10:   $m_t \leftarrow m_1 + \alpha_t m_2$  ▷ Mezcla
11:   $\theta_t \leftarrow \theta_{t-1} - \eta(m_t / (\sqrt{v} + \epsilon))$ 
12: end for return  $\theta_T$ 

```

MARS (Haciendo Brillar las Tasas de Aprendizaje Adaptativas). MARS combina métodos de gradiente preconditionado (ej., AdamW) con **reducción de varianza** mediante momentum recursivo estocástico escalado. La innovación central es una estimación del gradiente con varianza reducida:

$$c_t = g_t + \gamma(c_{t-1} - g_{t-1}) \quad (\text{estimación recursiva tipo SVRG}) \quad (18)$$

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) c_t \quad (19)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) c_t^2 \quad (20)$$

$$\theta_{t+1} = \theta_t - \eta \frac{m_t}{\sqrt{v_t} + \epsilon} \quad (21)$$

donde $\gamma \in [0, 0.1]$ controla cuánta información de gradiente histórico se retiene. El gradiente con varianza reducida c_t actúa como un filtro de ruido implícito, amplificando las direcciones de señal consistentes y amortiguando el ruido contradictorio. MARS-AdamW supera consistentemente a AdamW por márgenes significativos en el preentrenamiento de GPT-2, lo que sugiere que la reducción de varianza mejora sustancialmente la convergencia en el entrenamiento estocástico por mini-lotes.

Algorithm 10 MARS (Haciendo Brillar las Tasas de Aprendizaje Adaptativas)

Require: Parámetros iniciales θ_0 , tasa de aprendizaje η , β_1, β_2 , decadencia de peso λ

Require: Coeficiente de reducción de varianza $\gamma \in [0, 0.1, 0, 1]$

Ensure: Parámetros actualizados θ_T

```
1: Inicializar:  $m \leftarrow 0$ ,  $v \leftarrow 0$ ,  $c \leftarrow 0$  (gradiente con varianza reducida),  $g_{-1} \leftarrow 0$ 
2: for  $t = 1, 2, \dots, T$  do
3:    $g_t \leftarrow \nabla f(\theta_{t-1})$ 
4:   Reducción de varianza:  $c \leftarrow g_t + \gamma(c - g_{t-1})$ 
5:    $m \leftarrow \beta_1 m + (1 - \beta_1)c$ 
6:    $v \leftarrow \beta_2 v + (1 - \beta_2)c^2$ 
7:   if decadencia_de_peso  $> 0$  then
8:      $\theta_{t-1} \leftarrow \theta_{t-1}(1 - \eta\lambda)$ 
9:   end if
10:   $\theta_t \leftarrow \theta_{t-1} - \eta(m/(\sqrt{v} + \epsilon))$ 
11:   $g_{t-1} \leftarrow g_t$ 
12: end for return  $\theta_T$ 
```

Optimizadores Cautelosos (Cautious AdamW, Cautious Lion). El marco Cauteloso aplica una **modificación de una línea** a cualquier optimizador basado en momentum: enmascarar la actualización de modo que solo se apliquen las dimensiones donde el momentum y la dirección del gradiente coinciden:

$$\text{mask}_i = \begin{cases} 1 & \text{si } m_t[i] \cdot g_t[i] > 0 \quad (\text{acuerdo}) \\ 0 & \text{en caso contrario} \end{cases} \quad (22)$$

$$u_t = \left(\frac{m_t}{\sqrt{v_t} + \epsilon} \right) \odot \text{mask} \quad (\text{enmascaramiento elemento a elemento}) \quad (23)$$

$$\theta_{t+1} = \theta_t - \eta u_t \quad (24)$$

La intuición: el momentum m_t estima la dirección del gradiente a partir del historial; el gradiente actual g_t es la señal instantánea. Cuando ambos coinciden, el optimizador está seguro y debe actualizar agresivamente. Cuando discrepan, el optimizador está en conflicto—la tendencia histórica apunta hacia una región que pudo haber sido buena antes, pero la evidencia actual la contradice. Al enmascarar las dimensiones en conflicto, Cauteloso se vuelve más conservador y evita pasos corruptos. Empíricamente, este simple enmascaramiento logra hasta **1.47× de aceleración** en el preentrenamiento de Llama y MAE mientras preserva las garantías de convergencia.

Algorithm 11 Cautious AdamW (Enmascaramiento de Actualización por Consenso)

Require: Parámetros iniciales θ_0 , tasa de aprendizaje η , β_1, β_2 , decadencia de peso λ

Ensure: Parámetros actualizados θ_T

```
1: Inicializar:  $m \leftarrow 0, v \leftarrow 0$ 
2: for  $t = 1, 2, \dots, T$  do
3:    $g_t \leftarrow \nabla f(\theta_{t-1})$ 
4:    $m \leftarrow \beta_1 m + (1 - \beta_1) g_t$ 
5:    $v \leftarrow \beta_2 v + (1 - \beta_2) g_t^2$ 
6:   Máscara de consenso:  $\text{mask}_i \leftarrow 1$  si  $m[i] \cdot g_t[i] > 0$ , si no 0 ▷ Acuerdo
7:   Actualización base Adam:  $u \leftarrow m / (\sqrt{v} + \epsilon)$ 
8:   Aplicar máscara:  $u_{\text{masked}} \leftarrow u \odot \text{mask}$  ▷ Elemento a elemento
9:   if  $\text{decadencia\_de\_peso} > 0$  then
10:     $\theta_{t-1} \leftarrow \theta_{t-1} (1 - \eta \lambda)$ 
11:   end if
12:    $\theta_t \leftarrow \theta_{t-1} - \eta u_{\text{masked}}$ 
13: end for return  $\theta_T$ 
```

A.5. Optimizadores de Lote Grande, Eficientes en Memoria y Sin Tasa de Aprendizaje

LAMB (Layer-wise Adaptive Moments optimizer for Batch training). LAMB extiende Adam con **escalado de tasa adaptativo por capas** (inspirado en LARS), permitiendo entrenamiento estable con tamaños de lote extremos (e.g., 64K en BERT):

$$m_t^L = \beta_1 m_{t-1}^L + (1 - \beta_1) g_t^L \quad (\text{momento de primer orden por capas}) \quad (25)$$

$$v_t^L = \beta_2 v_{t-1}^L + (1 - \beta_2) (g_t^L)^2 \quad (26)$$

$$u_{\text{adam}}^L = \frac{m_t^L}{\sqrt{v_t^L} + \epsilon} + \lambda \theta_{t-1}^L \quad (\text{paso adaptativo base con decaimiento}) \quad (27)$$

$$\phi^L = \frac{\|\theta_{t-1}^L\|_2}{\|u_{\text{adam}}^L\|_2} \quad (\text{razón de confianza: normalización por capas}) \quad (28)$$

$$\theta_t^L = \theta_{t-1}^L - \eta \cdot \phi^L \cdot u_{\text{adam}}^L \quad (29)$$

La **razón de confianza** $\phi^L = \|\theta_{t-1}^L\|_2 / \|u_{\text{adam}}^L\|_2$ normaliza el paso de actualización efectivo en relación con la magnitud de los pesos. En lotes muy grandes, el ruido estocástico del gradiente puede superar a la señal, desestabilizando las tasas de aprendizaje por capas. Al escalar las actualizaciones proporcionalmente a las normas de los pesos, LAMB asegura cambios relativos en lugar de absolutos, evitando que actualizaciones pequeñas y seguras en vectores grandes dominen. LAMB preserva los beneficios de generalización de lotes pequeños mientras permite entrenamiento eficiente con lotes grandes.

Algorithm 12 LAMB (Layer-wise Adaptive Moments optimizer for Batch training)

Require: Parámetros iniciales θ_0 , tasa de aprendizaje η , β_1, β_2 , decaimiento de pesos λ

Ensure: Parámetros actualizados θ_T

```
1: Inicializar: Para cada capa  $L$ :  $m^L \leftarrow 0, v^L \leftarrow 0$ 
2: for  $t = 1, 2, \dots, T$  do
3:   for cada capa  $L$  do
4:      $g_t^L \leftarrow \nabla f(\theta_t^L)$ 
5:      $m^L \leftarrow \beta_1 m^L + (1 - \beta_1) g_t^L$ 
6:      $v^L \leftarrow \beta_2 v^L + (1 - \beta_2) (g_t^L)^2$ 
7:     Adam base:  $u_{\text{adam}}^L \leftarrow m^L / (\sqrt{v^L} + \epsilon)$ 
8:     if weight_decay > 0 then
9:        $u_{\text{adam}}^L \leftarrow u_{\text{adam}}^L + \lambda \theta_{t-1}^L$ 
10:    end if
11:    Razón de confianza:  $\phi^L \leftarrow \|\theta_{t-1}^L\|_2 / \|u_{\text{adam}}^L\|_2$  si ambos son no nulos, sino 1
12:     $\theta_t^L \leftarrow \theta_{t-1}^L - \eta \phi^L u_{\text{adam}}^L$ 
13:  end for
14: end for return  $\theta_T$ 
```

Schedule-Free AdamW. Los métodos Schedule-Free eliminan el diseño explícito de programación de la receta de optimización. El algoritmo mantiene dos flujos de parámetros: z_t (exploración) y x_t (promedio suavizado):

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \quad (\text{varianza}) \quad (30)$$

$$z_t = z_{t-1} (1 - \eta \lambda) - \eta \frac{g_t}{\sqrt{v_t} + \epsilon} \quad (\text{paso adaptativo con decaimiento}) \quad (31)$$

$$c_t = \frac{1}{t + 1} \quad (\text{coeficiente de promediado, decrece con el tiempo}) \quad (32)$$

$$x_t = (1 - c_t) x_{t-1} + c_t z_t \quad (\text{promediado de iteraciones}) \quad (33)$$

$$y_t = (1 - \beta) z_t + \beta x_t \quad (\text{interpolación para el punto de evaluación}) \quad (34)$$

El coeficiente de promediado $c_t = 1/(t + 1)$ implementa una programación de tasa de aprendizaje implícita sin especificar explícitamente el número total de pasos T . Este marco unifica la programación y el promediado de iteraciones: el algoritmo explora mediante z_t mientras acumula dirección estable mediante x_t . El punto de evaluación y_t (usado para el cómputo del gradiente) interpola entre exploración y estabilidad. Schedule-Free logra convergencia de última generación en optimización convexa, aprendizaje profundo a gran escala y aprendizaje por refuerzo, eliminando un hiperparámetro importante.

Algorithm 13 Schedule-Free AdamW

Require: Parámetros iniciales θ_0 , tasa de aprendizaje η (fija), β_1, β_2 , decaimiento de pesos λ

Ensure: Parámetros actualizados θ_T o promediado x_T

```
1: Inicializar:  $z \leftarrow \theta_0$  (exploración),  $x \leftarrow \theta_0$  (promedio),  $v \leftarrow 0, t \leftarrow 0$ 
2: for  $t = 1, 2, \dots, T$  do
3:    $g_t \leftarrow \nabla f(y_{t-1})$  (gradiente en el punto de interpolación)
4:    $v \leftarrow \beta_2 v + (1 - \beta_2) g_t^2$  ▷ Varianza
5:   Decaimiento de pesos sobre  $z$ :  $z \leftarrow z(1 - \eta \lambda)$ 
6:    $z \leftarrow z - \eta g_t / (\sqrt{v} + \epsilon)$  ▷ Paso adaptativo sobre el gradiente crudo
7:    $c_t \leftarrow 1/(t + 1)$  ▷ Coeficiente de promediado
8:    $x \leftarrow (1 - c_t) x + c_t z$  ▷ Promediado de iteraciones
9:    $y_t \leftarrow (1 - \beta_1) z + \beta_1 x$  ▷ Interpolación para la siguiente evaluación
10: end for return  $x_T$  o  $y_T$ 
```

Adafactor. Adafactor reduce la memoria del optimizador **factorizando** estadísticas de segundo momento para parámetros con forma de matriz, almacenando solo acumuladores de filas y columnas en lugar de estados de varianza densos. Para una matriz de gradiente $G_t \in \mathbb{R}^{m \times n}$:

$$R_t = \beta_2 R_{t-1} + (1 - \beta_2)(G_t^2) \mathbf{1}_n^T \quad (\text{varianza por filas}) \quad (35)$$

$$C_t = \beta_2 C_{t-1} + (1 - \beta_2) \mathbf{1}_m^T (G_t^2) \quad (\text{varianza por columnas}) \quad (36)$$

$$\hat{V}_t = \frac{R_t C_t}{\mathbf{1}_n^T R_t} \quad (\text{varianza reconstruida mediante producto exterior}) \quad (37)$$

$$U_t = \frac{G_t}{\sqrt{\hat{V}_t} + \epsilon} \quad (\text{paso adaptativo normalizado}) \quad (38)$$

$$\hat{U}_t = \frac{U_t}{\max(1, \text{RMS}(U_t))} \quad (\text{recorte de estabilidad}) \quad (39)$$

En lugar de almacenar $m \times n$ valores de segundo momento, Adafactor almacena solo $m + n$ acumuladores, reduciendo la memoria de $O(n_{\text{params}})$ a $O(\sqrt{n_{\text{params}}})$. Esto es más atractivo cuando la VRAM está dominada por el estado del optimizador en lugar de las activaciones. La contrapartida es una expresividad de optimización reducida y posible inestabilidad en algunas tareas.

Algorithm 14 Adafactor (Estadísticas de Segundo Momento Factorizadas)

Require: Matriz de gradiente $G_t \in \mathbb{R}^{m \times n}$, tasa de aprendizaje η , $\beta_2 \approx 0,999$

Ensure: Parámetros actualizados

- 1: Inicializar: Acumuladores de fila $R \leftarrow \epsilon \mathbf{1}_m^T$, Columna $C \leftarrow \epsilon \mathbf{1}_n$
 - 2: **for** $t = 1, 2, \dots, T$ **do**
 - 3: $G_t \leftarrow \nabla f(\theta_t)$ (gradiente matricial)
 - 4: $R \leftarrow \beta_2 R + (1 - \beta_2)(G_t^2) \mathbf{1}_n^T$ ▷ Productos internos por filas
 - 5: $C \leftarrow \beta_2 C + (1 - \beta_2) \mathbf{1}_m^T (G_t^2)$ ▷ Productos internos por columnas
 - 6: Varianza reconstruida: $\hat{V} \leftarrow (R \cdot C) / (\mathbf{1}_n^T R)$ ▷ Mediante producto exterior
 - 7: Paso adaptativo normalizado: $U_t \leftarrow G_t / (\sqrt{\hat{V}} + \epsilon)$
 - 8: Recorte RMS por fila: $\hat{U}_t \leftarrow U_t / \max(1, \text{RMS}(U_t))$
 - 9: $\theta_{t+1} \leftarrow \theta_t - \eta \hat{U}_t$
 - 10: **end for**
-

GaLore (Gradient Low-Rank Projection). GaLore proyecta gradientes 2D en un subespacio de bajo rango antes de la optimización, reduciendo la memoria del estado del optimizador mientras mantiene el escalado adaptativo del paso. Para una matriz de parámetros 2D $W \in \mathbb{R}^{m \times n}$:

$$U, S, V = \text{SVD}(G_t) \quad (\text{calcular descomposición singular}) \quad (40)$$

$$P \in \mathbb{R}^{n \times r} \quad \text{o} \quad P^T \in \mathbb{R}^{r \times m} \quad (\text{seleccionar los } r \text{ vectores singulares superiores}) \quad (41)$$

$$G_{\text{low}} = P^T G_t \quad (\text{proyectar al espacio de bajo rango}) \quad (42)$$

$$\Delta_{\text{low}} = \text{Adam}(G_{\text{low}}) \quad (\text{optimizar en el espacio comprimido}) \quad (43)$$

$$\Delta = P \Delta_{\text{low}} \quad (\text{reconstruir en el espacio original}) \quad (44)$$

Al proyectar en un subespacio de rango r (típicamente $r \ll \min(m, n)$), el estado del optimizador se reduce de $O(mn)$ a $O(r(m+n))$ para parámetros 2D. Este enfoque complementario de ahorro de memoria es especialmente relevante cuando el modelo es demasiado grande para el estado completo del optimizador. GaLore funciona sinérgicamente con otras técnicas y ha mostrado resultados empíricos sólidos en modelos de miles de millones de parámetros.

Algorithm 15 GaLore (Gradient Low-Rank Projection)

Require: Matriz de parámetros 2D $W \in \mathbb{R}^{m \times n}$, tasa de aprendizaje η , rango $r \ll \min(m, n)$

Ensure: Parámetros actualizados

```
1: Inicializar: Estado de Adam en el espacio de bajo rango (si es variante de rango 2)
2: for  $t = 1, 2, \dots, T$  do
3:    $G_t \leftarrow \nabla f(W_t)$ 
4:   if  $t \bmod K = 1$  then ▷ SVD periódico
5:      $U_t, S_t, V_t^\top \leftarrow \text{SVD}(G_t)$  (completa o delgada)
6:     Seleccionar proyección:  $P \leftarrow U_t[:, :r]$  o  $P \leftarrow V_t[:, :r]$ 
7:   end if
8:   Proyectar gradiente:  $G_{\text{low}} \leftarrow P^\top G_t$  (o  $G_t P^\top$  para proyección derecha)
9:   Ejecutar Adam en bajo rango:  $\Delta_{\text{low}} \leftarrow \text{Adam}(G_{\text{low}})$ 
10:  Reconstruir en espacio original:  $\Delta \leftarrow P \Delta_{\text{low}}$  (o  $\Delta_{\text{low}} P^\top$ )
11:   $W_t \leftarrow W_t - \eta \Delta$ 
12: end for
```

APOLLO, APOLLO-Mini, y Q-APOLLO. La familia APOLLO [56] parte de la observación de que el denominador elemento a elemento de AdamW puede **transformarse en una actualización estructurada de la tasa de aprendizaje**. En lugar de almacenar momentos densos para cada entrada de parámetro, APOLLO proyecta un gradiente matricial $G_t \in \mathbb{R}^{m \times n}$ en un subespacio aleatorio compacto y rastrea momentos estilo Adam allí:

$$R_t = P_t G_t \quad \text{o} \quad R_t = G_t P_t^\top \quad (45)$$

$$M_t^R = \beta_1 M_{t-1}^R + (1 - \beta_1) R_t \quad (46)$$

$$V_t^R = \beta_2 V_{t-1}^R + (1 - \beta_2) R_t^2 \quad (47)$$

$$\tilde{R}_t = \frac{M_t^R}{\sqrt{V_t^R} + \epsilon} \quad (48)$$

El estado proyectado *no* se expande de vuelta a una actualización densa de bajo rango como en los métodos basados en SVD. En su lugar, APOLLO estima un tensor de escalado estructurado S_t en el espacio original. En la variante estándar de APOLLO, ese escalado es **por canales**: cada fila o canal recibe su propia razón de norma. En APOLLO-Mini, el escalado se reduce a un único escalar **por tensor**, correspondiente al caso extremo de rango 1 descrito en el artículo. La actualización de parámetros resultante es, por tanto, similar a Adam en adaptación pero mucho más cercana a SGD en costo de estado:

$$W_t \leftarrow (1 - \eta\lambda)W_{t-1} - \eta\alpha(G_t \odot S_t)$$

donde α es el factor de escala adicional usado para estabilizar variantes altamente comprimidas.

La contribución práctica del artículo es doble. Primero, APOLLO reemplaza el costoso SVD repetido con **proyección aleatoria gaussiana** actualizada periódicamente, de modo que la carga computacional es multiplicación de matrices ordinaria en lugar de descomposición espectral. Segundo, el optimizador es inusualmente tolerante a la compresión extrema: incluso **APOLLO-Mini**, que mantiene solo estado auxiliar de rango 1, sigue siendo competitivo o mejor que AdamW en los experimentos de preentrenamiento reportados, mientras se acerca al costo de memoria de SGD.

Algorithm 16 APOLLO / APOLLO-Mini Escalado de Gradiente Estructurado

Require: Matriz de pesos $W \in \mathbb{R}^{m \times n}$ con $m \leq n$, tasa de aprendizaje η , factor de escala α , tasas de decaimiento (β_1, β_2) , decaimiento de pesos λ , rango r , intervalo de actualización de proyección T

Ensure: Parámetros actualizados W_T

```
1: Inicializar momentos proyectados:  $M^R \leftarrow 0, V^R \leftarrow 0$ , paso  $t \leftarrow 0$ 
2: repeat
3:   Calcular gradiente:  $G_t \leftarrow \nabla_W \phi(W_t)$ 
4:   if  $t \bmod T = 0$  then
5:     Muestrear proyector gaussiano  $P_t \sim \mathcal{N}(0, 1/r)$  con una semilla nueva
6:   end if
7:   Proyectar gradiente:  $R_t \leftarrow P_t G_t$   $\triangleright$  o  $G_t P_t^\top$  según la disposición
8:   Actualizar momentos proyectados AdamW:  $M_t^R, V_t^R \leftarrow \text{AdamWState}(R_t; \beta_1, \beta_2)$ 
9:   Normalizar estado proyectado:  $\tilde{R}_t \leftarrow M_t^R / (\sqrt{V_t^R} + \epsilon)$ 
10:  if APOLLO then
11:     $S_t \leftarrow \text{diag}(s_1^R, \dots, s_m^R)$ , donde  $s_i^R = \|\tilde{R}_t[i, :]\|_2 / \|R_t[i, :]\|_2$ 
12:  else
13:     $S_t \leftarrow s_t^R$ , donde  $s_t^R = \|\tilde{R}_t\|_2 / \|R_t\|_2$ 
14:  end if
15:  Actualizar pesos:  $W_t \leftarrow (1 - \eta\lambda)W_{t-1} - \eta\alpha(G_t \odot S_t)$ 
16:   $t \leftarrow t + 1$ 
17: until convergencia
```

APOLLO-Mini. APOLLO-Mini es el miembro de la familia optimizado para **eficiencia de memoria extrema**. El artículo lo motiva argumentando que, en un espacio compacto de rango 1, el escalado por canales se vuelve demasiado ruidoso, por lo que la actualización se simplifica aún más a un único escalar por tensor. La pérdida de granularidad se compensa parcialmente con el factor de escala explícito α (el artículo discute valores como 128 en este régimen), proporcionando un punto útil en la frontera de Pareto de optimización: estado muy pequeño, sin sobrecarga de SVD, y aún un comportamiento sólido en preentrenamiento.

Q-APOLLO. El artículo de APOLLO también enfatiza que la familia se combina naturalmente con **cuantización** para entrenamiento con memoria ultra-baja. Este repositorio convierte esa observación de sistemas en una variante concreta del optimizador, **q_apollo**. En la implementación, los momentos de primer y segundo orden de bajo rango de APOLLO se almacenan en forma cuantizada junto con escalas y desplazamientos por tensor, y se descuantizan solo cuando es necesario para el siguiente paso. Q-APOLLO preserva, por tanto, la lógica de gradiente proyectado de APOLLO mientras reduce la precisión del estado restante del optimizador, convirtiéndolo en el miembro más agresivo en ahorro de memoria de la pila local de optimizadores.

Anon (Adaptivity Non-restricted Optimizer with Novel convergence technique). Anon [1] introduce adaptividad ajustable continuamente mediante un único parámetro $\gamma \in \mathbb{R}$, cerrando la brecha entre comportamientos tipo SGD ($\gamma = 0$) y tipo Adam ($\gamma = 1$), y extrapolando más allá de ambos. La innovación central es el mecanismo Incremental Delay Update (IDU), que permite convergencia estable a través de todo el espectro de adaptividad de valores reales:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \quad (\text{momento de primer orden}) \quad (49)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \quad (\text{momento de segundo orden}) \quad (50)$$

$$\bar{v}_t = \frac{v_t}{1 - \beta_2^{\max(t,1)}} + \epsilon \quad (\text{momento de segundo orden con corrección de sesgo}) \quad (51)$$

$$\text{fixed_lr}_k = \begin{cases} \bar{v}_k^{-\gamma/2} & \text{si } k = 0 \\ \sqrt{2 / \left(\frac{1}{\text{fixed_lr}_{k-1}^2} + \bar{v}_k^\gamma \right)} & \text{si } k > 0 \end{cases} \quad (52)$$

$$\theta_{t+1} = \theta_t - \frac{\eta}{1 - \beta_1^t} \cdot m_t \odot \text{fixed_lr}_{\lfloor \log_2 t \rfloor} \quad (53)$$

A diferencia de los optimizadores adaptativos estándar, IDU actualiza el preconditionador solo en potencias de dos ($t = 2^k$), acumulando estadísticas de gradiente entre actualizaciones y agregándolas recursivamente. Este acumulador suave y multiescala evita el seguimiento estricto del máximo de AMSGrad mientras estabiliza demostrablemente la convergencia para cualquier $\gamma \in \mathbb{R}$. El parámetro γ controla directamente la adaptividad: $\gamma > 1$ acelera la salida de puntos de silla (favorable para arquitecturas complejas), $\gamma = 1$ recupera el comportamiento tipo Adam, $\gamma = 0$ produce escalado tipo SGD, y $\gamma < 0$ se sesga hacia mínimos más planos (beneficioso para arquitecturas clásicas como CNNs).

Anon mantiene tres buffers de estado $(m, v, \text{fixed_lr})$ con costo $O(n)$ por paso, situando en la misma clase de complejidad para problemas convexos y $O(\ln T / \sqrt{T})$ para entornos no convexos, igualando las garantías de los optimizadores adaptativos convencionales mientras soporta todo el rango de adaptividad de valores reales.

Algorithm 17 Anon (Adaptivity Non-restricted Optimizer con IDU)

Require: Parámetros iniciales θ_0 , tasa de aprendizaje η , β_1, β_2 , tolerancia ϵ , adaptividad $\gamma \in \mathbb{R}$

Ensure: Parámetros actualizados θ_T

```

1: Inicializar:  $m \leftarrow 0$ ,  $v \leftarrow 0$ ,  $\text{fixed\_lr} \leftarrow 0$ , paso  $t \leftarrow 0$ , contador IDU  $k \leftarrow -1$ 
2: for  $t = 1, 2, \dots, T$  do
3:    $g_t \leftarrow \nabla f(\theta_{t-1})$ 
4:    $m \leftarrow \beta_1 m + (1 - \beta_1) g_t$ 
5:    $v \leftarrow \beta_2 v + (1 - \beta_2) g_t^2$ 
6:   if  $t = 2^k$  then
7:      $k \leftarrow k + 1$ 
8:      $\bar{v} \leftarrow v / (1 - \beta_2^{\max(t/2, 1)}) + \epsilon$ 
9:     if  $k = 0$  then
10:       $\text{fixed\_lr} \leftarrow \bar{v}^{-\gamma/2}$ 
11:     else
12:       $\text{fixed\_lr} \leftarrow \sqrt{2 / (1 / \text{fixed\_lr}^2 + \bar{v}^\gamma)}$ 
13:     end if
14:      $v \leftarrow 0$ 
15:   end if
16:    $\theta_t \leftarrow \theta_{t-1} - \frac{\eta}{1 - \beta_1^t} \cdot m \odot \text{fixed\_lr}$ 
17: end for return  $\theta_T$ 

```

Prodigy (Approximating the Distance Estimate). Prodigy adapta la escala efectiva del paso mediante una estadística tipo distancia acumulativa, eliminando la necesidad de ajuste explícito de la tasa de aprendizaje. El algoritmo mantiene una estimación de distancia acumu-

lativa:

$$u_t = \frac{g_t}{\sqrt{v_t} + \epsilon} \quad (\text{paso adaptativo no normalizado}) \quad (54)$$

$$s_t = s_{t-1} + \langle u_t, \theta_t - \theta_0 \rangle \quad (\text{distancia signada acumulativa}) \quad (55)$$

$$d_t = \max(d_{t-1}, d_0 + d_{\text{coef}} \cdot s_t) \quad (\text{estimación de distancia con cota inferior}) \quad (56)$$

$$\theta_{t+1} = \theta_t - \eta \cdot d_t \cdot u_t \quad (57)$$

donde d_0 es una cota inicial y d_{coef} controla qué tan agresivamente se adapta la estimación. La estimación de distancia d_t captura la magnitud combinada de gradientes pasados ponderada por el desplazamiento real de los parámetros, implementando una tasa de aprendizaje efectiva implícita. Este escalado consciente de la distancia logra convergencia robusta en distintas escalas de problemas y tamaños de lote sin requerir una programación de la tasa de aprendizaje. Prodigy reduce la sensibilidad a hiperparámetros estimando los tamaños de paso a partir de la geometría de optimización en lugar del conocimiento previo específico del problema.

Algorithm 18 Prodigy (Approximating the Distance Estimate)

Require: Parámetros iniciales θ_0 , tasa de aprendizaje η , coeficiente d_{coef} , distancia inicial $d_0 > 0$

Ensure: Parámetros actualizados θ_T

- 1: Inicializar: $s \leftarrow 0$ (distancia signada acumulativa), $d \leftarrow d_0$
 - 2: **for** $t = 1, 2, \dots, T$ **do**
 - 3: $g_t \leftarrow \nabla f(\theta_{t-1})$
 - 4: $v_t \leftarrow \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$ ▷ Segundo momento
 - 5: $u_t \leftarrow g_t / (\sqrt{v_t} + \epsilon)$ ▷ Adaptativo no normalizado
 - 6: Actualizar distancia: $s \leftarrow s + \langle u_t, \theta_t - \theta_0 \rangle$ ▷ Distancia signada
 - 7: $d \leftarrow \max(d, d_0 + d_{\text{coef}} \cdot s)$ ▷ Distancia con cota inferior
 - 8: $\theta_t \leftarrow \theta_{t-1} - \eta \cdot d \cdot u_t$ ▷ Actualización escalada por distancia
 - 9: **end for** **return** θ_T
-

A.6. Optimizadores de Segundo Orden, Geométricos y de Ortogonalidad

Algorithm 19 Shampoo (Precondicionamiento Matricial mediante Productos de Kronecker)

Require: Parámetros iniciales θ_0 , tasa de aprendizaje η , intervalo de eigendescomposición K

Ensure: Parámetros actualizados θ_T

- 1: Inicializar: $L \leftarrow \epsilon I_m$, $R \leftarrow \epsilon I_n$ (matrices de Gram izquierda y derecha)
 - 2: **for** $t = 1, 2, \dots, T$ **do**
 - 3: $G \leftarrow \nabla f(\theta_{t-1})$ ▷ Gradiente
 - 4: Actualizar matrices de Gram: $L \leftarrow L + GG^\top$, $R \leftarrow R + G^\top G$
 - 5: **if** $t \bmod K == 0$ **then**
 - 6: $Q_L, \Lambda_L \leftarrow \text{eigh}(L)$ ▷ Eigendescomposición de L
 - 7: $Q_R, \Lambda_R \leftarrow \text{eigh}(R)$ ▷ Eigendescomposición de R
 - 8: $L^{-1/4} \leftarrow Q_L (\Lambda_L + \epsilon)^{-1/4} Q_L^\top$
 - 9: $R^{-1/4} \leftarrow Q_R (\Lambda_R + \epsilon)^{-1/4} Q_R^\top$
 - 10: **end if**
 - 11: $\Delta\theta \leftarrow L^{-1/4} G R^{-1/4}$ ▷ Gradiente precondicionado
 - 12: $\theta_t \leftarrow \theta_{t-1} - \eta \cdot \Delta\theta$
 - 13: **end for** **return** θ_T
-

Shampoo (Precondicionamiento Matricial mediante Productos de Kronecker). Shampoo es un **método estructurado de segundo orden** que calcula preconditionadores matriciales a partir de estadísticas de producto exterior con estructura de Kronecker. Para una matriz

de parámetros 2D $W \in \mathbb{R}^{m \times n}$:

$$L_t = L_{t-1} + G_t G_t^\top \quad (\text{matriz de Gram izquierda/filas, } m \times m) \quad (58)$$

$$R_t = R_{t-1} + G_t^\top G_t \quad (\text{matriz de Gram derecha/columnas, } n \times n) \quad (59)$$

$$L_t^{-1/4} = Q_L(\Lambda_L)^{-1/4} Q_L^\top \quad (\text{mediante eigendescomposición}) \quad (60)$$

$$R_t^{-1/4} = Q_R(\Lambda_R)^{-1/4} Q_R^\top \quad (61)$$

$$\Delta W_t = L_t^{-1/4} G_t R_t^{-1/4} \quad (\text{gradiente preconditionado}) \quad (62)$$

Shampoo aproxima el preconditionador de matriz completa de Adagrad $H^{-1/2}$ (donde H es el Hessiano) utilizando estructura factorizada de Kronecker. En lugar de almacenar un preconditionador de $(mn) \times (mn)$, Shampoo mantiene dos matrices más pequeñas ($m \times m$ y $n \times n$), capturando correlaciones entre parámetros dentro y entre grupos. El método ha demostrado ser efectivo a escala (sistemas de producción de Google) y es adecuado para arquitecturas transformer con estructura de alto rango. Las eigendescomposiciones se realizan periódicamente (e.g., cada $K = 10$ pasos) para amortizar el costo. Contrapartida: mayor cómputo por paso y eigendescomposición $O(m^3 + n^3)$ periódica frente a mejor condicionamiento y convergencia acelerada.

SOAP (Shampoo with Adam in eigenbasis). SOAP es una variante simplificada de Shampoo que desacopla el preconditionamiento del seguimiento de momento. En lugar de álgebra matricial compleja sobre gradientes preconditionados, SOAP ejecuta Adam estándar en la base de eigenvectores de los preconditionadores de Shampoo:

$$L_t = L_{t-1} + G_t G_t^\top, \quad R_t = R_{t-1} + G_t^\top G_t \quad (\text{acumulación de Gram}) \quad (63)$$

$$Q_L, \Lambda_L = \text{eigh}(L_t), \quad Q_R, \Lambda_R = \text{eigh}(R_t) \quad (\text{eigendescomposición periódica}) \quad (64)$$

$$G_t^{\text{rot}} = Q_L^\top G_t Q_R \quad (\text{rotar gradiente a la base de eigenvectores}) \quad (65)$$

$$m_t^{\text{rot}} = \beta_1 m_{t-1}^{\text{rot}} + (1 - \beta_1) G_t^{\text{rot}} \quad (66)$$

$$v_t^{\text{rot}} = \beta_2 v_{t-1}^{\text{rot}} + (1 - \beta_2) (G_t^{\text{rot}})^2 \quad (\text{Adam en el espacio rotado}) \quad (67)$$

$$U_t^{\text{rot}} = \frac{m_t^{\text{rot}}}{\sqrt{v_t^{\text{rot}} + \epsilon}}, \quad U_t = Q_L U_t^{\text{rot}} Q_R^\top \quad (\text{rotar de vuelta}) \quad (68)$$

La idea clave: todo el seguimiento de momento ocurre en la base de eigenvectores bien condicionada, simplificando la estabilidad numérica y el análisis teórico. SOAP combina los beneficios de Shampoo (estructura de curvatura explícita) y Adam (mecánica de momento probada), siendo más limpio y a menudo más estable que Shampoo completo. Las eigendescomposiciones se recalculan cada K pasos, amortizando el costo.

Algorithm 20 SOAP (Shampoo with Adam in Eigenbasis)

Require: Parámetros iniciales θ_0 , tasa de aprendizaje η , β_1, β_2 , intervalo de eigendescomposición K

Ensure: Parámetros actualizados θ_T

```
1: Inicializar:  $L \leftarrow \epsilon I_m$ ,  $R \leftarrow \epsilon I_n$ ,  $m \leftarrow 0$ ,  $v \leftarrow 0$ 
2: for  $t = 1, 2, \dots, T$  do
3:    $G \leftarrow \nabla f(\theta_{t-1})$  ▷ Gradiente
4:   Actualizar Gram:  $L \leftarrow L + GG^\top$ ,  $R \leftarrow R + G^\top G$ 
5:   if  $t \bmod K == 0$  then
6:      $Q_L, \Lambda_L \leftarrow \text{eigh}(L)$ ,  $Q_R, \Lambda_R \leftarrow \text{eigh}(R)$ 
7:   end if
8:    $G^{\text{rot}} \leftarrow Q_L^\top G Q_R$  ▷ Rotar gradiente a la base de eigenvectores
9:    $m \leftarrow \beta_1 m + (1 - \beta_1) G^{\text{rot}}$  ▷ Media móvil exponencial (corregir sesgo externamente)
10:   $v \leftarrow \beta_2 v + (1 - \beta_2) (G^{\text{rot}})^2$  ▷ Segundo momento (corregir sesgo externamente)
11:   $u \leftarrow m / (\sqrt{v} + \epsilon)$  ▷ Paso adaptativo en la base de eigenvectores
12:   $\Delta\theta \leftarrow Q_L u Q_R^\top$  ▷ Rotar de vuelta al espacio de parámetros
13:   $\theta_t \leftarrow \theta_{t-1} - \eta \cdot \Delta\theta$ 
14: end for return  $\theta_T$ 
```

Lion (EvoLved Sign Momentum). Lion logra un **sobrecosto de memoria mínimo** al usar actualizaciones basadas en signo en lugar de escalado adaptativo completo:

$$c_t = \beta_1 m_t + (1 - \beta_1) g_t \quad (\text{entrada de momento}) \quad (69)$$

$$\theta_{t+1} = \theta_t - \eta (\text{sign}(c_t) + \lambda \theta_t) \quad (\text{actualización con operador signo}) \quad (70)$$

$$m_{t+1} = \beta_2 m_t + (1 - \beta_2) g_t \quad (\text{acumulación de momento}) \quad (71)$$

Todas las operaciones de escalado elemento a elemento se reemplazan con `sign()`, que produce $\{-1, 0, +1\}$. Esto reduce drásticamente la memoria en comparación con métodos tipo Adam: Lion almacena solo el buffer de momento, sin varianza de segundo momento. La contrapartida: las actualizaciones basadas en signo sacrifican la adaptación de tasa de aprendizaje elemento a elemento que hace efectivo a Adam. Lion se posiciona no como un optimizador universalmente superior sino como una alternativa especializada de baja memoria y alto rendimiento para escenarios donde la memoria de activaciones domina y se puede sacrificar algo de sofisticación del optimizador. Funciona bien en regímenes de lotes grandes y cuando el rendimiento del hardware es la restricción principal.

Algorithm 21 Lion (EvoLved Sign Momentum)

Require: Parámetros iniciales θ_0 , tasa de aprendizaje η , β_1, β_2 , decaimiento de pesos λ

Ensure: Parámetros actualizados θ_T

```
1: Inicializar:  $m \leftarrow 0$  (buffer de momento)
2: for  $t = 1, 2, \dots, T$  do
3:    $g \leftarrow \nabla f(\theta_{t-1})$  ▷ Gradiente
4:    $c \leftarrow \beta_1 m + (1 - \beta_1) g$  ▷ Entrada de momento
5:    $\theta_t \leftarrow \theta_{t-1} - \eta (\text{sign}(c) + \lambda \theta_{t-1})$  ▷ Actualización basada en signo con decaimiento de pesos
6:    $m \leftarrow \beta_2 m + (1 - \beta_2) g$  ▷ Acumulación de momento (desacoplada)
7: end for return  $\theta_T$ 
```

Sophia (Second-Order Hessian Information with Optimized Approximation). Sophia utiliza **estimaciones diagonales del Hessiano** para escalado consciente de la curvatura sin

el costo de memoria y cómputo de los métodos densos de segundo orden.

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \quad (\text{momento de primer orden}) \quad (72)$$

$$h_t = \beta_2 h_{t-(k-1)} + (1 - \beta_2) \hat{h}_t \quad (\text{Hessiano diagonal, actualizado cada } k \text{ pasos}) \quad (73)$$

$$\text{clip}(x, C) = \min(\max(x, -C), C) \quad (\text{recorte elemento a elemento}) \quad (74)$$

$$\theta_{t+1} = \theta_t - \eta \cdot \text{clip}\left(\frac{m_t}{\max(\gamma h_t, \epsilon)}, 1\right) \quad (75)$$

donde $\hat{h}_t$ es una estimación diagonal (e.g., estimador de traza de Hutchinson) y γ es un factor de escala. El Hessiano diagonal h_t captura la curvatura local, permitiendo que el optimizador tome pasos más pequeños en direcciones pronunciadas y pasos más grandes en direcciones planas. La operación de recorte $\text{clip}(\cdot, 1)$ evita que los pasos adaptativos exploten. Sophia pertenece a la familia de métodos conscientes de la curvatura que buscan mejor condicionamiento sin el costo $O(n^2)$ o $O(n^3)$ de los métodos completos de segundo orden. Funciona particularmente bien en escenarios de ajuste fino de segunda pasada.

Algorithm 22 Sophia (Hessiano Diagonal con Actualizaciones Recortadas)

Require: Parámetros iniciales θ_0 , tasa de aprendizaje η , β_1, β_2 , frecuencia de actualización del Hessiano k , umbral de recorte C

Ensure: Parámetros actualizados θ_T

```

1: Inicializar:  $m \leftarrow 0$ ,  $h \leftarrow \epsilon$  (estimación diagonal del Hessiano)
2: for  $t = 1, 2, \dots, T$  do
3:    $g \leftarrow \nabla f(\theta_{t-1})$ 
4:    $m \leftarrow \beta_1 m + (1 - \beta_1) g$   $\triangleright$  Momento de primer orden (corregir sesgo externamente)
5:   if  $t \bmod k == 0$  then
6:      $\hat{h} \leftarrow \text{HutchinsonEstimate}(g)$   $\triangleright$  Hessiano diagonal mediante Hutchinson
7:      $h \leftarrow \beta_2 h + (1 - \beta_2) \hat{h}$   $\triangleright$  Actualizar estimación del Hessiano
8:   end if
9:    $u \leftarrow \frac{m}{\max(\gamma h, \epsilon)}$   $\triangleright$  Paso adaptativo (elemento a elemento)
10:   $u \leftarrow \text{clip}(u, C)$   $\triangleright$  Recorte elemento a elemento a  $[-C, C]$ 
11:   $\theta_t \leftarrow \theta_{t-1} - \eta \cdot u$ 
12: end for return  $\theta_T$ 

```

Muon y Turbo-Muon (Optimizadores Basados en Ortogonalidad). Muon y Turbo-Muon son **optimizadores orientados a la ortogonalidad** que reformulan la geometría de actualización utilizando iteraciones polinomiales de Newton-Schulz para ortogonalización. Para una matriz de parámetros 2D W con gradiente G_t :

$$X_0 = \frac{G_t}{\|G_t\|_F + \epsilon} \quad (\text{gradiente normalizado}) \quad (76)$$

$$A_k = X_k X_k^\top \quad (\text{Gramiana}) \quad (77)$$

$$B_k = b A_k + c A_k^2 \quad (\text{paso polinomial, coeficientes } b, c \text{ de Newton-Schulz}) \quad (78)$$

$$X_{k+1} = a X_k + B_k X_k \quad (\text{iteración } k = 0, 1, \dots, 4) \quad (79)$$

$$W_{t+1} = W_t - \eta X_K \quad (\text{actualización con dirección ortogonalizada}) \quad (80)$$

Muon realiza 5 iteraciones de Newton-Schulz para producir una dirección aproximadamente ortogonal, asegurando que las actualizaciones respeten restricciones geométricas. Turbo-Muon añade un paso de **precondicionamiento casi-ortogonal** (AOL) antes de las iteraciones, reduciendo el número de pasos de ortogonalización requeridos a 4. La justificación: las actualizaciones ortogonales preservan las normas durante el entrenamiento, evitando la deriva de norma observada en

métodos basados en momento. Estos optimizadores muestran potencial en entrenamiento a gran escala pero requieren kernels CUDA personalizados para eficiencia. Contrapartida: alto cómputo por paso (multiplicaciones de matrices e iteraciones polinomiales) frente a una geometría de actualización fundamentalmente mejor condicionada.

Algorithm 23 Muon (Ortogonalización de Newton-Schulz)

Require: Parámetros iniciales θ (matrices), tasa de aprendizaje η , iteraciones Newton-Schulz K

Ensure: Parámetros actualizados θ

```

1: for cada matriz de parámetros 2D  $W$  do
2:    $G \leftarrow \nabla f(W)$  ▷ Gradiente
3:    $X_0 \leftarrow \frac{G}{\|G\|_{F+\epsilon}}$  ▷ Normalizar gradiente por norma de Frobenius
4:   for  $k = 0, 1, \dots, K - 1$  do
5:      $A_k \leftarrow X_k X_k^\top$  ▷ Gramiana (costo:  $O(mn^2)$  o  $O(m^2n)$  para matrices altas/anchas)
6:      $B_k \leftarrow (3/2)A_k - (1/2)A_k^2$  ▷ Coeficientes de Newton-Schulz
7:      $X_{k+1} \leftarrow B_k X_k$  ▷ Paso polinomial
8:   end for
9:    $W \leftarrow W - \eta X_K$  ▷ Actualizar con dirección ortogonalizada
10: end for return  $\theta$  actualizado

```

Algorithm 24 Turbo-Muon (Ortogonalización Precondicionada con AOL)

Require: Parámetros iniciales θ (matrices), tasa de aprendizaje η , iteraciones Newton-Schulz K' (típicamente 4)

Ensure: Parámetros actualizados θ

```

1: for cada matriz de parámetros 2D  $W$  do
2:    $G \leftarrow \nabla f(W)$  ▷ Gradiente
3:    $X_0 \leftarrow \frac{G}{\|G\|_{F+\epsilon}}$  ▷ Normalizar gradiente
4:   AOL (Almost-Orthogonal preconditioner): ▷ Paso de preconditionamiento
5:    $P \leftarrow (1,5I - 0,5X_0 X_0^\top)$  ▷ Matriz de preconditionamiento
6:    $X_0 \leftarrow P X_0$  ▷ Inicio preconditionado
7:   for  $k = 0, 1, \dots, K' - 1$  do
8:      $A_k \leftarrow X_k X_k^\top$ 
9:      $B_k \leftarrow (3/2)A_k - (1/2)A_k^2$ 
10:     $X_{k+1} \leftarrow B_k X_k$  ▷ Iteraciones reducidas gracias al preconditionamiento AOL
11:   end for
12:    $W \leftarrow W - \eta X_{K'}$  ▷ Actualizar con dirección ortogonalizada preconditionada
13: end for return  $\theta$  actualizado

```

Grupo	Métodos	Objetivo Principal	Interpretación desde el Estudio
Línea base clásica	SGD, AdamW, RADam	estabilidad y referencias base	Definen el piso de comparación para las afirmaciones de optimizadores más nuevos.
Rediseño de momento	Adan, AdEMAMix, MARS, Cautious AdamW	adaptación de primer orden más rápida o segura	Mejor cuando la velocidad de convergencia o la estabilidad ante gradiente ruidoso es la preocupación principal.

Grupo	Métodos	Objetivo Principal	Interpretación desde el Estudio
Simplificación de lotes grandes y programación	LAMB, Schedule-Free AdamW	robustez operacional a escala	Reducen la fragilidad debida al crecimiento del tamaño de lote o la ingeniería de programación.
Eficiencia de memoria	Adafactor, GaLore, APOLLO, APOLLO-Mini, Q-APOLLO, Lion	reducción del estado del optimizador	Más útil cuando la VRAM está dominada por el estado del optimizador en lugar de las activaciones; los métodos de la familia APOLLO reemplazan los momentos densos de AdamW con escalado estructurado proyectado o cuantizado.
Consciente de curvatura	Shampoo, Sophia	mejor condicionamiento	Preferir cuando una geometría más rica justifica la sobrecarga de implementación y cómputo.
Orientado a geometría	Muon, Turbo-Muon	estructura de actualización ortogonalizada	Opciones especializadas para geometría matricial y modelado de representaciones.

B. Anexo B: Transformadores Densos, Recurrentes y Aumentados con Memoria

Este anexo exhaustivo sintetiza las arquitecturas modernas de transformers más allá de la línea base estándar de softmax. El campo ha evolucionado hacia seis paradigmas principales: atención densa global con variantes, compresión de estado recurrente con decaimiento, modelos de espacio de estado selectivos, integración numérica de profundidad continua, arquitecturas aumentadas con memoria y enfoques híbridos. Comprender esta taxonomía ilumina las compensaciones fundamentales entre expresividad, costo computacional, huella de memoria y simplicidad de despliegue.

B.1. Atención Densa de Referencia: Estándar y Sigmoide

B.1.1. Atención Softmax Estándar

La atención multi-cabeza estándar [39] calcula la similitud de producto punto escalado entre embeddings de tokens:

- 1: **Entrada:** Matriz de consulta $\mathbf{Q} \in \mathbb{R}^{n \times d}$, Matriz de clave $\mathbf{K} \in \mathbb{R}^{n \times d}$, Matriz de valor $\mathbf{V} \in \mathbb{R}^{n \times d}$
- 2: puntajes $\leftarrow \frac{\mathbf{QK}^\top}{\sqrt{d}} \in \mathbb{R}^{n \times n}$ ▷ calcular similitudes por pares
- 3: pesos_atencion $\leftarrow \text{softmax}(\text{puntajes}, \text{eje} = 1) \in \mathbb{R}^{n \times n}$ ▷ normalizar sobre las claves
- 4: **Salida:** $\mathbf{Y} = \text{pesos_atencion} \cdot \mathbf{V} \in \mathbb{R}^{n \times d}$ ▷ combinación ponderada de valores

Para generación autorregresiva, el almacenamiento en caché KV guarda claves y valores pasados para evitar el recálculo $\mathcal{O}(n^2)$. Sin embargo, este caché de crecimiento lineal ocupa $\sim n \cdot d_{\text{oculto}}$ bytes, lo que puede consumir cientos de gigabytes para modelos de miles de millones de parámetros. La atención estándar logra una **expresividad perfecta** dentro de la ventana de contexto—cualquier token puede atender a cualquier otro token con pesos aprendidos—pero paga el precio del cómputo denso.

- **Complejidad de entrenamiento:** $\mathcal{O}(n^2 \cdot d)$ tiempo, $\mathcal{O}(n^2)$ espacio (materialización de la matriz de atención).
- **Complejidad de inferencia:** $\mathcal{O}(n)$ tiempo por token, $\mathcal{O}(n)$ espacio (caché KV).

- **Fortalezas:** Expresividad incomparable; recuperación perfecta del historial; altamente paralelizable.
- **Debilidades:** El cuello de botella cuadrático prohíbe contextos muy largos; el caché KV domina la memoria durante la generación.

B.1.2. Atención Sigmoides

La atención sigmoides reemplaza el softmax por filas con una activación sigmoides elemento a elemento [31]:

- 1: **Entrada:** $\mathbf{Q}, \mathbf{K}, \mathbf{V}$ como arriba, sesgo aprendible $\mathbf{b} \in \mathbb{R}^{n \times n}$
- 2: $\text{logits} \leftarrow \frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d}} + \mathbf{b}$
- 3: $\text{pesos_atencion} \leftarrow \sigma(\text{logits})$ ▷ sigmoides elemento a elemento, no softmax
- 4: **Salida:** $\mathbf{Y} = \text{pesos_atencion} \odot \mathbf{V}$

A diferencia de softmax, la sigmoides no impone una distribución de probabilidad (los pesos no necesitan sumar 1), lo que permite una independencia de tokens más fuerte. El análisis teórico mediante mezcla de expertos muestra que la sigmoides alcanza una complejidad muestral superior: convergencia $\mathcal{O}(n^{-0.51})$ para expertos ReLU frente a $\mathcal{O}(n^{-0.24})$ de softmax. Sin embargo, el entrenamiento empírico reveló inestabilidades de gradiente a escala. El remedio es la **norma híbrida**—agregar normalización después de la salida de atención—que estabiliza los gradientes sin sacrificar los beneficios teóricos del enrutamiento elemento a elemento.

- **Complejidad de entrenamiento:** $\mathcal{O}(n^2 \cdot d)$ (idéntico al estándar), pero las operaciones elemento a elemento permiten una aceleración de inferencia del 17 % mediante FlashSigmoid.
- **Complejidad de inferencia:** $\mathcal{O}(n)$ por token con caché KV (asintóticamente igual, pero con factores constantes más bajos).
- **Fortalezas:** Supera la competencia de suma cero; evita la sincronización por filas; implementación eficiente en hardware.
- **Debilidades:** Requiere estabilización cuidadosa (norma híbrida); inestabilidad de entrenamiento a grandes escalas sin pérdida auxiliar.

B.2. Arquitecturas Recurrentes y Retentivas

B.2.1. Redes Retentivas (RetNet)

RetNet [37] unifica tres paradigmas de cómputo: entrenamiento paralelo, inferencia recurrente y despliegue por fragmentos. Su innovación central es el **mecanismo de retención**, que utiliza una matriz de decaimiento exponencial fija para modelar la importancia temporal:

- 1: **Forma paralela (entrenamiento):**
- 2: $\text{matriz_decaimiento}[i, j] \leftarrow \gamma^{i-j}$ para $i \geq j$, si no 0 ▷ decaimiento exponencial causal
- 3: $\text{matriz_decaimiento}[i, j] \leftarrow 0$ para $i < j$ ▷ enmascaramiento causal
- 4: $\mathbf{Y}_{\text{paralelo}} \leftarrow (\mathbf{Q}\mathbf{K}^\top \odot \text{matriz_decaimiento})\mathbf{V}$ ▷ multiplicación elemento a elemento con decaimiento
- 1: **Forma recurrente (inferencia):**
- 2: **for** $t = 1, \dots, n$ **do**
- 3: $\mathbf{s}_t \leftarrow \gamma \mathbf{s}_{t-1} + \mathbf{k}_t \mathbf{v}_t^\top$ ▷ actualización de estado con decaimiento
- 4: $\mathbf{y}_t \leftarrow \mathbf{q}_t \mathbf{s}_t$ ▷ salida mediante interacción consulta-estado
- 5: **end for**

El escalar de decaimiento $\gamma \in (0, 1)$ controla la ventana temporal. RetNet utiliza **retención multi-escala** con diferentes valores de γ por cabeza (ej., $\gamma = 1 - 2^{-5}, 1 - 2^{-6}, \dots$), permitiendo

dependencias a corto y largo plazo simultáneamente. El modo recurrente por fragmentos divide las secuencias en fragmentos, procesa cada fragmento en paralelo y enhebra un estado recurrente entre fragmentos.

- **Complejidad de entrenamiento:** $\mathcal{O}(n^2 \cdot d)$ (paralela), o $\mathcal{O}(n \cdot c \cdot d)$ (recurrente por fragmentos con tamaño de fragmento c).
- **Complejidad de inferencia:** $\mathcal{O}(1)$ por token, $\mathcal{O}(d^2)$ espacio de estado (matriz fija).
- **Fortalezas:** Inferencia en tiempo constante; triple paradigma de cómputo; consolidación multi-escala.
- **Debilidades:** El decaimiento fijo impone un sesgo inductivo rígido; puede truncar patrones de largo alcance aprendidos.

B.2.2. Mamba: Modelos de Espacio de Estado Selectivos

Mamba [13] enmarca la recurrencia como un sistema dinámico de tiempo continuo con parámetros dependientes de la entrada, logrando tanto entrenamiento lineal como inferencia en tiempo constante:

- 1: **Dinámica continua:** $h'(t) = \mathbf{A}h(t) + \mathbf{B}x(t), \quad y(t) = \mathbf{C}h(t)$
- 2: **Discretización con tamaño de paso Δ_t :**
- 3: $\bar{\mathbf{A}}_t \leftarrow \exp(\Delta_t \mathbf{A})$
- 4: $\bar{\mathbf{B}}_t \leftarrow (\Delta_t \mathbf{A})^{-1}(\exp(\Delta_t \mathbf{A}) - \mathbf{I})\Delta_t \mathbf{B}_t$
- 5: **Actualización recurrente:**
- 6: **for** $t = 1, \dots, n$ **do**
- 7: $\Delta_t \leftarrow \text{softplus}(\text{Lineal}(x_t))$ ▷ tamaño de paso dependiente de la entrada
- 8: $\mathbf{B}_t \leftarrow \text{Lineal}(x_t), \quad \mathbf{C}_t \leftarrow \text{Lineal}(x_t)$ ▷ matrices de proyección dependientes de la entrada
- 9: $h_t \leftarrow \bar{\mathbf{A}}_t h_{t-1} + \bar{\mathbf{B}}_t x_t$ ▷ transición de estado
- 10: $y_t \leftarrow \mathbf{C}_t h_t$ ▷ proyección de salida
- 11: **end for**

La innovación crítica es que $\mathbf{A}$ (la matriz del sistema) *no* depende de la entrada, pero Δ_t , $\mathbf{B}_t$ y $\mathbf{C}_t$ sí, haciendo que el sistema sea **variante en el tiempo**. Esta selectividad permite al modelo *ignorar* información irrelevante estableciendo Δ_t cerca de cero, creando efectivamente una compuerta. Un algoritmo de escaneo paralelo consciente del hardware implementa la recurrencia eficientemente en GPUs fusionando el cómputo dentro de SRAM, evitando el costoso ancho de banda de HBM.

- **Complejidad de entrenamiento:** $\mathcal{O}(n \cdot d)$ mediante escaneo consciente del hardware (lineal en la longitud de la secuencia).
- **Complejidad de inferencia:** $\mathcal{O}(1)$ por token, $\mathcal{O}(d)$ estado (vector oculto).
- **Fortalezas:** Logra entrenamiento lineal e inferencia constante simultáneamente; práctico en secuencias largas (millones de tokens).
- **Debilidades:** La compresión del vector de estado puede debilitar la copia exacta y la recuperación asociativa densa en comparación con la atención completa.

B.3. Transformers de Profundidad Continua: Integración EDO

B.3.1. Transformer EDO

El Transformer EDO [53] interpreta la profundidad de la red como integración numérica de un sistema dinámico continuo, utilizando solucionadores Runge-Kutta de orden superior para reducir el error de truncamiento:

- 1: **Formulación continua:** $\frac{dh(t)}{dt} = f_\theta(h(t), t)$ donde f_θ es la subred del transformer
- 2: **Aproximación discreta Runge-Kutta-4:**
- 3: $k_1 \leftarrow f_\theta(h_t, t)$
- 4: $k_2 \leftarrow f_\theta(h_t + \frac{1}{2}k_1, t + \frac{1}{2}\Delta t)$
- 5: $k_3 \leftarrow f_\theta(h_t + \frac{1}{2}k_2, t + \frac{1}{2}\Delta t)$
- 6: $k_4 \leftarrow f_\theta(h_t + k_3, t + \Delta t)$
- 7: $h_{t+1} \leftarrow h_t + \frac{\Delta t}{6}(k_1 + 2k_2 + 2k_3 + k_4)$ ▷ combinación ponderada

En lugar de las conexiones residuales simples de Euler $h_{t+1} = h_t + f_\theta(h_t)$, el bloque RK4 calcula cuatro evaluaciones intermedias y las combina con los pesos clásicos de RK4. Para evitar gradientes que se desvanecen, la arquitectura introduce **compuertas aprendidas** que interpolar entre aproximaciones intermedias:

- 1: $g \leftarrow \sigma(\text{Lineal}([k_1, k_2, k_3, k_4]))$ ▷ compuerta aprendible
- 2: $h_{t+1} \leftarrow h_t + g \cdot k_1 + (1 - g) \cdot k_2$ ▷ refinamiento interpolado

Esta formulación reduce el número efectivo de parámetros mediante el uso compartido de pesos—la misma f_θ se evalúa múltiples veces—proporcionando un refinamiento de trayectoria más rico.

- **Complejidad de entrenamiento:** $\mathcal{O}(k \cdot n^2 \cdot d)$ donde k es el orden RK (4 para RK4).
- **Complejidad de inferencia:** $\mathcal{O}(k \cdot n)$ por token (mayor sobrecarga constante).
- **Fortalezas:** Precisión significativamente mayor en tareas de generación (BLEU de última generación); eficiente en parámetros mediante uso compartido de pesos.
- **Debilidades:** Alto costo de cómputo por paso; la latencia de inferencia aumenta por un factor constante k ; requiere compuertas complejas para la estabilidad.

B.4. Memoria en Tiempo de Prueba: Titans

La arquitectura Titans [3] introduce una dimensión ortogonal: en lugar de pesos estáticos, el modelo mantiene una memoria aprendible que se *actualiza durante la inferencia* basada en una señal impulsada por sorpresa:

- 1: **Atención local a corto plazo:** Aplicar atención estándar o dispersa dentro de una ventana de contexto fija c
- 2: $y_t^{\text{local}} \leftarrow \text{Atencion}(q_t, k_{[t-c:t]}, v_{[t-c:t]})$
- 3: **Señal de actualización de memoria (sorpresa):**
- 4: $S_t \leftarrow \eta_t S_{t-1} - \theta_t \nabla_\ell(\mathcal{M}_{t-1}; x_t)$ ▷ sorpresa como gradiente de la pérdida respecto a la memoria
- 5: $\mathcal{M}_t \leftarrow (1 - \alpha_t)\mathcal{M}_{t-1} + S_t$ ▷ memoria actualizada mediante EMA
- 6: **Recuperación de memoria a largo plazo:**
- 7: $y_t^{\text{memoria}} \leftarrow \mathcal{M}_t^*(q_t)$ ▷ consultar módulo de memoria para información recuperada
- 8: **Combinación de salida:**
- 9: $y_t \leftarrow y_t^{\text{local}} + \text{compuerta}(y_t^{\text{memoria}})$ ▷ combinar memoria local y recuperada

El módulo de memoria $\mathcal{M}$ se actualiza literalmente durante el paso hacia adelante calculando gradientes de una pérdida asociativa y aplicando pasos de SGD con momento. Las tasas de

decaimiento $\eta_t, \theta_t, \alpha_t$ son en sí mismas dependientes de la entrada, permitiendo al modelo cambiar de paradigma de memoria cuando el contexto cambia.

- **Complejidad de entrenamiento:** Aproximadamente $\mathcal{O}(c^2 + n \cdot f)$ donde c es la ventana local y f es la sobrecarga de actualización de memoria.
- **Complejidad de inferencia:** $\mathcal{O}(c^2)$ atención local más $\mathcal{O}(1)$ recuperación de memoria por token.
- **Fortalezas:** Maneja longitudes de contexto extremas; permite recuperación asociativa real; la memoria se adapta a la entrada.
- **Debilidades:** Significativamente más complejo; la inferencia incluye cálculo de gradientes; mayor sobrecarga de coordinación.

B.5. Comparación y Síntesis Arquitectónica

Arquitectura	Entrenar	Inferir	Estado	Característica Clave
Atención Estándar	$O(n^2d)$	$O(n)$ caché	$O(nd)$	Expresividad perfecta, enrutamiento completo de tokens, cuello de botella KV.
Atención Sigmoide	$O(n^2d)$	$O(n)$ caché	$O(nd)$	Compuerta elemento a elemento, eficiente en hardware, estable a escala con norma híbrida.
RetNet	$O(n^2d)$	$O(1)$	$O(d^2)$	Decaimiento multi-escala, triple modo de cómputo, patrón de olvido fijo.
Mamba	$O(nd)$	$O(1)$	$O(d)$	Dinámica selectiva dependiente de la entrada, entrenamiento e inferencia lineales, escaneo de hardware.
Transformer EDO	$O(kn^2d)$	$O(kn)$	$O(nd)$	Refinamiento por integración numérica, uso compartido de pesos por etapas, mayor precisión.
Titans	$O(c^2 + nf)$	$O(c^2 + 1)$	$O(1)$	Adaptación de memoria en tiempo de prueba, contexto extremo, actualizaciones de parámetros durante inferencia.

El campo exhibe una progresión clara a lo largo de dos ejes: (i) **eficiencia computacional**, pasando de $O(n^2)$ a $O(n)$ u $O(1)$, y (ii) **adaptabilidad de memoria**, pasando de pesos estáticos a modelos dinámicos actualizados en tiempo de prueba. La atención estándar sigue siendo la línea base de expresividad; Mamba y RetNet representan la frontera práctica de eficiencia; Titans introduce un eje de innovación ortogonal (aprendizaje en tiempo de prueba). La elección de arquitectura refleja la restricción fundamental de ingeniería: expresividad versus costo de despliegue.

C. Annex C: Comprehensive Sparse Attention Mechanisms

C.1. Resumen Ejecutivo

Los mecanismos de atención dispersa abordan la prohibitiva complejidad computacional y de memoria $\mathcal{O}(n^2)$ de la atención de producto punto escalado estándar al restringir el conjunto de pares clave-valor atendidos. La atención dispersa moderna abarca un rico espacio de diseño distinguido por cuatro dimensiones ortogonales: (i) **patrón de dispersidad** (geométrico fijo, dependiente de datos o basado en frecuencia), (ii) **capacidad de entrenamiento** (entrenado de extremo a extremo o solo inferencia), (iii) **unidad de dispersidad** (tokens individuales, ventanas locales o bloques), y (iv) **mecanismo** (enmascaramiento, selección, filtrado o compresión). Este anexo sintetiza siete familias principales de atención dispersa—Sparse Transformer, Longformer, BigBird, SparseK, NSA, SpargeAttn y FASA—proporcionando formulaciones matemáticas, pseudocódigo algorítmico y comparaciones arquitectónicas exhaustivas.

C.2. Sparse Transformer: Patrones Estrideados y Fijos Factorizados

C.2.1. Formulación Matemática

Sparse Transformer [10] reduce la complejidad cuadrática a $\mathcal{O}(n\sqrt{n})$ factorizando la atención dispersa en dos cabezas dispersas complementarias. Sea n la longitud de la secuencia y $l = \lfloor \sqrt{n} \rfloor$ el stride.

Cabeza de atención estrideada : Cada posición i atiende a cada l -ésima posición anterior:

$$\mathcal{A}_i^{(\text{estrídeada})} = \{j : j \leq i, (i - j) \bmod l = 0\}$$

Cabeza de atención fija : Cada posición i atiende a posiciones dentro de su bloque local más columnas de resumen fijas:

$$\mathcal{A}_i^{(\text{fija})} = \{j : \lfloor j/l \rfloor = \lfloor i/l \rfloor\} \cup \{j : j \bmod l \in \{l - c, \dots, l - 1\}\}$$

donde c es un hiperparámetro que controla el número de columnas de resumen (típicamente $c = 1$ o $c = 2$).

El cálculo de atención para cada cabeza h sigue las reglas estándar de producto punto escalado restringido al conjunto disperso:

$$\text{Attn}_h(Q, K, V)_i = \text{softmax} \left(\frac{q_i^h K_{\mathcal{A}_i}^h \top}{\sqrt{d_k}} \right) V_{\mathcal{A}_i}^h$$

La idea clave es que con las dos cabezas factorizadas operando en paralelo a través de una profundidad de L capas transformer, cada posición puede alcanzar cualquier otra posición a través de un camino de longitud máxima $L + 1$, preservando la alcanzabilidad de largo alcance con una fracción del costo de la atención densa.

C.2.2. Pseudocódigo Algorítmico

Algorithm 25 Atención Sparse Transformer: Cabezas Duales Estrideada + Fija

Require: Consulta $\mathbf{Q} \in \mathbb{R}^{n \times d}$, Clave $\mathbf{K} \in \mathbb{R}^{n \times d}$, Valor $\mathbf{V} \in \mathbb{R}^{n \times d}$

Require: Stride $l = \lfloor \sqrt{n} \rfloor$, columnas de resumen $c \in \{1, 2\}$

Ensure: Salida $\mathbf{Y} \in \mathbb{R}^{n \times d}$

```

1: Cabeza 1: Atención Estrideada
2: for  $i = 1, \dots, n$  do
3:    $\mathcal{A}_i \leftarrow \{j : j \leq i \text{ y } (i - j) \bmod l = 0\}$  ▷ Cada  $l$ -ésima posición
4:    $\text{puntajes}_i \leftarrow \mathbf{q}_i \mathbf{K}_{\mathcal{A}_i}^\top / \sqrt{d_k}$ 
5:    $\text{attn}_i \leftarrow \text{softmax}(\text{puntajes}_i)$ 
6:    $\mathbf{y}_i^{(1)} \leftarrow \text{attn}_i \mathbf{V}_{\mathcal{A}_i}$ 
7: end for
8: Cabeza 2: Atención de Bloque Fijo + Resumen
9: for  $i = 1, \dots, n$  do
10:   $\text{inicio\_bloque} \leftarrow \lfloor i/l \rfloor \cdot l$ ,  $\text{fin\_bloque} \leftarrow \min(i + 1, \text{inicio\_bloque} + l)$ 
11:   $\mathcal{A}_i \leftarrow [\text{inicio\_bloque}, \text{fin\_bloque}) \cup \{\text{columnas de las últimas } c \text{ columnas}\}$ 
12:   $\text{puntajes}_i \leftarrow \mathbf{q}_i \mathbf{K}_{\mathcal{A}_i}^\top / \sqrt{d_k}$ 
13:   $\text{attn}_i \leftarrow \text{softmax}(\text{puntajes}_i)$ 
14:   $\mathbf{y}_i^{(2)} \leftarrow \text{attn}_i \mathbf{V}_{\mathcal{A}_i}$ 
15: end for
16: Concatenar:  $\mathbf{Y} = [\mathbf{y}^{(1)} \parallel \mathbf{y}^{(2)}]$  y proyectar mediante capa lineal de salida return  $\mathbf{Y}$ 

```

C.2.3. Características Clave

- **Complejidad:** $\mathcal{O}(n\sqrt{n} \cdot d)$ durante el entrenamiento; $\mathcal{O}(n)$ por token durante la generación.
- **Entrenable:** Sí; retropropagación completa a través de las posiciones seleccionadas.
- **Patrón de dispersidad:** Fijo e independiente de los datos; los patrones no se adaptan al contenido.
- **Fortaleza:** Alcanzabilidad estructurada que garantiza dependencias de largo alcance; demostrado efectivo en secuencias largas (Enwik8, imágenes de texto).
- **Limitación:** Los patrones fijos pueden pasar por alto relaciones importantes pero no contiguas; requiere kernels CUDA personalizados para eficiencia práctica.

C.3. Longformer: Ventana Deslizante, Dilatación y Tokens Globales

C.3.1. Formulación Matemática

Longformer [4] logra una complejidad lineal $\mathcal{O}(n \cdot w)$ combinando tres patrones de atención complementarios.

Atención de ventana deslizante : Vecindario local de tamaño w :

$$\mathcal{A}_i^{(\text{ventana})} = \{j : |i - j| \leq w/2\}$$

Ventana deslizante dilatada : Atención con ventana y dilatación d , saltando stride $d + 1$:

$$\mathcal{A}_i^{(\text{dilatada})} = \{j : |i - j| \leq (w/2)(d + 1) \text{ y } (i - j) \bmod (d + 1) = 0\}$$

Atención global : Tokens globales designados (ej., [CLS]) atienden a todas las posiciones y todos atienden a ellos:

$$\mathcal{A}_i^{(\text{global})} = \begin{cases} \{1, \dots, n\} & \text{si } i \in \mathcal{G} \\ \mathcal{A}_i^{(\text{ventana})} \cup \mathcal{G} & \text{en otro caso} \end{cases}$$

Los tres patrones se aplican mediante proyecciones separadas. La atención local y global pueden usar las mismas proyecciones o unas separadas específicas para cada tarea.

C.3.2. Pseudocódigo Algorítmico

Algorithm 26 Longformer: Ventana Deslizante + Dilatada + Global

Require: Consulta $\mathbf{Q} \in \mathbb{R}^{n \times d}$, Clave $\mathbf{K}$, Valor $\mathbf{V}$, tamaño de ventana w , dilatación d , índices de tokens globales $\mathcal{G}$

Ensure: Salida $\mathbf{Y}$

```

1: for  $i = 1, \dots, n$  do
2:   if  $i \in \mathcal{G}$  then
3:      $\mathcal{A}_i \leftarrow \{1, \dots, n\}$  ▷ Token global atiende a todos
4:   else
5:     Ventana local:  $\mathcal{A}^{(\text{local})} \leftarrow [i - w/2, i + w/2] \cap [1, n]$ 
6:     Desplazamientos dilatados:  $\mathcal{A}^{(\text{dilatada})} \leftarrow \{j : (i - j) \bmod (d + 1) = 0\} \cap \mathcal{A}^{(\text{rango dilatado})}$ 
7:     Combinar:  $\mathcal{A}_i \leftarrow \mathcal{A}^{(\text{local})} \cup \mathcal{A}^{(\text{dilatada})} \cup \mathcal{G}$ 
8:   end if
9:    $\text{puntajes}_i \leftarrow \mathbf{q}_i \mathbf{K}_{\mathcal{A}_i}^\top / \sqrt{d_k}$ 
10:   $\text{attn}_i \leftarrow \text{softmax}(\text{puntajes}_i)$ 
11:   $\mathbf{y}_i \leftarrow \text{attn}_i \mathbf{V}_{\mathcal{A}_i}$ 
12: end for return  $\mathbf{Y}$ 

```

C.3.3. Características Clave

- **Complejidad:** $\mathcal{O}(n \cdot w)$ donde w es el tamaño de ventana (lineal cuando $w \ll n$).
- **Entrenable:** Sí; reemplazo directo para la atención estándar.
- **Cobertura:** Con L capas y tamaño de ventana w , el campo receptivo crece a $L \times w$, cubriendo toda la secuencia con redes poco profundas.
- **Fortaleza:** Práctico para tareas a nivel de documentos; la dilatación expande los campos receptivos locales con sobrecarga mínima; los tokens globales proporcionan correspondencia consulta-documento.
- **Limitación:** El tamaño de ventana sigue siendo un hiperparámetro de diseño; subóptimo para tareas que requieren patrones de atención densos.

C.4. BigBird: Grafo Disperso Aleatorio, Local y Global

C.4.1. Formulación Matemática

BigBird [51] preserva la aproximación universal y la completitud de Turing utilizando una combinación fundamentada de tres patrones de dispersidad.

Conexiones aleatorias : Cada posición se conecta a r posiciones muestreadas aleatoriamente:

$$\mathcal{A}_i^{(\text{aleatoria})} = \text{MuestraAleatoria}(\{1, \dots, n\}, r)$$

Ventana local : Vecindario de ventana deslizante:

$$\mathcal{A}_i^{(\text{local})} = \{j : |i - j| \leq w/2\}$$

Tokens globales : Ancas globales específicas de la tarea:

$$\mathcal{A}_i^{(\text{global})} = \begin{cases} \{1, \dots, n\} & \text{si } i \in \mathcal{G} \\ \mathcal{A}_i^{(\text{aleatoria})} \cup \mathcal{A}_i^{(\text{local})} \cup \mathcal{G} & \text{en otro caso} \end{cases}$$

La máscara dispersa compuesta M es:

$$M_{ij} = \mathbf{1}[j \in \mathcal{A}_i^{(\text{aleatoria})} \cup \mathcal{A}_i^{(\text{local})} \cup \mathcal{A}_i^{(\text{global})}]$$

En la práctica, BigBird agrupa tokens en bloques de tamaño b y aplica la decisión de dispersidad a nivel de bloque, permitiendo implementaciones eficientes de bloques dispersos.

C.4.2. Garantía Teórica

Los autores demuestran que con $O(1)$ conexiones aleatorias y tokens globales, el grafo disperso mantiene las mismas propiedades de aproximación universal y completitud de Turing que la atención densa. Formalmente, cualquier función computable puede representarse y cualquier secuencia de operaciones puede simularse dentro del grafo de conectividad de BigBird.

C.4.3. Pseudocódigo Algorítmico

Algorithm 27 BigBird: Atención Dispersa por Bloques Aleatoria + Local + Global

Require: Consulta $\mathbf{Q}$, Clave $\mathbf{K}$, Valor $\mathbf{V}$, tamaño de bloque b , enlaces aleatorios por bloque r , índices globales $\mathcal{G}$

Ensure: Salida $\mathbf{Y}$

```

1: Reformar en bloques:  $n_b = \lceil n/b \rceil$  bloques
2: for bloque  $i = 0, \dots, n_b - 1$  do
3:   for posición  $j$  en bloque  $i$  do
4:     Ventana local:  $\mathcal{A}_j^{(\text{local})} =$  posiciones cercanas en el bloque  $\cup$  posiciones frontera
5:     Muestreo aleatorio de bloques: Muestrear  $r$  bloques uniformemente al azar, agregar todas las posiciones de esos bloques
6:     Global: Agregar todas las posiciones de tokens globales
7:      $\mathcal{A}_j \leftarrow \mathcal{A}_j^{(\text{local})} \cup \mathcal{A}_j^{(\text{aleatoria})} \cup \mathcal{G}$ 
8:   end for
9: end for
10: for  $i = 1, \dots, n$  do
11:    $\text{puntajes}_i \leftarrow \mathbf{q}_i \mathbf{K}_{\mathcal{A}_i}^\top / \sqrt{d_k}$ 
12:    $\text{attn}_i \leftarrow \text{softmax}(\text{puntajes}_i)$ 
13:    $\mathbf{y}_i \leftarrow \text{attn}_i \mathbf{V}_{\mathcal{A}_i}$ 
14: end for return  $\mathbf{Y}$ 
```

C.4.4. Características Clave

- **Complejidad:** $\mathcal{O}(n)$ o casi lineal en implementación óptima de bloques dispersos.
- **Entrenable:** Sí.
- **Base teórica:** Mantiene demostrablemente la aproximación universal y la completitud de Turing con conectividad dispersa.

- **Fortaleza:** Rendimiento sólido en documentos largos para preguntas-respuestas, resúmenes y tareas de genómica.
- **Limitación:** El muestreo aleatorio introduce varianza y no determinismo; el ajuste de los hiperparámetros r, w, g sigue dependiendo de la tarea.

C.5. Atención SparseK: Selección Diferencial Top- k

C.5.1. Formulación Matemática

SparseK [24] permite dispersidad aprendible mediante un **operador top- k diferenciable**. Una red de puntuación ϕ_θ evalúa la importancia de cada par clave-valor:

$$u_j = \phi_\theta(\mathbf{k}_j, \mathbf{q}_i) \in \mathbb{R}$$

El **operador SparseK** selecciona los puntajes top- k manteniéndose diferenciable. Calcula un umbral $\tau(u)$ tal que la suma de los puntajes activos sea igual a k :

$$\text{SparseK}(u, k)_j = \max(u_j - \tau(u), 0), \quad \text{donde} \quad \sum_j \max(u_j - \tau, 0) = k$$

El umbral τ se encuentra mediante bisección. La atención se convierte en:

$$m_j = \text{SparseK}(u, k)_j, \quad \text{Attn}_i = \text{softmax} \left(\frac{\mathbf{q}_i K_{\text{sel}}^\top}{\sqrt{d_k}} \right) V_{\text{sel}}$$

donde $K_{\text{sel}}, V_{\text{sel}}$ contienen solo las entradas top- k (aquellas con $m_j > 0$).

C.5.2. Pseudocódigo Algorítmico

Algorithm 28 SparseK: Atención Top- k Diferenciable

Require: Consulta $\mathbf{q} \in \mathbb{R}^d$, Matriz de clave $\mathbf{K} \in \mathbb{R}^{n \times d}$, Matriz de valor $\mathbf{V} \in \mathbb{R}^{n \times d}$

Require: Red de puntuación ϕ_θ , dispersidad objetivo k

Ensure: Salida de atención $\mathbf{y}$

```

1: Calcular puntajes de importancia:
2: for  $j = 1, \dots, n$  do
3:    $u_j \leftarrow \phi_\theta(\mathbf{k}_j, \mathbf{q})$ 
4: end for
5: Calcular top- $k$  diferenciable mediante umbral:
6:  $u_{\text{ordenado}} \leftarrow \text{ordenar}(u, \text{descendente} = \text{Verdadero})$ 
7:  $\text{suma\_acum} \leftarrow \text{suma\_acumulada}(u_{\text{ordenado}})$ 
8: Encontrar  $\rho = \max\{i : u_{\text{ordenado}}[i] > 0 \text{ y } \text{suma\_acum}[i] \leq k\}$ 
9:  $\tau \leftarrow (u_{\text{ordenado}}[\rho] - k) / (\rho + 1)$  ▷ Umbral para  $k$  elementos activos
10:  $m \leftarrow \max(u - \tau, 0)$  ▷ Máscara de selección diferenciable
11: Aplicar máscara y calcular atención:
12:  $K_{\text{sel}} \leftarrow K[m > 0], V_{\text{sel}} \leftarrow V[m > 0]$ 
13:  $\text{puntajes} \leftarrow \mathbf{q} K_{\text{sel}}^\top / \sqrt{d_k}$ 
14:  $\text{attn} \leftarrow \text{softmax}(\text{puntajes})$ 
15:  $\mathbf{y} \leftarrow \text{attn} \cdot V_{\text{sel}}$  return  $\mathbf{y}$ 

```

C.5.3. Características Clave

- **Complejidad:** $\mathcal{O}(n \cdot d)$ durante el entrenamiento (lineal en la longitud de la secuencia); $\mathcal{O}(k)$ por token durante la generación.

- **Entrenable:** Sí; flujo de gradiente de extremo a extremo a través del operador SparseK.
- **Generación incremental:** Soporta generación autorregresiva eficiente con memoria constante.
- **Fortaleza:** Se integra perfectamente en arquitecturas LLM existentes; mínimo ajuste fino necesario.
- **Limitación:** El acceso a memoria disperso de la selección top- k puede limitar la eficiencia de caché en algunos hardwares; la red de puntuación añade sobrecarga.

C.6. NSA (Native Sparse Attention): Ramas Jerárquicas Alineadas con el Hardware

C.6.1. Formulación Matemática

NSA [49] descompone la atención dispersa en tres ramas paralelas que se combinan mediante compuertas aprendidas.

Rama de compresión : Un MLP aprendible φ comprime bloques de clave-valor:

$$\tilde{K}_t^{\text{cmp}} = [\varphi(k_{id+1:id+l})]_{1 \leq i \leq \lfloor (t-l)/d \rfloor}, \quad \tilde{V}_t^{\text{cmp}} = [\varphi(v_{id+1:id+l})]$$

Rama de selección : Los bloques de alta importancia se seleccionan basándose en puntajes comprimidos:

$$p_t^{\text{cmp}} = \text{softmax}(q_t^\top \tilde{K}_t^{\text{cmp}} / \sqrt{d}), \quad I_t = \text{TopK}(p_t^{\text{cmp}}, n), \quad \tilde{K}_t^{\text{sel}} = \text{Recolectar}(K, I_t)$$

Rama de ventana deslizante : Contexto local fijo:

$$\tilde{K}_t^{\text{win}} = K_{t-w:t}, \quad \tilde{V}_t^{\text{win}} = V_{t-w:t}$$

Combinación con compuertas : Las tres salidas de atención se combinan mediante compuertas aprendidas:

$$\alpha_t^{(\text{cmp})} = \sigma(\text{Lineal}_{\text{cmp}}(q_t)), \quad \alpha_t^{(\text{sel})} = \sigma(\text{Lineal}_{\text{sel}}(q_t)), \quad \alpha_t^{(\text{win})} = \sigma(\text{Lineal}_{\text{win}}(q_t))$$

$$y_t = \alpha_t^{(\text{cmp})} \cdot \text{Attn}(q_t, \tilde{K}^{\text{cmp}}, \tilde{V}^{\text{cmp}}) + \alpha_t^{(\text{sel})} \cdot \text{Attn}(q_t, \tilde{K}^{\text{sel}}, \tilde{V}^{\text{sel}}) + \alpha_t^{(\text{win})} \cdot \text{Attn}(q_t, \tilde{K}^{\text{win}}, \tilde{V}^{\text{win}})$$

C.6.2. Pseudocódigo Algorítmico

Algorithm 29 NSA: Ramas Comprimida + Seleccionada + Ventana con Compuertas Aprendidas

Require: Consulta $\mathbf{q}_t$, secuencias de Clave/Valor almacenadas en caché, tamaño de bloque l , stride d , conteo de selección n , ventana w

Require: MLP de compresión φ , redes de compuerta $\text{Lineal}_{\text{cmp}}$, $\text{Lineal}_{\text{sel}}$, $\text{Lineal}_{\text{win}}$

Ensure: Salida $\mathbf{y}_t$

```

1: Rama de compresión:
2: for  $i = 1$  to  $\lfloor t/d \rfloor$  do
3:    $k_i^{\text{cmp}} \leftarrow \varphi(\mathbf{K}_{id+1:id+l})$  ▷ Comprimir bloque mediante MLP
4:    $v_i^{\text{cmp}} \leftarrow \varphi(\mathbf{V}_{id+1:id+l})$ 
5: end for
6:  $\tilde{\mathbf{K}}^{\text{cmp}} \leftarrow [k_1^{\text{cmp}}, \dots, k_{\lfloor t/d \rfloor}^{\text{cmp}}]$ ,  $\tilde{\mathbf{V}}^{\text{cmp}} \leftarrow [v_1^{\text{cmp}}, \dots, v_{\lfloor t/d \rfloor}^{\text{cmp}}]$ 
7:  $\mathbf{y}_t^{(\text{cmp})} \leftarrow \text{SDPA}(\mathbf{q}_t, \tilde{\mathbf{K}}^{\text{cmp}}, \tilde{\mathbf{V}}^{\text{cmp}})$ 
8: Rama de selección:
9: Calcular puntajes sobre comprimidos:  $\mathbf{s} \leftarrow \text{softmax}(\mathbf{q}_t \tilde{\mathbf{K}}^{\text{cmp}\top} / \sqrt{d})$ 
10: Seleccionar los  $n$  bloques más importantes:  $I \leftarrow \text{TopK}(\mathbf{s}, n)$ 
11: Recolectar bloques completos:  $\tilde{\mathbf{K}}^{\text{sel}} \leftarrow [\mathbf{K}_{I_i d+1:I_i d+l}]_i$ ,  $\tilde{\mathbf{V}}^{\text{sel}} \leftarrow [\mathbf{V}_{I_i d+1:I_i d+l}]_i$ 
12:  $\mathbf{y}_t^{(\text{sel})} \leftarrow \text{SDPA}(\mathbf{q}_t, \tilde{\mathbf{K}}^{\text{sel}}, \tilde{\mathbf{V}}^{\text{sel}})$ 
13: Rama de ventana:
14:  $\mathbf{y}_t^{(\text{win})} \leftarrow \text{SDPA}(\mathbf{q}_t, \mathbf{K}_{t-w:t}, \mathbf{V}_{t-w:t})$ 
15: Fusión con compuertas:
16:  $\alpha^{(\text{cmp})} \leftarrow \sigma(\text{Lineal}_{\text{cmp}}(\mathbf{q}_t))$ ,  $\alpha^{(\text{sel})} \leftarrow \sigma(\text{Lineal}_{\text{sel}}(\mathbf{q}_t))$ ,  $\alpha^{(\text{win})} \leftarrow \sigma(\text{Lineal}_{\text{win}}(\mathbf{q}_t))$ 
17:  $\mathbf{y}_t \leftarrow \alpha^{(\text{cmp})} \mathbf{y}_t^{(\text{cmp})} + \alpha^{(\text{sel})} \mathbf{y}_t^{(\text{sel})} + \alpha^{(\text{win})} \mathbf{y}_t^{(\text{win})}$  return  $\mathbf{y}_t$ 

```

C.6.3. Características Clave

- **Complejidad:** $\mathcal{O}(t/d + n \cdot l + w)$ tokens atendidos por paso (significativamente reducido frente a $\mathcal{O}(t)$ para densa).
- **Entrenable:** Sí; entrenamiento directo con todas las ramas diferenciables.
- **Alineación con hardware:** Diseñado para utilización eficiente de núcleos tensoriales; la compresión y selección reducen la presión sobre el ancho de banda de memoria.
- **Fortaleza:** $11.6\times$ de aceleración en decodificación y $9\times$ de aceleración hacia adelante en secuencias de 64k; supera a la atención completa en muchas tareas.
- **Limitación:** Requiere kernels Triton/CUDA personalizados; la arquitectura multi-rama compleja aumenta la sobrecarga de ingeniería.

C.7. FASA: Atención Dispersa Consciente de la Frecuencia

C.7.1. Formulación Matemática

FASA [42] es un **método en tiempo de inferencia sin entrenamiento** que explota la estructura del embedding de posición rotatorio (RoPE). Bajo RoPE, el embedding del token se rota por $\theta_i = B^{-2(i-1)/d}$, descomponiendo el espacio d -dimensional en $d/2$ fragmentos de frecuencia (FC).

Idea clave : Solo un subconjunto pequeño ($< 1\%$) de los FC importa para la conciencia contextual; la mayoría codifica patrones posicionales.

Identificación de fragmento de frecuencia dominante : Para cada capa l y cabeza h , identificar FC dominantes mediante concordancia contextual (CA):

$$\text{CA}^{l,h,i} = \frac{|\text{TopK}(\alpha^{l,h}) \cap \text{TopK}(\alpha^{l,h,i})|}{K}$$

donde $\alpha^{l,h}$ es la máscara de atención completa y $\alpha^{l,h,i}$ es la máscara usando solo el FC i . Los FC dominantes tienen CA alta—contribuyen significativamente al patrón de atención final.

Eta de predicción de importancia de tokens (TIP) : Usando solo FC dominantes, calcular importancia ligera por token:

$$S_t^{l,h} = \sum_{i \in \mathcal{I}_{\text{dom}}^{l,h}} \alpha^{l,h,i}(\mathbf{q}_t, \mathbf{K}_{1:t}), \quad \mathcal{T}_t = \text{TopK-Índices}(S_t, N_{\text{fac}})$$

Eta de cálculo de atención enfocada (FAC) : Calcular atención de precisión completa en los tokens seleccionados:

$$\hat{\alpha}_{\text{FAC}} = \text{softmax} \left(\frac{\mathbf{q}_t \mathbf{K}_{\mathcal{T}_t}^\top}{\sqrt{d}} \right), \quad \mathbf{o}_t = \hat{\alpha}_{\text{FAC}} \mathbf{V}_{\mathcal{T}_t}$$

C.7.2. Pseudocódigo Algorítmico

Algorithm 30 FASA: Atención Dispersa Consciente de la Frecuencia (en Tiempo de Inferencia)

Require: Consulta actual $\mathbf{q}_t$, Clave/Valor almacenados en caché $\mathbf{K}_{1:t}, \mathbf{V}_{1:t}$, base RoPE B , FC dominantes $\mathcal{I}_{\text{dom}}$

Require: Presupuesto TIP N_{tip} , presupuesto FAC N_{fac}

Ensure: Salida $\mathbf{o}_t$

```

1: Eta 1: Predicción de Importancia de Tokens (TIP)
2: for capa  $l$  y cabeza  $h$  do
3:   for posición  $i = 1$  to  $t$  do
4:     Calcular importancia desde FC dominantes:  $s_i \leftarrow 0$ 
5:     for  $fc \in \mathcal{I}_{\text{dom}}^{l,h}$  do
6:       Rotar consulta y clave en FC  $fc$ :  $\mathbf{q}^{(fc)}, \mathbf{k}_i^{(fc)}$ 
7:        $s_i^{(fc)} \leftarrow \text{softmax}(\mathbf{q}^{(fc)} \mathbf{k}_i^{(fc)\top} / \sqrt{d})$ 
8:        $s_i \leftarrow s_i + s_i^{(fc)}$ 
9:     end for
10:  end for
11:   $\mathcal{T}_t \leftarrow \text{TopK-ndices}([s_1, \dots, s_t], N_{\text{fac}})$  ▷ Seleccionar tokens principales
12: end for
13: Eta 2: Cálculo de Atención Enfocada (FAC)
14:  $\text{puntajes}_{\text{fac}} \leftarrow \mathbf{q}_t \mathbf{K}_{\mathcal{T}_t}^\top / \sqrt{d}$  ▷ Producto punto de precisión completa
15:  $\alpha_{\text{fac}} \leftarrow \text{softmax}(\text{puntajes}_{\text{fac}})$  ▷ Softmax completo
16:  $\mathbf{o}_t \leftarrow \alpha_{\text{fac}} \mathbf{V}_{\mathcal{T}_t}$  ▷ Suma ponderada de valores return  $\mathbf{o}_t$ 

```

C.7.3. Características Clave

- **Complejidad:** TIP es $\mathcal{O}(t \cdot N_{\text{tip}})$; FAC es $\mathcal{O}(N_{\text{fac}} \cdot d)$.
- **Entrenable:** No; optimización de inferencia sin entrenamiento.

- **Aplicabilidad:** Requiere modelos basados en RoPE; extendido a ALiBi y MLA con modificaciones.
- **Fortaleza:** Precisión cercana a la óptima con ≤ 256 tokens de entre millones; hasta $2.56\times$ de aceleración en decodificación; ortogonal a la cuantización.
- **Limitación:** Requiere calibración fuera de línea; la identificación de FC dominantes añade sobrecarga de preprocesamiento.

C.8. SpargeAttn: Filtrado de Dos Etapas a Nivel de Bloques

C.8.1. Formulación Matemática

SpargeAttn [54] es un **método universal sin entrenamiento** que predice qué bloques de la matriz de atención contendrán valores insignificantes y omite el cálculo para esos bloques.

Etapas 1 — Predicción dispersa : Calcular importancia proxy de bajo costo $\hat{s}_{ij}$ para el bloque (i, j) :

$$\hat{s}_{ij} = f_{\text{pred}}(\mathbf{Q}_i, \mathbf{K}_j; \text{hiperparámetros}), \quad \text{omitir si } \hat{s}_{ij} < \epsilon_1$$

Los predictores comunes incluyen similitud de media de bloque o estadísticas de autosimilitud.

Etapas 2 — Filtrado consciente de softmax : Después de calcular $\tilde{P}_{ij} = \text{softmax}(Q_i K_j^\top / \sqrt{d})$, verificar si la probabilidad máxima del bloque es insignificante en relación con el máximo de softmax en ejecución:

$$\text{omitir } P_{ij} V_j \quad \text{si} \quad \max(\tilde{P}_{ij}) < e^{m_{\text{antiguo}} - m_{\text{nuevo}}} \cdot \epsilon_2$$

donde $m_{\text{antiguo}}, m_{\text{nuevo}}$ son máximos del softmax en línea en ejecución (calculados mediante numéricos estilo FlashAttention).

C.8.2. Pseudocódigo Algorítmico

Algorithm 31 SpargeAttn: Filtrado Disperso de Dos Etapas por Bloques

Require: Bloques de consulta $\mathbf{Q}_i$ para $i = 1, \dots, n_b$, Bloques de clave $\mathbf{K}_j$ para $j = 1, \dots, n_b$, Bloques de valor $\mathbf{V}_j$

Require: Umbral de predicción ϵ_1 , umbral de softmax ϵ_2 , tamaño de bloque b_s

Ensure: Salida $\mathbf{Y}$

```

1: Inicializar: máximo de softmax en línea  $m_{\text{global}} \leftarrow -\infty$ 
2: for fila de bloque  $i = 1$  to  $n_b$  do
3:   Inicializar: salida de fila de bloque  $\mathbf{Y}_i \leftarrow 0$ , máximo local  $m_i \leftarrow -\infty$ 
4:   for columna de bloque  $j = 1$  to  $n_b$  do
5:     Etapa 1: Predicción Dispersa
6:     Calcular importancia proxy:  $\hat{s}_{ij} \leftarrow f_{\text{pred}}(\mathbf{Q}_i, \mathbf{K}_j)$ 
7:     if  $\hat{s}_{ij} < \epsilon_1$  then
8:       omitir este bloque  $[i, j]$  ▷ Terminación temprana
9:       continuar ▷ Pasar al siguiente bloque
10:    end if
11:    Etapa 2: Calcular y Filtrar
12:     $\tilde{P}_{ij} \leftarrow \text{softmax}(\mathbf{Q}_i \mathbf{K}_j^\top / \sqrt{d})$ 
13:     $m_{\text{bloque}} \leftarrow \text{máx}(\tilde{P}_{ij})$ 
14:    if  $m_{\text{bloque}} < e^{m_i - m_{\text{global}}} \cdot \epsilon_2$  then
15:      El bloque contribuye insignificamente; omitir ▷ Poda consciente de softmax
16:      continuar
17:    end if
18:    Actualizar máximo global:  $m_i \leftarrow \text{máx}(m_i, m_{\text{bloque}})$ 
19:    Acumular:  $\mathbf{Y}_i \leftarrow \mathbf{Y}_i + \tilde{P}_{ij} \mathbf{V}_j$ 
20:  end for
21:   $m_{\text{global}} \leftarrow \text{máx}(m_{\text{global}}, m_i)$ 
22: end for return  $\mathbf{Y}$ 

```

C.8.3. Características Clave

- **Complejidad:** Empírica $\mathcal{O}(n^2 \cdot s)$ donde s es la fracción de bloques no omitidos (típicamente 0.2–0.5).
- **Entrenable:** No; aceleración plug-and-play para modelos existentes.
- **Universalidad:** Funciona en modelos de lenguaje, modelos de difusión de imágenes y generación de video.
- **Fortaleza:** 2.5–5× de aceleración comparado con métodos densos o dispersos anteriores; compatible con cuantización (integración SageAttention).
- **Limitación:** La aceleración depende de la dispersidad inherente; la granularidad a nivel de bloque puede pasar por alto patrones más finos; se requiere ajuste de umbral por familia de modelos.

C.9. Resumen Comparativo

Método	Complejidad Entrenable		Unidad		Año	Contribución Principal
Sparse Transformer	$O(n\sqrt{nd})$	Sí	Patrón de token		2019	Máscaras fijas y estrideas factorizadas
Longformer	$O(nwd)$	Sí	Local + global		2020	Escalado lineal con dilatación
BigBird	$O(n)$	Sí	Grafo disperso		2020	Prueba de completitud teórica
SparseK	$O(nd)$	Sí	Tokens top- k		2024	Selección diferenciable
NSA	$O((t/d + nl' + w)d)$	Sí	Multi-rama		2025	Diseño de 3 ramas alineado con hardware
FASA	$O(tN_{\text{tip}} + N_{\text{fac}}d)$	No	Fragmentos de frecuencia		2026	Conocimiento de frecuencia RoPE
SpargeAttn	$O(n^2s \cdot d)$	No	Filtro de bloques		2025	Filtrado universal de dos etapas

C.10. Espacio de Diseño y Criterios de Selección

Los siete métodos de atención dispersa ocupan posiciones complementarias en un espacio de diseño multidimensional:

- **Patrones geométricos/fijos** (Sparse Transformer, Longformer): Simples de implementar y analizar, pero los patrones no se adaptan al contenido.
- **Selección aprendida** (SparseK, NSA): Permiten adaptación mediante redes de selección entrenables, a costa de mayor complejidad de implementación.
- **Aceleración sin entrenamiento** (FASA, SpargeAttn): Permiten aceleración directa de modelos existentes sin reentrenamiento, adecuados para sistemas desplegados.
- **Garantías teóricas** (BigBird): Proporcionan demostraciones formales de completitud expresiva, importantes para comprender los márgenes de seguridad.

Orientación de selección:

1. **Entrenamiento de nuevos modelos** donde los recursos lo permitan: NSA o SparseK para máxima aceleración de inferencia; BigBird para garantía teórica.
2. **Comprensión de documentos de contexto largo**: Longformer o FASA por simplicidad práctica; NSA para escala extrema.
3. **Aceleración de modelos existentes**: FASA para LLMs basados en RoPE; SpargeAttn para cualquier arquitectura.
4. **Entornos con recursos limitados**: Sparse Transformer por simplicidad y eficiencia probada a escala moderada.

D. Annex D: Gated Attention Families—Complete Literature Analysis

D.1. Resumen Ejecutivo: Compuertas para el Control de Memoria

El campo emergente de los *mecanismos de atención con compuerta* aborda un desafío fundamental en el modelado de secuencias: ¿cómo debe retenerse, olvidarse y actualizarse la información a medida que el modelo procesa nuevos tokens? Los estados recurrentes aditivos

tradicionales acumulan toda la información sin borrado, lo que lleva a la saturación de memoria y a la incapacidad de adaptarse a contextos cambiantes. La atención softmax proporciona expresividad completa pero usa un caché KV de $\mathcal{O}(Ld)$ durante la inferencia, limitando el tamaño de la ventana de contexto. Los mecanismos con compuerta proporcionan un punto intermedio: control selectivo sobre la supervivencia de la información.

El principio unificador entre las arquitecturas con compuerta es que las compuertas controlan *qué información sobrevive*. Las compuertas operan en tres niveles distintos:

1. **Decaimiento del estado recurrente** (GLA, HGRN2): Compuertas multiplicativas sobre la matriz de memoria S_t controlan cuánto del estado anterior persiste en el siguiente paso.
2. **Intensidad de escritura en actualizaciones recurrentes** (DeltaNet, Gated DeltaNet): Las compuertas controlan la magnitud de la nueva información escrita en la memoria.
3. **Sesgo de logits softmax** (FoX): Las compuertas inyectan sesgo de actualidad a nivel de token en el cálculo de logits de atención.
4. **Dispersidad post-atención** (Gated Softmax): Las compuertas escalan selectivamente los canales de salida de SDPA.

Este apéndice sintetiza siete arquitecturas principales de atención con compuerta publicadas entre 2023 y 2025, analizando sus fundamentos matemáticos, características de eficiencia en hardware y compensaciones empíricas.

D.2. 1. Atención Lineal con Compuerta (GLA)

D.2.1. Fundamento Matemático

La Atención Lineal con Compuerta [45] aumenta la atención lineal estándar (que usa un estado recurrente con valor de matriz) con decaimiento multiplicativo dependiente de los datos. La atención lineal estándar reformula el mecanismo softmax tradicional como una recurrencia sobre estados ocultos:

$$\mathbf{S}_t = \mathbf{S}_{t-1} + \mathbf{v}_t \mathbf{k}_t^\top, \quad \mathbf{o}_t = \mathbf{S}_t \mathbf{q}_t$$

donde el estado $\mathbf{S}_t \in \mathbb{R}^{d_v \times d_k}$ acumula productos externos. Esta formulación puramente aditiva sufre de “sobrecarga de memoria”: la información se acumula monótonamente y no puede borrarse, degradando el rendimiento en tareas que requieren selección de contexto.

GLA introduce una **matriz de compuerta diagonal** $\mathbf{G}_t \in [0, 1]^{d_k \times d_k}$ dependiente de los datos:

$$\mathbf{S}_t = \mathbf{G}_t \odot \mathbf{S}_{t-1} + \mathbf{v}_t \mathbf{k}_t^\top, \quad \mathbf{o}_t = \mathbf{S}_t \mathbf{q}_t$$

La compuerta $\mathbf{G}_t$ se parametriza con una estructura de producto externo:

$$\mathbf{G}_t = \mathbf{1} \alpha_t^\top, \quad \alpha_t = \text{logsigmoid}(\mathbf{W}_{gk} \mathbf{x}_t) / c$$

donde $\mathbf{W}_{gk}$ es una proyección de rango bajo ($\mathbb{R}^{d \rightarrow d_k}$) y $c \approx 16$ es un normalizador. La activación logsigmoid mapea $\alpha_t \in \mathbb{R}$ a aproximadamente $[-1/2, 0]$, y la división por c contrae esto aún más para que esté cerca de la identidad en la inicialización. La compuerta resultante $G_t = 1 + \alpha_t \approx 1$ cerca de la inicialización, luego aprende a decaer ($\alpha_t \rightarrow -\infty$) información histórica.

Algorithm 32 Atención Lineal con Compuerta (Forma Recurrente)

Require: Entrada $\mathbf{x}_t$; estado anterior $\mathbf{S}_{t-1}$

Require: Proyecciones: W_q, W_k, W_v (estándar); W_{gk} (proyección de compuerta)

Ensure: Salida $\mathbf{o}_t$ y estado actualizado $\mathbf{S}_t$

- 1: $\mathbf{q}_t \leftarrow W_q \mathbf{x}_t$
 - 2: $\mathbf{k}_t \leftarrow W_k \mathbf{x}_t$
 - 3: $\mathbf{v}_t \leftarrow W_v \mathbf{x}_t$
 - 4: Calcular compuerta: $\alpha_t \leftarrow \text{logsigmoid}(W_{gk} \mathbf{x}_t)/16$ $\triangleright$ Decaimiento por dimensión de clave
 - 5: Aplicar compuerta de producto externo: $\mathbf{G}_t \leftarrow \mathbf{1} \alpha_t^\top$ $\triangleright$ producto externo diagonal
 - 6: Decaer estado anterior: $\mathbf{S}_t^{\text{decaimiento}} \leftarrow \mathbf{G}_t \odot \mathbf{S}_{t-1}$ $\triangleright$ producto elemento a elemento
 - 7: Agregar nueva asociación: $\mathbf{S}_t \leftarrow \mathbf{S}_t^{\text{decaimiento}} + \mathbf{v}_t \mathbf{k}_t^\top$
 - 8: Recuperar mediante consulta: $\mathbf{o}_t^{\text{crudo}} \leftarrow \mathbf{S}_t \mathbf{q}_t$
 - 9: Aplicar compuerta de salida: $\mathbf{o}_t \leftarrow \text{GroupNorm}(\mathbf{o}_t^{\text{crudo}}) \otimes \text{SiLU}(W_g \mathbf{x}_t)$ **return** $\mathbf{o}_t, \mathbf{S}_t$
-

D.2.2. Entrenamiento Eficiente en Hardware

Para el entrenamiento paralelo, GLA usa un algoritmo por fragmentos que agrupa tokens en fragmentos C y calcula transiciones recurrentes en paralelo:

$$\mathbf{o}_{[t]} = \overleftarrow{\mathbf{Q}}_{[t]} \mathbf{S}_{[t]}^\top + \left(\mathbf{Q}_{[t]} \mathbf{K}_{[t]}^\top \odot \mathbf{\Gamma}_{[t]} \right) \mathbf{V}_{[t]}$$

donde $\overleftarrow{\mathbf{Q}}_{[t]}$ atiende al estado en el límite del fragmento (retrospectiva), y $\mathbf{\Gamma}_{[t]}$ es la máscara causal con escalado consciente del decaimiento:

$$\mathbf{\Gamma}_{[t],ij} = \begin{cases} \prod_{l=j}^{i-1} \alpha_l & \text{si } i > j \text{ (retrospectiva)} \\ \prod_{l=1}^{i-j} \alpha_l & \text{si } i \leq j \text{ (en-fragmento)} \end{cases}$$

Este diseño maximiza las operaciones matmul susceptibles de aceleración mediante núcleos tensoriales, logrando una complejidad total subcuadrática $\mathcal{O}(nLd^2/C)$ donde L es el número de cabezas de atención y C es el tamaño del fragmento.

D.2.3. Fortalezas y Limitaciones

Aspecto	Fortalezas	Limitaciones
Eficiencia computacional	Entrenamiento subcuadrático mediante paralelismo por fragmentos. Inferencia con memoria constante $\mathcal{O}(d^2)$.	Requiere kernels CUDA/Triton personalizados; no disponible en todos los frameworks.
Generalización de contexto	Extiende entrenamiento de 2K tokens a más de 20K tokens con degradación de perplejidad insignificante.	Aún rinde por debajo de softmax en benchmarks con mucha recuperación (aguja en el pajar).
Selectividad	Decaimiento dependiente de los datos por dimensión de clave.	La estructura de compuerta limita la selectividad por par clave-valor; no hay control directo sobre qué asociaciones específicas borrar.

D.3. 2. DeltaNet: Atención Lineal con Corrección de Errores

D.3.1. Fundamento Matemático

DeltaNet [46] aplica una **regla de aprendizaje delta** clásica a las actualizaciones de estado de la atención lineal. En lugar de acumulación aditiva, DeltaNet calcula la diferencia entre el valor predicho (recuperado de la memoria) y el valor objetivo, luego realiza una actualización de corrección de errores:

$$\mathbf{S}_t = \mathbf{S}_{t-1}(\mathbf{I} - \beta_t \mathbf{k}_t \mathbf{k}_t^\top) + \beta_t \mathbf{v}_t \mathbf{k}_t^\top$$

donde $\beta_t \in (0, 1)$ es un escalar de **intensidad de escritura** dependiente de los datos. Esta actualización puede descomponerse como una operación de borrado-escritura:

$$\mathbf{v}_t^{\text{anterior}} = \mathbf{S}_{t-1} \mathbf{k}_t \quad (\text{predicción actual}), \quad \mathbf{v}_t^{\text{actualizado}} = \beta_t \mathbf{v}_t + (1 - \beta_t) \mathbf{v}_t^{\text{anterior}} \quad (\text{mezcla anterior/nuevo})$$

de modo que:

$$\mathbf{S}_t = \mathbf{S}_{t-1} - \mathbf{v}_t^{\text{anterior}} \mathbf{k}_t^\top + \mathbf{v}_t^{\text{actualizado}} \mathbf{k}_t^\top$$

La idea clave es que esta regla de actualización minimiza una pérdida de ECM en línea en cada paso temporal:

$$\mathcal{L}_t(\mathbf{S}) = \frac{1}{2} \|\mathbf{S} \mathbf{k}_t - \mathbf{v}_t\|^2 \Rightarrow \nabla_{\mathbf{S}} \mathcal{L}_t = (\mathbf{S} \mathbf{k}_t - \mathbf{v}_t) \mathbf{k}_t^\top$$

Un paso de SGD con tasa de aprendizaje β_t produce la actualización de la regla delta. Esta conexión con el entrenamiento en tiempo de prueba (TTT) proporciona fundamentación teórica: DeltaNet optimiza directamente la predicción de valores en tiempo de inferencia.

D.3.2. Entrenamiento Paralelo Eficiente

La matriz de transición de DeltaNet $\mathbf{I} - \beta_t \mathbf{k}_t \mathbf{k}_t^\top$ es una matriz de Householder generalizada (actualización de rango 1 a la identidad). Para el entrenamiento por fragmentos, DeltaNet usa la **representación WY**, que representa el producto de m de estas actualizaciones de rango 1 de forma compacta:

$$\prod_{i=1}^m (\mathbf{I} - \beta_i \mathbf{k}_i \mathbf{k}_i^\top) = \mathbf{I} - \mathbf{W} \mathbf{Y}^\top$$

donde $\mathbf{W}, \mathbf{Y} \in \mathbb{R}^{d \times m}$ se construyen recursivamente. Esta factorización permite paralelismo rico en matmul sin materializar el producto completo, manteniendo el entrenamiento por fragmentos eficiente.

Algorithm 33 Actualización de Estado de DeltaNet con Representación WY

Require: Fragmento de entrada $\mathbf{X}_{[t]}$; estado anterior $\mathbf{S}_{t-1}$

Require: Consultas normalizadas L2 $\hat{\mathbf{Q}}_t$, claves $\hat{\mathbf{K}}_t$, valores $\mathbf{V}_t$

Ensure: Salida $\mathbf{O}_{[t]}$ y estado actualizado $\mathbf{S}_t$

- 1: **Fase por fragmentos:**
 - 2: **for** posición $i = 1$ to tamaño_fragmento **do**
 - 3: Normalizar: $\hat{\mathbf{k}}_i \leftarrow \hat{\mathbf{K}}_t[i] / \|\hat{\mathbf{K}}_t[i]\|$, $\hat{\mathbf{q}}_i \leftarrow \hat{\mathbf{Q}}_t[i] / \|\hat{\mathbf{Q}}_t[i]\|$
 - 4: Calcular intensidad de escritura: $\beta_i \leftarrow \text{sigmoid}(W_\beta \mathbf{x}_i)$
 - 5: Predicción: $\hat{\mathbf{v}}^{\text{pred}} \leftarrow \mathbf{S}_{i-1}^{\text{en-fragmento}} \hat{\mathbf{k}}_i$
 - 6: Mezcla consciente del error: $\mathbf{v}_i^{\text{actualización}} \leftarrow \beta_i (\mathbf{v}_i - \hat{\mathbf{v}}^{\text{pred}})$
 - 7: Borrar-escribir: $\mathbf{S}_i^{\text{en-fragmento}} \leftarrow \mathbf{S}_{i-1}^{\text{en-fragmento}} - \hat{\mathbf{v}}^{\text{pred}} \hat{\mathbf{k}}_i^\top + (\beta_i \mathbf{v}_i + (1 - \beta_i) \hat{\mathbf{v}}^{\text{pred}}) \hat{\mathbf{k}}_i^\top$
 - 8: Salida: $\mathbf{o}_i \leftarrow \mathbf{S}_i^{\text{en-fragmento}} \hat{\mathbf{q}}_i$
 - 9: **end for**
 - 10: **Fase entre fragmentos:** Usar representación WY de los productos de transición **return** $\mathbf{O}_{[t]}, \mathbf{S}_t$
-

D.3.3. Fortalezas y Limitaciones

Aspecto	Fortalezas	Limitaciones
Recuperación asociativa	Recuperación perfecta en el benchmark MQAR (Recuperación Asociativa Multi-Consulta); la semántica de corrección de errores se ajusta naturalmente a patrones de recuperación.	Sin olvido global, la memoria aún se satura en longitudes de secuencia extremas; requiere mecanismos de reinicio periódico para contexto ilimitado.
Teoría	Fundamentada en optimización de ECM en línea; conexión clara con el paradigma de entrenamiento en tiempo de prueba.	Requiere claves normalizadas L2 para estabilidad numérica; la normalización de doble ancho añade sobrecarga.
Rendimiento	La representación WY permite entrenamiento eficiente por fragmentos.	Ligeramente más lento que Mamba2 por token debido a matrices de transición más ricas (rango 1 en lugar de diagonales).

D.4. 3. Gated DeltaNet: Síntesis de Compuerta y Corrección de Errores

D.4.1. Fundamento Matemático

Gated DeltaNet [47] sintetiza las fortalezas de GLA y DeltaNet combinando una **compuerta de decaimiento** α_t (de GLA) con la **compuerta de intensidad de escritura** β_t (de DeltaNet):

$$\mathbf{S}_t = \mathbf{S}_{t-1} \left(\alpha_t (\mathbf{I} - \beta_t \mathbf{k}_t \mathbf{k}_t^\top) \right) + \beta_t \mathbf{v}_t \mathbf{k}_t^\top$$

Reescribiendo para mayor claridad:

$$\mathbf{S}_t = \alpha_t \mathbf{S}_{t-1} (\mathbf{I} - \beta_t \mathbf{k}_t \mathbf{k}_t^\top) + \beta_t \mathbf{v}_t \mathbf{k}_t^\top$$

Las dos compuertas son complementarias:

- $\alpha_t \rightarrow 0$ borra rápidamente el estado histórico: $\mathbf{S}_t \approx \beta_t \mathbf{v}_t \mathbf{k}_t^\top$ (reinicio de contexto).
- $\alpha_t \rightarrow 1$ recupera el comportamiento puro de regla delta: $\mathbf{S}_t \approx \mathbf{S}_{t-1} (\mathbf{I} - \beta_t \mathbf{k}_t \mathbf{k}_t^\top) + \beta_t \mathbf{v}_t \mathbf{k}_t^\top$ (actualizaciones dirigidas).
- $\beta_t \rightarrow 0$ omite la escritura pero decae: $\mathbf{S}_t \approx \alpha_t \mathbf{S}_{t-1}$ (olvido puro).
- $\beta_t \rightarrow 1$ reemplaza la memoria agresivamente: $\mathbf{S}_t \approx \alpha_t \mathbf{S}_{t-1} + \mathbf{v}_t \mathbf{k}_t^\top$ (reemplazo).

Desde una perspectiva de aprendizaje en línea, Gated DeltaNet corresponde a:

$$\min_{\mathbf{S}_t} \|\mathbf{S}_t - \alpha_t \mathbf{S}_{t-1}\|_F^2 - 2 \langle \mathbf{S}_t \mathbf{k}_t, \beta_t (\mathbf{v}_t - \alpha_t \mathbf{S}_{t-1} \mathbf{k}_t) \rangle$$

que introduce un decaimiento de pesos adaptativo α_t en una actualización tipo SGD—análogo al decaimiento de pesos desacoplado en la optimización de aprendizaje profundo.

Algorithm 34 Paso Adelante Paralelo por Fragmentos de Gated DeltaNet

Require: Fragmento de entrada $\mathbf{X}_{[i]}$; estado anterior $\mathbf{S}_{i-1}$; tamaño de fragmento C

Require: Proyecciones: $W_q, W_k, W_v, W_\alpha, W_\beta$

Ensure: Salida $\mathbf{O}_{[i]}$ y estado $\mathbf{S}_i$

1: **Cálculo en-fragmento:**

2: **for** $j = 1$ to C **do**

3: $\mathbf{q}_j \leftarrow W_q \mathbf{x}_j, \mathbf{k}_j \leftarrow W_k \mathbf{x}_j, \mathbf{v}_j \leftarrow W_v \mathbf{x}_j$

4: $\alpha_j \leftarrow \text{sigmoid}(W_\alpha \mathbf{x}_j), \beta_j \leftarrow \text{sigmoid}(W_\beta \mathbf{x}_j)$

5: Aplicar compuerta combinada: $\mathbf{S}_j \leftarrow \alpha_j \mathbf{S}_{j-1} (\mathbf{I} - \beta_j \mathbf{k}_j \mathbf{k}_j^\top) + \beta_j \mathbf{v}_j \mathbf{k}_j^\top$

6: $\mathbf{o}_j \leftarrow \mathbf{S}_j \mathbf{q}_j$

7: **end for**

8: **Recurrencia entre fragmentos:**

9: $\mathbf{S}_i \leftarrow \mathbf{S}_C$ del en-fragmento; propagar entre fragmentos mediante máscaras $\prod \alpha$ acumulativas

10: **Compuerta de salida:** $\mathbf{O}_{[i]} \leftarrow \text{GroupNorm}(\mathbf{O}^{\text{crudo}}) \otimes \text{SiLU}(W_g \mathbf{X}_{[i]})$ **return** $\mathbf{O}_{[i]}, \mathbf{S}_i$

D.4.2. Rendimiento Empírico y Compensaciones

Gated DeltaNet se ha integrado en los modelos de producción Qwen3-Next y Qwen3.5 de Alibaba. Empíricamente:

Tarea	Gated DeltaNet	Mamba2	DeltaNet	GLA
Modelado de Lenguaje	1,0 $\times$	1,08 $\times$	1,05 $\times$	1,12 $\times$
Recuperación Asociativa	1,0 $\times$	— — (<i>sinperfecto</i>)	1,0 $\times$	0,75
Extrapolación de Longitud	1,0 $\times$	0,98 $\times$	0,96 $\times$	0,94 $\times$
Rendimiento (tokens/seg)	0,85 $\times$	1,0 $\times$	0,82 $\times$	0,88 $\times$

Gated DeltaNet logra el mejor equilibrio entre diversas tareas a costa de un rendimiento ligeramente reducido debido a matrices de transición más ricas.

D.5. 4. HGRN2: Compuerta Jerárquica con Expansión de Producto Externo

D.5.1. Fundamento Matemático

HGRN2 [29] usa una expansión de estado basada en producto externo con **compuertas de olvido acotadas inferiormente de forma jerárquica**. La actualización de estado es:

$$\mathbf{S}_t = \text{diag}(\mathbf{g}_t) \cdot \mathbf{S}_{t-1} + \mathbf{v}_t \mathbf{k}_t^\top, \quad \mathbf{o}_t = \mathbf{S}_t \mathbf{q}_t$$

donde $\mathbf{g}_t \in [b_\ell, 1]^d$ es una compuerta de olvido con cota inferior para la capa ℓ . Las cotas satisfacen:

$$0 \leq b_{\ell_{\text{poco profunda}}} < b_{\ell_{\text{media}}} < b_{\ell_{\text{profunda}}} \leq 1$$

es decir, las cotas aumentan (se vuelven menos restrictivas) en las capas más profundas. La compuerta en sí se calcula como:

$$\mathbf{g}_t = b_\ell + (1 - b_\ell) \cdot \sigma(W_g \mathbf{x}_t)$$

donde σ es sigmoide. Cuando W_g produce logits débilmente negativos, $\mathbf{g}_t \approx b_\ell$ (retención forzada en capas superficiales). Cuando las salidas son fuertemente positivas, $\mathbf{g}_t \approx 1$ (refresco de memoria en cualquier capa).

La estructura jerárquica anima a diferentes capas a especializarse en diferentes escalas temporales: las capas superficiales modelan dependencias locales (alta retención debido a la cota b_ℓ), mientras que las capas profundas capturan estructura de largo alcance (compuerta flexible con cota superior alta).

Algorithm 35 Paso Adelante de HGRN2 con Cotas Jerárquicas

Require: Entrada $\mathbf{x}_t$; estado anterior $\mathbf{S}_{t-1}$; índice de capa $\ell \in [0, L-1]$

Require: Cotas no decrecientes: $0 \leq b_0 < b_1 < \dots < b_{L-1} \leq 1$

Ensure: Salida $\mathbf{o}_t$ y estado $\mathbf{S}_t$

- 1: Cota de retención específica de capa: $b \leftarrow b_\ell$
 - 2: Proyectar: $\mathbf{q}_t \leftarrow W_q \mathbf{x}_t$, $\mathbf{k}_t \leftarrow W_k \mathbf{x}_t$, $\mathbf{v}_t \leftarrow W_v \mathbf{x}_t$
 - 3: Calcular compuerta de olvido con cota: $\mathbf{g}_t \leftarrow b + (1-b) \cdot \sigma(W_g \mathbf{x}_t)$ $\triangleright$ Confinado a $[b, 1]$
 - 4: Multiplicación diagonal (vectorizada): $\mathbf{S}_t \leftarrow \mathbf{S}_{t-1} \circ \text{diag}(\mathbf{g}_t) + \mathbf{v}_t \mathbf{k}_t^\top$
 - 5: Recuperar: $\mathbf{o}_t \leftarrow \mathbf{S}_t \mathbf{q}_t$
 - 6: Normalización opcional: $\mathbf{o}_t \leftarrow \text{RMSNorm}(\mathbf{o}_t)$ **return** $\mathbf{o}_t$, $\mathbf{S}_t$
-

D.5.2. Escalado y Resultados Empíricos

A escala de 3B en 100B tokens, HGRN2 supera ligeramente a Mamba2 y a los Transformers con arquitectura LLaMA en modelado de lenguaje. La compuerta jerárquica proporciona modelado temporal multi-escala sin variantes arquitectónicas explícitas por capa. Sin embargo, la compuerta diagonal con valor vectorial (en oposición a la selección elemento a elemento) proporciona menos flexibilidad por elemento que las variantes de regla delta.

D.6. 5. Forgetting Transformer (FoX): Compuerta en el Espacio de Logits Softmax

D.6.1. Fundamento Matemático

Forgetting Transformer (FoX), propuesto por Lin et al. [20], incrusta una compuerta de olvido directamente en los logits de atención softmax. En lugar de reemplazar softmax con recurrencia lineal, FoX preserva la expresividad completa de softmax mientras añade control de actualidad mediante un sesgo de logit dependiente de los datos.

Para cada posición de token t y todas las claves $j \leq t$, calcular un escalar de compuerta de olvido:

$$f_t = \sigma(w_f^\top \mathbf{x}_t + b_f)$$

El sesgo de logit en la posición (i, j) es el producto acumulativo log-olvido:

$$d_{ij} = \sum_{l=j+1}^i \log f_l = \log \prod_{l=j+1}^i f_l$$

La salida de atención completa se convierte en:

$$\mathbf{O} = \text{softmax} \left(\frac{\mathbf{QK}^\top}{\sqrt{d_k}} + \mathbf{D} \right) \mathbf{V}$$

donde $\mathbf{D}_{ij} = d_{ij}$. Equivalentemente:

$$\text{Atencion}(i, j) = \frac{\exp(\mathbf{q}_i^\top \mathbf{k}_j / \sqrt{d_k} + d_{ij})}{\sum_{j'=1}^i \exp(\mathbf{q}_i^\top \mathbf{k}_{j'} / \sqrt{d_k} + d_{ij'})}$$

Esta ponderación reduce el peso de los tokens más antiguos multiplicativamente: un token de hace 10 pasos se escala por $\prod_{l=j+1}^i f_l$, que es típicamente mucho menor que 1 si $f_t < 1$ en promedio.

El mecanismo es matemáticamente equivalente a una **variante aprendible dependiente de los datos de ALiBi (Atención con Sesgos Lineales)**, donde trabajos anteriores usaban $d_{ij} = -\infty \cdot |i - j|$ fijo o $d_{ij} = -(i - j)$.

Algorithm 36 FoX: Atención con Olvido y Sesgo de Actualidad

Require: Consultas $\mathbf{Q}$, claves $\mathbf{K}$, valores $\mathbf{V}$; entrada $\mathbf{X}$

Require: Parámetros de compuerta de olvido: $w_f \in \mathbb{R}^d$, $b_f \in \mathbb{R}$

Ensure: Salida de atención $\mathbf{O}$

```
1: Calcular compuertas de olvido:
2: for  $t = 1$  to  $T$  do
3:    $f_t \leftarrow \sigma(w_f^\top \mathbf{x}_t + b_f)$  ▷ Probabilidad de olvido por token
4: end for
5: Calcular sesgos acumulativos de log-olvido:
6: for  $i = 1$  to  $T$  do
7:   for  $j = 1$  to  $i$  do
8:      $d_{ij} \leftarrow \sum_{l=j+1}^i \log f_l$  ▷ Producto acumulativo en espacio logarítmico
9:   end for
10: end for
11: SDPA estándar con sesgo:
12: Calcular puntajes:  $\mathbf{S} \leftarrow \mathbf{QK}^\top / \sqrt{d_k}$ 
13: Agregar sesgo:  $\mathbf{S} \leftarrow \mathbf{S} + \mathbf{D}$ 
14: Aplicar softmax:  $\mathbf{A} \leftarrow \text{softmax}(\mathbf{S}, \text{dim} = -1)$ 
15: Ponderar valores:  $\mathbf{O} \leftarrow \mathbf{AV}$  return  $\mathbf{O}$ 
```

D.6.2. Integración con FlashAttention

La ventaja clave de FoX es la compatibilidad con FlashAttention y las implementaciones optimizadas existentes. Los sesgos de olvido se añaden en el cálculo de logits softmax, que ya es una operación central en el algoritmo por bloques de FlashAttention. La sobrecarga es:

$$\text{Sobrecarga} \approx 0,5 - 2\% \text{ del tiempo de pared}$$

ya que la adición de sesgo se amortiza entre las operaciones matmul.

D.6.3. Fortalezas y Limitaciones

FoX logra resultados de última generación en extrapolación de longitud, recuperación casi perfecta de aguja en el pajar y comprensión superior de contexto largo en comparación con los Transformers. Sin embargo, sigue siendo $\mathcal{O}(L^2d)$ en entrenamiento y $\mathcal{O}(Ld)$ por paso en inferencia con caché KV, limitando las longitudes de secuencia extremas.

D.7. 6. Atención con Compuerta (Post-SDPA Sigmoide)

D.7.1. Fundamento Matemático

El Mejor Artículo de NeurIPS 2025 del equipo de Qwen propone aplicar una compuerta sigmoide **después** de la atención de producto punto escalado (SDPA):

$$\mathbf{Y} = \text{SDPA}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{QK}^\top}{\sqrt{d_k}}\right) \mathbf{V}$$

$$\mathbf{Y}' = \mathbf{Y} \odot \sigma(\mathbf{XW}_\theta)$$

donde el puntaje de compuerta $\sigma(\mathbf{XW}_\theta)$ puede tener forma $\mathbb{R}^{n \times q \times d_k}$ (compuerta elemento a elemento por cabeza de consulta) o $\mathbb{R}^{n \times q}$ (compuerta fija por cabeza). En más de 30 variantes y modelos MoE de 15B + densos de 1.7B entrenados en 3.5T tokens, el equipo encontró:

- La compuerta por cabeza añade solo $\sim 1,6M$ parámetros a un modelo de 15B (sobrecarga insignificante).

- Las compuertas efectivas son **dispersas**: activación media $\approx 0,116$ (es decir, el 88 % son cero o casi cero).
- Las compuertas actúan como filtros dependientes de la consulta, suprimiendo canales de bajo valor mientras preservan señales de alto valor.
- Las compuertas eliminan demostrablemente el fenómeno de **sumidero de atención**, donde el patrón de atención se vuelve dominado por un único token temprano.

Algorithm 37 Atención Softmax con Compuerta (Post-SDPA)

Require: Consultas $\mathbf{Q}$, Claves $\mathbf{K}$, Valores $\mathbf{V}$; entrada $\mathbf{X}$

Require: Matriz de proyección de compuerta $W_g \in \mathbb{R}^{d \rightarrow d_{\text{compuerta}}}$ donde $d_{\text{compuerta}} \in \{1, d_k\}$

Ensure: Salida con compuerta $\mathbf{Y}'$

```

1: SDPA estándar:
2: Calcular atención:  $\mathbf{A} \leftarrow \text{softmax}(\mathbf{QK}^\top / \sqrt{d_k})$ 
3: Ponderar valores:  $\mathbf{Y} \leftarrow \mathbf{AV}$ 
4: Calcular compuertas:
5: Proyectar entrada:  $\mathbf{G} \leftarrow \mathbf{XW}_g$  ▷ forma:  $[n, q, d_{\text{compuerta}}]$ 
6: Aplicar sigmoide:  $\mathbf{G} \leftarrow \sigma(\mathbf{G})$  ▷ Compuerta elemento a elemento
7: Aplicar compuerta:
8: if  $d_{\text{compuerta}} = 1$  then
9:   Transmitir y escalar:  $\mathbf{Y}' \leftarrow \mathbf{Y} \cdot \mathbf{G}$  ▷ Escalado por cabeza
10: else ▷  $d_{\text{compuerta}} = d_k$ 
11:   Modulación elemento a elemento:  $\mathbf{Y}' \leftarrow \mathbf{Y} \odot \mathbf{G}$  ▷ Compuerta por dimensión
12: end if return  $\mathbf{Y}'$ 

```

D.7.2. Hallazgos Clave

- **Supresión del sumidero de atención:** La compuerta rompe el ciclo de retroalimentación donde softmax se concentra en el primer token, permitiendo una atención más equilibrada.
- **Estabilidad de entrenamiento:** Los modelos con compuerta toleran tasas de aprendizaje más grandes (+30 % mayor η_{max} estable).
- **Sobrecarga mínima:** El impacto en latencia es solo del 1,6 % en GPUs H100; el rendimiento cae como máximo 2 – 3 %.
- **Mejora en la ley de escalado:** Mejora la pérdida en todas las escalas de modelo, desde 800M hasta 110B parámetros de forma consistente.

D.8. Análisis Comparativo: Taxonomía de Mecanismos con Compuerta

Arquitectura	Año de Publicación	Representación de Estado	Tipo de Compuerta	Complejidad de Entrenamiento	Complejidad de Inferencia	Ref
GLA	2023	Matriz (recurrente)	Decaimiento diagonal	$\mathcal{O}(Ld^2)$	$\mathcal{O}(d^2)$ memoria	[45]
DeltaNet	2024	Matriz (recurrente)	Escritura escalar (β)	$\mathcal{O}(Ld^2)$	$\mathcal{O}(d^2)$ memoria	[46]
Gated DeltaNet	2024	Matriz (recurrente)	Decaimiento + escritura	$\mathcal{O}(Ld^2)$	$\mathcal{O}(d^2)$ memoria	[47]
RetNet	2023	Matriz (recurrente)	Exponencial fija	$\mathcal{O}(Ld^2)$	$\mathcal{O}(d)$ por paso	[37]

Arquitectura	Año de Publicación	Representación de Estado	Tipo de Compuerta	Complejidad de Entrenamiento	Complejidad de Inferencia	Ref
HGRN2	2024	Matriz (recurrente)	Diagonal (acotada)	$\mathcal{O}(Ld^2)$	$\mathcal{O}(d^2)$ memoria	[29]
FoX	2025	Atención completa	Sesgo en espacio de logits	$\mathcal{O}(L^2d)$	$\mathcal{O}(L)$ caché KV	[20]
Gated Soft-max	2025	Atención completa	Sigmoide post-SDPA	$\mathcal{O}(L^2d)$	$\mathcal{O}(L)$ caché KV	[30]

Arquitectura	Entrenable	Recuperación en Contexto	Recuperación Asociativa	Extrapolación de Longitud	Ventaja Clave
GLA	Sí	Moderada	Débil	Buena (20K)	Eficiencia en hardware; paralelismo por fragmentos
DeltaNet	Sí	Fuerte	Perfecta (MQAR)	Buena	Semántica de corrección de errores; recuperación sólida
Gated DeltaNet	Sí	Muy fuerte	Perfecta	Excelente	Equilibrada: olvido + actualizaciones dirigidas
RetNet	Sí	Débil	Débil	Buena	Simplicidad; no necesita caché KV
HGRN2	Sí	Moderada	Débil	Moderada	Modelado temporal multi-escala
FoX	Sí	Muy fuerte	Muy fuerte (casi perfecta)	Excelente	Preserva softmax; sobrecarga mínima
Gated Soft-max	Sí	Fuerte	Buena	Buena	Simplicidad; no reemplaza SDPA

Arquitectura	Rendimiento Típico	Memoria por Inferencia	Caso de Uso Principal
GLA	Alto (CUDA por fragmentos)	$\mathcal{O}(d^2)$ constante	Contexto largo con restricciones de memoria
DeltaNet	Medio-Alto	$\mathcal{O}(d^2)$ constante	Recuperación y razonamiento asociativo
Gated DeltaNet	Medio	$\mathcal{O}(d^2)$ constante	Producción: Qwen3-Next, rendimiento equilibrado
RetNet	Alto	$\mathcal{O}(d)$ por paso	Inferencia extremadamente rápida; modelos simples
HGRN2	Medio-Alto	$\mathcal{O}(d^2)$ constante	Patrones temporales multi-escala

Arquitectura		Rendimiento Típico	Memoria por In- ferencia	Caso de Uso Princi- pal
FoX		Moderado (cua- drático)	$O(Ld)$ caché KV	Extrapolación de longi- tud; generación de con- texto largo
Gated max	Soft-	Alto (sobrecarga mínima)	$O(Ld)$ caché KV	Mejora directa para mo- delos existentes

D.9. Principio Unificado de Compuerta

Las siete arquitecturas comparten un principio común: **la compuerta es un mecanismo para controlar el flujo de información y la retención selectiva de memoria**. Las decisiones de diseño específicas divergen a lo largo de varias dimensiones:

1. **Ubicación de la compuerta:** Actualizaciones de estado recurrente (GLA, DeltaNet, Gated DeltaNet, HGRN2) versus espacio de logits/salida softmax (FoX, Gated Softmax, RetNet).
2. **Granularidad de control:** Por dimensión (GLA, HGRN2 diagonal), por clave (DeltaNet), por token (FoX) o por cabeza (Gated Softmax).
3. **Dependencia de datos:** Completamente dependiente de datos (GLA, DeltaNet, Gated DeltaNet, FoX, Gated Softmax) versus esquemas fijos (RetNet con decaimiento exponencial).
4. **Compensación de expresividad:** Eficiencia recurrente lineal (GLA, DeltaNet, Gated DeltaNet, HGRN2, RetNet) versus expresividad completa de softmax (FoX, Gated Softmax).

D.10. Implementación y Orientación Práctica

D.10.1. Cuándo Usar Cada Arquitectura

1. **GLA:** Inferencia rápida con contextos moderados a largos (2K–20K tokens), cuando los kernels CUDA personalizados son aceptables y el rendimiento de recuperación no es crítico.
2. **DeltaNet:** Fuerte recuperación asociativa y tareas de aprendizaje en contexto; bueno para tareas sintéticas (MQAR) y pruebas de asociación clave-valor.
3. **Gated DeltaNet:** Modelos de producción que requieren el mejor equilibrio de recuperación, olvido y rendimiento (probado en Qwen3-Next; ICLR 2025).
4. **RetNet:** Inferencia extremadamente eficiente sin almacenamiento en caché KV; adecuado para despliegues en dispositivos móviles/de borde o modelos de juguete.
5. **HGRN2:** Jerarquías temporales multi-escala; cuando los límites de olvido específicos por capa son deseables.
6. **FoX:** Extrapolación de longitud y comprensión de contexto largo; cuando el entrenamiento cuadrático es aceptable y no se desea reemplazar softmax.
7. **Gated Softmax:** Mejora rápida para modelos softmax existentes con cambios mínimos de código; mejor para profesionales que desean ganancias inmediatas sin una reestructuración importante.

D.10.2. Consideraciones de Hardware

Objetivo de Hardware	Arquitecturas Re-comendadas	Notas
GPU (H100/A100)	Gated Softmax, FoX, Gated DeltaNet	Implementaciones maduras de softmax/lineal. GLA/DeltaNet requieren kernels personalizados.
TPU (alta memoria)	GLA, DeltaNet, Gated DeltaNet, HGRN2	Hardware soporta matmuls eficientes; los métodos recurrentes lineales brillan.
Móvil/Borde	RetNet, Gated Softmax ligero	Inferencia con memoria constante es crítica.

E. Annex E: Conceptual Introduction—Transformers and Attention for Beginners

Este apéndice proporciona una introducción accesible a los conceptos fundamentales de los transformers y el mecanismo de atención, dirigido a lectores sin conocimientos profundos previos en aprendizaje profundo. Los transformers son el motor detrás de los sistemas modernos de inteligencia artificial como ChatGPT, pero su funcionamiento puede entenderse mediante analogías del mundo real.

E.1. ¿Qué es un Transformer?

Un transformer es una arquitectura neuronal que procesa texto o secuencias de datos interpretando el significado de cada palabra mientras considera todas las demás palabras en contexto. Imagina que lees una oración:

“El banco estaba lleno de gente esperando. María fue al banco.”

¿Qué significa “banco” en cada caso? En la primera oración, se refiere a una institución financiera (o un asiento). En la segunda, no está claro sin contexto. Un transformer resuelve esto automáticamente al mirar todas las palabras circundantes. En la segunda oración, al ver “María fue”, el modelo comprende mejor que probablemente se trata de una ribera (donde va a nadar) o una institución financiera (donde va a realizar una transacción).

Esta capacidad de entender el significado completo de una palabra considerando toda la oración se llama *contextualización*, y es el corazón de cómo funcionan los transformers modernos.

E.2. El Mecanismo de Atención: Una Analogía

El mecanismo de atención es el “truco mágico” que permite a los transformers entender el contexto. Piensa en ello como participar en una conversación importante en una habitación ruidosa:

1. **Mucho ruido:** Alguien está hablando y es muy importante para ti.
2. **Ignoras el resto:** Tu cerebro automáticamente enfoca la atención en esa persona, ignorando otros sonidos.
3. **Entiendes mejor:** Al concentrarte, captas cada palabra claramente.

El mecanismo de atención neuronal funciona de la misma manera: cada palabra “pregunta” qué tan importante es cada otra palabra para entender su significado, luego “enfoca” su atención en las más relevantes.

E.3. Ejemplo Práctico Paso a Paso

Veamos cómo un transformer entiende la palabra “come” en dos contextos diferentes:

E.3.1. Contexto 1: “El gato come pescado”

El transformer, al procesar “come”, pregunta internamente:

- ¿Qué tan relevante es “El”? Poco (es solo un artículo). **Atención baja.**
- ¿Qué tan relevante es “gato”? Muy relevante (es el sujeto, quien come). **Atención alta.**
- ¿Qué tan relevante es “pescado”? Muy relevante (es lo que se come). **Atención alta.**

Por lo tanto, la representación mental de “come” se enfoca principalmente en “gato” y “pescado”.

E.3.2. Contexto 2: “El restaurante come los márgenes de beneficio”

Aquí el transformer pregunta en un contexto diferente:

- ¿Qué tan relevante es “restaurante”? Muy relevante (es el sujeto). **Atención alta.**
- ¿Qué tan relevante es “beneficio”? Muy relevante (se ve afectado). **Atención alta.**
- ¿Qué tan relevante es “márgenes”? Muy relevante (es lo que se consume). **Atención alta.**

La palabra “come” obtiene una representación completamente diferente porque “atiende” a palabras distintas en contextos distintos.

E.4. Cómo Funciona la Atención Matemáticamente (Versión Simple)

Detrás del cambio de atención hay matemáticas. Aquí está la versión simplificada sin demasiado detalle técnico:

E.4.1. Paso 1: Consultas, Claves y Valores

Cada palabra en la oración se transforma en tres versiones:

- **Consulta:** “¿Qué información necesito saber?”
- **Clave:** “¿Qué información tengo?”
- **Valor:** “Aquí está mi información importante.”

Imagina en una biblioteca, cada visitante es una “consulta”, cada libro es una “clave” y el contenido es el “valor”. El bibliotecario (atención) empareja las consultas con las claves más relevantes para acceder al valor correcto.

E.4.2. Paso 2: Compatibilidad

El sistema examina qué tan *compatible* es cada consulta con cada clave. Las consultas similares a las claves obtienen un puntaje de compatibilidad *alto*. Esto se calcula multiplicando la consulta por la clave (matemáticamente, el producto punto).

E.4.3. Paso 3: Enfoque

Los puntajes de compatibilidad se convierten en “pesos de enfoque”. Las compatibilidades altas significan “enfocar atención aquí”, y las bajas significan “ignorar esto”. Matemáticamente, una función llamada **softmax** convierte los puntajes en porcentajes (como: 40 % atención aquí, 35 % aquí, 25 % allá).

E.4.4. Paso 4: Combinar

Finalmente, el sistema combina todos los valores, ponderados por los pesos de enfoque. Si una palabra tiene un 80 % de atención en el sujeto, el 80 % de la información del sujeto se mezcla en la representación de esa palabra.

E.5. Múltiples Cabezas de Atención: Múltiples Perspectivas

Una característica clave es que los transformers no usan una única atención sino *múltiples cabezas de atención* en paralelo. Es como si tuvieras 8 personas analizando la oración *simultáneamente*, cada una prestando atención a diferentes aspectos:

- **Persona 1:** “Me enfoco en las relaciones sujeto-verbo.”
- **Persona 2:** “Busco el objeto directo.”
- **Persona 3:** “Sigo la información sobre el tiempo verbal.”
- **Persona 4:** “Busco modificadores y adjetivos.”

Cada “cabeza” aprende a enfocarse en diferentes patrones del lenguaje. Juntas, capturan una comprensión mucho más rica que una sola cabeza.

E.6. Apilar Capas

Los transformers procesan información en *múltiples capas* (generalmente de 12 a 48 para modelos prácticos). Imagina editar un ensayo:

1. **Primera pasada:** Corrige ortografía y gramática básica.
2. **Segunda pasada:** Mejoras la claridad y la estructura de las oraciones.
3. **Tercera pasada:** Aseguras consistencia y flujo narrativo.

Los transformers funcionan de la misma manera: cada capa refina la comprensión del texto. La primera capa captura características básicas (palabras simples, géneros), mientras que las capas posteriores entienden conceptos complejos (relaciones entre palabras, significados abstractos).

E.7. ¿Por Qué es Importante?

El mecanismo de atención fue revolucionario porque:

1. **Paralelismo:** Encuentra contexto para *cualquier palabra con cualquier otra palabra*, sin procesarlas secuencialmente (a diferencia de sistemas anteriores). Esto lo hace muy rápido.
2. **Flexibilidad:** Aprende qué patrones buscar automáticamente a partir de los datos, sin necesidad de programar reglas manualmente.
3. **Escalabilidad:** Funciona desde textos pequeños hasta contextos con millones de palabras.
4. **Capacidades generales:** El mismo mecanismo funciona para traducción, resumen, preguntas-respuestas, generación de texto, visión por computadora y más.

E.8. Desafíos Prácticos

A pesar del extraordinario éxito de los transformers, presentan desafíos:

E.8.1. Complejidad Computacional

Si una oración tiene 1,000 palabras, el mecanismo de atención estándar debe comparar cada palabra con las otras 1,000 palabras. Eso es $1,000 \times 1,000 = 1,000,000$ de comparaciones. Para documentos largos con millones de palabras, esto se vuelve prohibitivamente costoso en tiempo y memoria.

E.8.2. Requisitos de Memoria

Especialmente durante la inferencia (cuando se usan modelos entrenados), almacenar la matriz de atención completa puede consumir enormes cantidades de RAM o memoria de GPU, limitando las longitudes de secuencia que se pueden procesar.

E.8.3. Eficiencia del Dispositivo

Los transformers fueron diseñados para TPUs y GPUs potentes. Ejecutarlos en teléfonos o dispositivos integrados es un desafío.

E.9. Soluciones Modernas

Para resolver estos desafíos, la investigación ha propuesto muchas variantes:

- **Atención Dispersa:** En lugar de comparar cada palabra con TODAS las demás, compara solo con un subconjunto estratégico (vecinos cercanos, patrones periódicos). Reduce la complejidad de $\mathcal{O}(n^2)$ a $\mathcal{O}(n \log n)$ o $\mathcal{O}(n)$.
- **Modelos Recurrentes:** Como Mamba, incorporan aspectos de las antiguas redes neuronales recurrentes pero con mejor eficiencia moderna.
- **Atención con Compuerta:** Mecanismos que aprenden selectivamente qué información pasar hacia adelante, reduciendo las necesidades de almacenamiento.
- **Cuantización:** Usar números más pequeños (enteros en lugar de decimales) para reducir la memoria sin perder demasiada precisión.

Estas innovaciones son lo que este kit de herramientas (Frankenstein) te permite experimentar fácilmente.

E.10. Conclusiones Clave

- Un **transformer** es una red neuronal que entiende el contexto del lenguaje muy bien.
- El **mecanismo de atención** permite que cada palabra “se enfoque” en qué otras palabras son relevantes para entender su significado.
- La atención funciona calculando la **compatibilidad** entre cada palabra (consulta) y todas las demás (claves), luego combinando información basada en esa compatibilidad (valores).
- **Múltiples cabezas** de atención exploran diferentes patrones en paralelo.
- **Múltiples capas** refinan progresivamente la comprensión.

- El principal desafío es que la atención estándar tiene complejidad cuadrática, lo que es costoso para textos largos.
- Muchas **soluciones modernas** (atención dispersa, modelos recurrentes, cuantización) abordan estos desafíos, y este kit de herramientas te permite explorarlas todas.